UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Academic Master in Geomatic Engineering and Geoinformation
WEB DEVELOP AND GEOPORTALS

Guide of developed contents in lectures.

J. Gaspar Mora Navarro.

Univesitat Politècnica de València.
Departamento de Ingeniería Cartográfica, Geodesia y Fotogrametría Department of Cartographic Engineering and Photogrametry

Valencia, February 2026

WEB DEVELOP AND GEOPORTALES.

J. Gaspar Mora Navarro

6 de marzo de 2026

Índice general

Índice de figuras

CAPÍTULO 1
Evaluation

1.1 Evaluation

The evaluation consists in three parts, each of which is also divided in two parts, a written exam and a practical exercise. The dates of the exams are the published in the official site.

- Evaluation 1. Section 4.9, pag. 61. Total value 30 % = 3 points. (On 26 of March)
  - Theoretical written exam 2 points.
  - Practical exercise or project. 1 point. In this exercise you will create Python functions to connect with PostGIS and be able manage spatial data.
- Evaluation 2. Section 9.14, pag. 125. Total value 30 % = 3 points. (On 21 of May)
  - Theoretical written exam 2 points.
  - Practical exercise or project 1 point. In this exercise you will create a basic web, and will make requests to the server with Ajax. The request will execute your Python functions of the exercise 1, to manage the database spatial data.
- Evaluation 3. Section 10.9, pag. 140. Total value 40 % = 4 points. (On 16 of June)
  - Theoretical written exam 2 points.
  - Practical exercise or project 2 point. In this exercise you will add to your basic web, made in the exercise 2, a map that publishes your database spatial data. You will add the functionality of draw new elements on the map, and add them to the spatial database in the server.
- Last chance. All course content. On 26 June.

Practical exercises requirements:

- There are three practical exercises in total. The practical exercises must be related forming a geoportal project. The practical exercise 2 must continue the work done in the practical exercise 1, and the practical exercise 3 must continue the exercise 2. On finishing the practical exercise 3 you must have a geoportal with some minimum requirements. This is the project specified in the subject guide.
- The practical exercises can be done individually, in pairs or in groups of three people. You must form groups at the beginning of the course, inform the teacher of the group members. This groups can not be changed.
- The exercises are the ones which appears in this document. The delivery of the exercises consists in showing the exercise to the lecturer, and to answer some questions about them. Before this, the code of the exercise must have been uploaded to the shared space in Poliformat, otherwise you will lose the corresponding point or points. The exercises consist in coding functions. They have to work and you have to have a demonstration prepared, with proper data to run these functions.
- The written exams consist in individual online written exams, where you will be able to use any document, but not your mobile, nor AI, or Google. Usually you will have to type some small functions to demonstrate you master the subject contents. The functions will be similar, but not equal, to those developed in your practical exercises. The small differences are though to you demonstrate your understanding about important concepts.

- The three practical exercises must form a geoportal at the end of the course. The subject and data of the geoportal must be chosen by the student from the beginning, at the exercise 1. The geoportal has a minimum requirements.
- If you are doing the *Gestores de contenidos Geoespaciales y Smart Cities* subject, you will learn about web servers. You will have the opportunity of setup a web server to publish you geoportal, so anyone will be able to visit your it. This connect also *Desarrollo Web y Geoportales* subject with the subject *Gestores de contenidos Geoespaciales y Smart Cities*.
- The minimum number of tables to use, to make the exercises, and the geoportal are three, and all of them must have a geometry column of different geometry type (point, polygon or linestring).
- The geoportal will be showed to the teacher, and will be evaluated at that moment. If the geoportal does not work you will not get any score.

CAPÍTULO 2

Previous knowledge

2.1 Introduction

A geoportal y composed by may parts. In the Figura 2.1 you can find a schema:

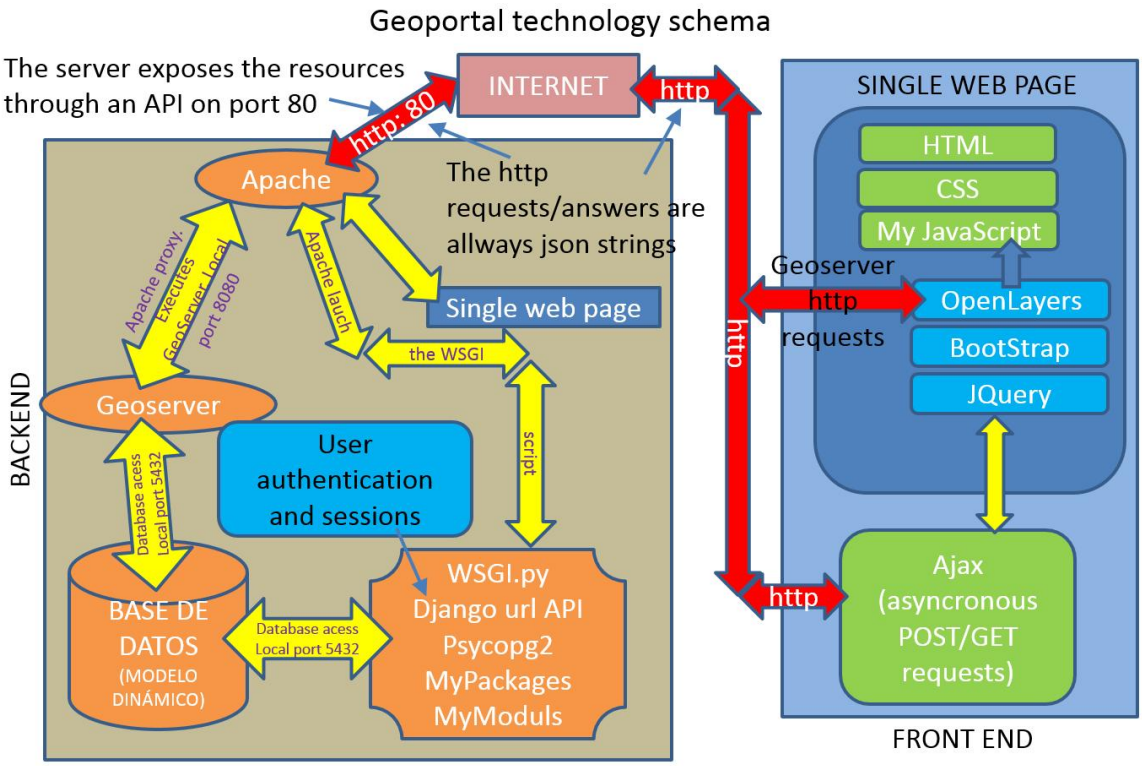

Figura 2.1: Geoportal parts schema

As you can see at first sight it is not a simple task to build a geoportal. Is is composed by many parts and each part is composed by many parts, each of which needs a good level of developing skills. Its a too big task for one solely subject. The goal of the subject is to see all the parts, so that the student will have a good overview of how a geoportal works. Of course we can not see each part in deep, but knowing how is the philosophy the student will be able to go more deep if he needs it. The good thing is that once the geoportal is built all the user have access and all of them are working with the same data, so the editions of one users are inmediatelly seen by the rest of the users.

The teacher will explain how to create a basic geoportal to add polygons to a database. To pass this subject you will have to do the same with your own tables. What the teacher is expecting about you, as minimum, is you understand the examples and you be able to modify them for your own project. These tables will come from the subject *Distribución de la Información Espacial*, in order to continue your work about INSPIRE.

To learn we are going to build a simple geoportal, but which contains all the parts showed in the schema. You can access to the geoportal with the url *https://gisserver.car.upv.es/desweb/*.

You can form groups of one, two or three people. I recommend you groups of two people, because it is good for you to be forced to join code from others. Also it is good to work in group because you can share knowledge.

You have to keep in mind that 6 out of 10 points of this subject are in written tests where you are alone, so you have to understand all the code of your project or exercises.

2.2 Software and libraries

2.2.1 Linux software

To follow the course you need the following software:

- Python 3.*. It cams preinstalled in Linux, as is part of the core system.
- git
- Docker. Docker desktop is not available en Linux. To install Docker in Linux you can use the following script:

```
#bin/bash
for pkg in docker.io docker-doc docker-compose docker-compose-v2
    podman-docker containerd runc; do sudo apt-get remove $pkg
   -y; done

# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
   /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/
     keyrings/docker.asc] https://download.docker.com/linux/
     ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io
   docker-buildx-plugin docker-compose-plugin
```

- Visual Studio Code, with the following extensions:
  - Docker (Microsoft).
  - Dev Containers (Microsoft).
  - Python (Microsoft).
  - Pylance (Microsoft).
  - Python Debugger (Microsoft).
  - WSL (Microsoft).

2.2.2 Windows software

To follow the course you need the following software:

- Python 3.*. It must be accessible from the terminal from any path by typing *python*.
- git
- Docker Desktop
- Visual Studio Code, with the following extensions:
  - Docker (Microsoft).
  - Dev Containers (Microsoft).
  - Python (Microsoft).
  - Pylance (Microsoft).
  - Python Debugger (Microsoft).
  - WSL (Microsoft).

2.3 Python necessary previous knowledge

2.3.1 Python venvs

> There is a caveat here though: we can only install a single version of a package at any one time. While this isn't a problem when you create your first project, when you create the second any changes you make to the dependencies will also affect the first project. If any of your dependencies depend on other packages themselves (usually the case!) you may encounter conflicts where fixing the dependencies for one project breaks the dependencies for another.
>
> This is known as dependency hell.
>
> Thankfully, Python has a solution for this: Python virtual environments. Using virtual environments you can manage the packages for each project independently.
>
> In this tutorial, we will learn how to create virtual environments and use them to manage our Python projects and their dependencies. We will also learn why virtual environments are an essential tool in any Python developer's arsenal[(1)].

So all Python projects have to have its own Pyhton venv. To create a venv and isolate the packages type the following (Windows and Linux). Create a folder called c:/pythonProjects. Inside cretate the folder poo. Inside the folder c:/pythonProjects/poo type the following commands:

```
python -m venv ./venv
```

The venv is a folder called venv, but can be any other name.

To use the venv you must activate it. Linux:

```
source venv/bin/activate
(venv) project/

PS> venv\Scripts\activate
(venv) PS C:\Users\gaspa\temp>
```

[(1)] source:https://www.pythonguis.com/tutorials/python-virtual-environments/

Now you can use python command or pip to install-uninstall packages. To deactivate the venv. Linux:

```
deactivate
```

Windows:

```
exit
```

2.3.2 Install the Python necessary packages in a venv

Put the following in a textfile called requirements.txt:

```
asgiref==3.8.1
click==8.1.7
Django==5.0.4
gunicorn==22.0.0
h11==0.14.0
packaging==24.0
psycopg2==2.9.9
sqlparse==0.5.0
typing_extensions==4.11.0
requests==2.32.3
django-cors-headers==4.6.0
djangorestframework==3.15.2
six==1.17.0
debugpy==1.8.11
cryptography==44.0.0
django-rest-knox==5.0.2
pgOperations==0.0.3
Datetime==5.5
django-filter==24.3
drf-access-policy==1.5.0
```

Note all packages have an specific version. This is recommended.

With the venv activated type the following:

```
(venv) PS C:\temp> pip install -r requirements.txt
```

That command will install all packages at once.

To get the list of packages of a venv, in a text file, you can use the following command:

```
(venv) PS C:\temp> pip freeze > requirements.txt
```

2.3.3 How to import Python modules

Open the folder c:/pythonProjects/poo with VS and make your tests there. VS will automatically use your *venv* Python virtual envionment.

You are used to make scripts in a single file. In web developing that approach is not possible. A minimum geoportal is a very big program. You will need to divide the code in files, according a logic. The first step is to be able to create python modules and packages and know how to load them, sou that you can divide the code in small parts.

It is very important you understand the Python import system. In the following, the current folder is the folder that contains the current file, and the current file is the file that is currently being executed, imported or edited:

- A Python module is a text file, that is a file with the .py extension.

- A package is a folder with several python modules, and a special empty file called *__init__.py*. **Without the file *__init__.py*, any module could be imported. A folder with modules, but without the file *__init__.py* is a normal folder, not a package, so python will not be able to import any module there**.

- When you type *import a* Python searches for the module *a* in the current folder, if it is not, searches for the *a* package, also in the current folder, if it is not, searches for the *a.py* in the *sys.path* folders list, if it is not, searches for the *a* package in the *sys.path* folders list. Yu can see the sys.path folders list with the following script:

```
import sys
print(sys.path)
```

- If you execute a file called myAwesomeProgram.py with python3 myAwesomeProgram.py, the path of the myAwesomeProgram.py is added to the sys.path folders list, so any python module in the same place is going to be found with import.

- In all the python modules, in all the functions is available the variavle __file__. It contains the file name. For example for the module *a.py*, the value of the variable is *a.py*.

- __file__ is commonly used to the absolute path of the current file name. To get the absolute path have to use the function os.path.abspath(__file__). For example the function could return */home/vagrant/apps/desweb/a.py*.

- The function *os.path.dirname()* returns the folder containing the file or folder. For example. If you are editing the file /home/vagrant/apps/desweb/a.py.

- __file__, os.path.abspath, and os.path.dirname() are commonly used to navigate back in the path to be able to import modules backwards the current module folder. The next script shows how it works:

```
import os
print(os.path.abspath(__file__)) #print /home/vagrant/apps/
    desweb/a.py
fileName=os.path.abspath(__file__)
print(os.path.dirname(fileName)) #print /home/vagrant/apps/
    desweb
print(os.path.dirname(os.path.dirname(fileName))) #print /home/
    vagrant/apps
print(os.path.dirname(os.path.dirname(os.path.dirname(fileName))
    )) #print /home/vagrant
```

If you want to import the module *m1.py* inside the package *p1*, which is inside the current folder you can do it easily:

```
from p1 import m1
```

But if you want to import the module *padre* in /home/vagrant/apps/p1/padre.py, and the current file is /home/vagrant/apps/desweb/p2/master.py, you must to add /home/vagrant/apps/desweb to the sys.path list **before to try the import**. Of curse you could use absolute paths:

```
import sys
sys.path.append("/home/vagrant/apps/desweb")
from p1 import padre
```

The above code will work, but **it is a very beginner mistake**. You could not move your project, because the absolute path will change.

You must **always use relative paths** to the location of the current file, stored in the variable __file__. You must upload one level in the folder that contains the current file and add that folder to the sys.path list.

```
import os, sys
print(sys.path)
print(__file__)
BASE_DIR= os.path.dirname(os.path.dirname(os.path.abspath(__file__))
    )
sys.path.append(BASE_DIR)
print(sys.path)
from p1 import padre
print("padre imported")
```

The above script has the following result:

```
["/home/vagrant/apps/desweb/conf", "/home/vagrant/apps/env/lib/
    python3.6", "/home/vagrant/apps/env/lib/python3.6/lib-dynload",
    "/usr/lib/python3.6", "/home/vagrant/apps/env/lib/python3.6/site-
    packages", "/home/vagrant/apps/env/lib/python36.zip"]
/home/vagrant/apps/desweb/conf/m2.py
["/home/vagrant/apps/desweb/conf", "/home/vagrant/apps/env/lib/
    python3.6", "/home/vagrant/apps/env/lib/python3.6/lib-dynload",
    "/usr/lib/python3.6", "/home/vagrant/apps/env/lib/python3.6/site-
    packages", "/home/vagrant/apps/env/lib/python36.zip", "/home/
    vagrant/apps/desweb"]
padre imported
```

In the above listing we added the folder */home/vagrant/apps/desweb* to the list sys.path, so any module in that folder, if __init__.py exists, wil be available to import.

In our case, we will import allways the same way: with the relative path from the project folder. For example, having the following project modules:

```
/home/user/pythonProjects/project1/main.py      /**main file of the
    project**/
/home/user/pythonProjects/project1/lib1/a.py
/home/user/pythonProjects/project1/lib2/b.py
```

We execute the project from /home/user/pythonProjects/project1/ with the command:

```
python3 main.py
```

So python is going to add the folder /home/user/pythonProjects/project1/ (the project folder) to sys.path, and any module inside that folder is going to be found with relative paths to the project folder:

- To import a.py from main.py:

  ```
  from lib1 import a
  ```

- To import b.py from a.py:

  ```
  from lib2 import b
  ```

Something you have to understand: *from lib2 import b*, is going to work as long as you execute *python3 main.py*, and from here you first import *a*, and from *a* you can import *b*. But if you execute *a*,

from *lib2 import b* is going to fail. The reason is because the folder /home/user/pythonProjects/project1/ is not going to be added to sys.path, but /home/user/pythonProjects/project1/lib1, and the sentence *from lib2 import b*, is going to search*lib2* inside *lib1*, and it is not.

2.3.4 Object Oriented Programming (OOP)

To be able to use the frameworks we are going to use Django, Django rest Framework, and Angular, it is necessary an advanced knoledge of Object Oriented Progamming approach.

Algorithms are the same in lineal coding than in OOP, the difference is the organization of the code. You have to learn some concepts, and think in a different way, but with some practice you can get used.

The minimum concepts you have to learn are:

- How create classes.
- What is, and how to use a constructor (__init__).
- What is *self*, that in other languages is called *this*.
- What are class class variables
- What are instance variables.
- Differences between class variables and instance variables.
- How create methods, and the difference between public and private methods.
- How to initialize the class variables from the constructor.
- How to access to the class variables from any class method.
- Difference between class variables and local variables.
- How inherit from other class and what this means.
- What is a father class, or super class, and what is a child class.
- How to initialize the constructor of the father class from the constructor of the child class, and why you have to do it.
- How to overwrite a father class method from the child class, and what is this useful for.

The most important utilities are:

- Classes can check the data before to be stored in the class, in the constructor or in the setter method.
- All class variables are accessible from all methods and:
  - You also avoid passing parameters to the methods.
  - You know all class variables have been checked and transformed to be used in all methods.
- Classes can transform the data in different ways when we want to retrieve it, in the getters. So you can store the class variables in a way and retrieve them in one other, or others, ways.

2.3.4.1. Create a class

Open the folder c:/pythonProjects/poo with VS and make your tests there. VS will automatically use your *venv* Python virtual envionment.

A class is a template of objects all of them do the same, and have the same properties, but the difference between them is that the properties can be different. For example the class Point, can have the properties x, y and each point have different coordinates, but all points have the same properties x, y. It is a template. Also the Point class can have a method to translate the point, and all points will have the same method:

```
class Point:
    def __init__(self, x, y):
        self.x=x
        self.y=y
    def translate(self, transX, transY):
        self.x=self.x+transX
        self.y=self.y+transY
```

To use this class:

```
pt1=Point(2,2)
pt2=Point(3,3)
pt1.translate(100,100)
pt2.translate(200,200)
pt1.x  --> 102
pt2.y  --> 203
```

As you can see, all points have the same structure: properties x, y and the method translate.
To create a class:

```
class Animal:
    # This is a class with no methods or properties yet.
    pass
```

2.3.4.2. Create a class constructor

The constructor is a special method called when an object is created. Usually it needs parameters used to initialize the class variables. The good think is that you can check the validity of the constructor parameters and deny invalid values. Example:

```
class Dog:
    def __init__(self, name, age):
            #CONSTRUCTOR
        # Instance variables (attributes) are defined in the
           constructor.
        # self.name, and self.age are instance variables

        # Here you can check if the name and the age are valid
           values, or generate an error.
            # This ensures that all the instance variables are ok
               and can be used by the methods
        self.name = name
        if age < 0 or age > 100:
                raise Exception("Age should be > 0 and < 100")

        self.age = age
```

The previous code can be improved by using a setter method. A setter method is a common method used by set an instance variable, and their name starts by *set*.

It is better to define a setter for the *age* property, in order to be able to change the age of the instance at any moment, and the new value is also checked:

```
class Dog:
    def __init__(self, name, age):

        # Instance variables (attributes) are defined in the
           constructor.
        # self.name, and self.age are instance variables

        # Here you can check if the name and the age are valid
           values, or generate an error.
            # This ensures that all the instance variables are ok
                and can be used by the methods
        self.name = name
        self.setAge(age)

    def setAge(self, age):
        #METHOD
        age=float(age)#converts to float or error if the value is
           not a number, or a valid text
        if age < 0 or age > 100:
            raise Exception("Age should be > 0 and < 100")
        self.age = age
```

Use example:

```
dog1=Dog('a',1)
dog2=Dog('b',2)
dog2.setAge(3)
dog.age --> 3
```

2.3.4.3. What is *self* in a Python class

self represents the object instance to access to the instance methods and properties. Properties are class variables, and instance variables. *self* is available everywhere in the code. It allows to access to the properties and the methods;

```
#To access to the property name:
    p = self.property
#To execute a mehtod:
    self.methodName(param1, param2,...)  <-parameters exclude self
       in method calls.
```

self is a parameter that all methods have to receive in its definition, but it is a parameter that you never have to pass to the methods: Eg.:

```
#Method definition:
    def setAge(self, age):   <-- self is in the definition. Always
       first parameter
#Method use:
    self.setAge(age):   <-- self is not sent as parameter to the
       method
```

2.3.4.4. Class variables and instance variables

This is quite difficult to understand, but important to not get lost in some situations, more offend that you think.

There are three differences:

- Class variables are defined outside, and above the constructor.
- To change the class variable value you must use the class name ClassName.classVariable=newValue.
- A class variable is a variable which value is shared by all the instances of the class. Any change (ClassName.classVariable=newValue) **changes the value of the class variable in all instances at once**.
- An instance variable can be different in all the instances, and to change a instance variable value, you must use the instance, not the class name: p1=Point(1,1); p2=Point(8,8). *p1* and *p2* are the instances. Point is the class name. p1.x=2 changes the x property, in the instance *p1* only. Point.type='Point2d' changes the class variable in all the instances of Point, *p1* ans *p2*.

See an example:

```
class Bird:
    species = "Bird"  # This is a class variable shared by all
       instances.
    def __init__(self, name):
        self.name = name  # This is an instance variable, unique to
           each instance.

bird1 = Bird("Parrot")
bird2 = Bird("Sparrow")

#print the class variable inboth instances
print(bird1.species)  # Output: Bird
print(bird2.species)  # Output: Bird

# Change the class variable
Bird.species = "dog"
#the class variable is changed for al the Bird instances
print(bird1.species)   # Output: dog
print(bird2.species)   # Output: dog

#the instance variables can be different in each instance
print(bird1.name)   # Output: Parrot
print(bird2.name)   # Output: Sparrow
```

Important: If you try to change the class variable in the following way: instance1.classVariable=newValue unexpected things are going to happen:

- Python is going to create a new instance variable, only in the *instance1*, with the same name that the class variable. So it can be different in any instance, as any other instance variable.
- The class variable is not going to be accessible any more in the instance variable *instance1*.

Example:

```
class Bird:
    species = "Bird"  # This is a class variable shared by all
       instances.
    def __init__(self, name):
        self.name = name  # This is an instance variable, unique to
           each instance.

bird1 = Bird("Parrot")
bird2 = Bird("Sparrow")

#print the class variable inboth instances
print(bird1.species)  # Output: Bird
print(bird2.species)  # Output: Bird

# Change the class variable
bird1.species = "dog" # <--NEW INSTANCE VARIABLE WITH THE SAME NAME
#the class variable is changed for al the Bird instances
print(bird1.species)   # Output: dog # <--The instance variable is
   returned
print(bird2.species)   # Output: Bird

Bird.species = "Fish"
print(bird1.species)   # Output: dog  # <--The instance variable is
   returned. The class variable is not accessible any more
print(bird2.species)   # Output: Fish # <--The instance variable has
    changed
```

2.3.4.5. Class methods

Methods are functions defined within a class that operate on its instance variables. The first parameter of any method must be the word *self*. The method calls must exclude the parameter *self*. Example:

```
class Cat:
    def __init__(self, name):
        self.name = name

    def speak(self): <--self is the first parameter. In this case is
        not necessary any other parameter
        return f"{self.name} says Meow!"

c1=Cat("Misyfuzz")
c1.speak()  # <-- self is not passed to the method call. Output:
   Misyfuzz says Meow!
```

2.3.4.6. Local variables

Local variables are those defined inside a method, even the constructor, but without self. By the way if you use self to define a variable (self.a=25), a is an instance variable. By convenience all the instance variables are defined in the constructor, but can be defined in any place. Be careful with this.

Local variables are accessible only in the function or method they are defined.

```
class Example:
    class_var = "I am a class variable"

    def __init__(self):
        self.instance_var = "I am an instance variable"
```

```
    def method1(self):
        local_var = "I am a local variable"
        print(local_var)  # Accessing a local variable

    def method2(self):
        local_var = "I am different" #this is a local variable with
           the same name,
                                     #that the one in method1,
                                        but different as it is
                                     #in a different method or
                                        scope
        print(local_var)  # Accessing a local variable

    def thisCausesError(self):
            print(local_var)  # local_var is not in this scope. Not
               defined. It will trigger an error
```

2.3.4.7. Inheritance

Inheritance means that one class can inherit everything of other. The inherited class is called *child* class, and the original class is called *father*.

Why to inherit from an other class?:

- Because the father class do not do something new you need. You need to extend the father class. In this case you inherit, initialize the father constructor, from the child constructor, and define the new methods and properties that you need to add.
- Because the father class do something but you need to change it. That is, the father has a method that you need to redefine, *override*, in order to do something different. This is called overload.

Inheritance to extend the father class: Inheritance example to extend the properties that the father class has:

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):  # Car inherits from Vehicle
    def __init__(self, brand, model):
        super().__init__(brand)  # Initialize the parent class. VERY
            IMPORTANT
        self.model = model
```

Inheritance to override a method of a father class:

```
class Parent:
    def __init__(self, name):
        self name=name
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def __init__(self, name, name2):
        super().__init__(name)
        self.name2=name2
```

```
    def greet(self, name, name2):  # Overriding the greet method
        return "Hello from Child"
```

2.3.5 OOP example. Point operations

In this example you are going to practice many of the OOP knowledge learned in the previous section.

2.3.5.1. Create a class

- Create a folder called *c:/pythonProjects*.
- Inside *c:/pythonProjects* create a folder called *pointOperations*.
- Inside *c:/pythonProjects/points*, create a venv called venv, activate it, and install all the libraries in the file *requirements.txt*, listed in the section 2.3.2, page 8.
- Create a Python package, called *pointOperations*, and and create the modules point2d.py, point3d.py, and pointOperations.py (remember create the file __init__.py to convert the folder in a package, otherwise you could not import these modules).
- In the point2d.py module, create a Python class called Point2d:
  - Create a class variable called pointType, and set the value to "Point2d".
  - Create a constructor that receive the x, and y coordinates, and the point number. The constructor must create the number and x, y instance variables.
  - Check if the x and y are numbers. Check if they are strings and if they can be converted to numbers. If they can be converted, convert them to numbers, or raise an exception.
  - Raise an exception if any of the coordinates are negative.
  - To change the coordinates of the point create setters for the n, x, y properties.
  - Create a method called getAsList, that return a list with the n, x, y coordinates: [n,x,y]
  - Create a method called getAsDict, that returns a dict; {'n':n, 'x':x, 'y':y}
  - Create a method called printCoordinates that prints the coordinates.

2.3.5.2. Think in the OOP way

Now imagine you need the following funcions:

sumXY : Receives a list of 2d points [[X1,Y1], [X2,Y2],...] and calculates the sum of all the X and all the Y. Returns a list [sumX, sumY].

geocenterXY : Receives a list of 2d points [[X1,Y1], [X2,Y2],...], and returns the a list with [averageX, averageY]. You must use sumXY in this function.

maxXY : Receives a list of 2d points [[X1,Y1], [X2,Y2],...] and returns a list with [maxX,maxY].

minXY : Receives a list of 2d points [[X1,Y1], [X2,Y2],...] and returns a list with [minX,minY].

traslationXY : Receives a list of 2d points [[X1,Y1], [X2,Y2],...] , and other list with two numbers [dX, dY]. The function returns a list of points moved dx and dy: [[X1+dX, Y1+dY], [X2+dX,Y2+dY],...].

Now you know OOP, so you can design the previous functionalities in a more clever way:

- Instead of using list of coordinates [[X1,Y1], [X2,Y2],...], you can use lists of Point2d instances [p1,p2,p3,...]. The advantage is evident, on create the Point2d instances, you are sure the coordinates are ok and also, you can get the coordinates in several ways.
- If you realize, all the previous functions receive a list of 2d coordinates. This common arguments between functions are always suitable to be in a class as properties. This means that you can use a class with a property to store a list of points, and you can get the same functionality by creating methods,that do not receive anything, they access to the instance variable that stores the list of points ans can apply the same functionality.

Create the module Point2dListOperations.py and create a class with methods to achieve the same functionalities of the previous functions, according using list of Points2d, and receiving the list in the constructor. Check if the received parameter is a list, is the list has more than 0 elements, and if the first element is a Point2d instance. Raise an exception if any of the conditions is not accomplished.

Create a module called test.py. Import the Point2d, and the Point2dListOperations classes. Manually create 4 points, create a list of those 4 points. Create an instance of the Point2dListOperations class, and execute its methods.

Check if the constructor accept any invalid value.

2.3.5.3. Inherit from a class

To practice with inheritance, create a class called Point3d that:

- Inherits from Point2D,
- Extends its properties by creating the z coordinate
- Overrides all its methods to consider the Z coordinate

Create other module, and try the class by creating a Point3d instance and checking all its methods.

2.3.5.4. OOP proposal exercise

Create a class called *CentesimalAzimut*, with the public class property *centesimalAzimut*. The constructor receives a float , the azimut, in centesimal mode. The constructor makes the following angle checks:

- If the centesimalAzimut is bigger than 800 the program could calculate wrong values so you should raise an error, with a proper message.
- If the centesimalAzimut is less than 0, sums 400 to the angle.
- If the centesimalAzimut is bigger than 400 then centesimalAzimut=centesimalAzimut-400.
- In none of the previous cases occur, the centesimalAzimut values is correct and must remain unchanged.

All those checks can be in the constructor, or in a public method that sets the proper value of the azimut in the class variable. Better call a public method called *setCentesimalAzimut*, that receives the angle passed to the constructor make the prevous checks, and sets proper azimut value in the class variable centesimalAzimut. I recommend make the setCentesimalAzimut method public because, in that way can be used to change the azimut of the object, at any moment. That is, if I have ca=CentesimalAzimut(560), latter I can change its azimut: ca.setCentesimalAzimut(230), and the new azimut for the *ca* object will be also checked.

Create the following public class methods: getAsSexagesimal, getAsRadians and getAsCentesimal, to return the azimut in several formats:

- The method getAsCentesimal simply returns the class property centesimalAzimut.
- The method getAsSexagesimal gets the centesimalAzimut property multiplies by 360 and divides by 400, and returns the result.
- The method getAsRadians gets the centesimalAzimut property multiplies by math.PI and divides by 200, and returns the result.

I have to remark that the centesimalAzimut property must remain unchanged after the initialization, and the public methods simply transform the value internally and return the value that the user requested. You must aware that, if you change the centesimalAzimut value from any of the methods, the second time you use any of the methods will return a wrong value.

Create the class Observation2d, that inherits from CentesimalAzimut, and adds the public property distance2d. The constructor receives the azimut, and the distance, both supposedly observed from a base point to a measured point with a total station. The idea is to have the CentesimalAzimut functionalities to check the observed azimuth, and add some more checks to the distance, all in the same class. The idea of inheritance is to be enlarging functionality in inherited classes.

The child class Observation2d initializes the father class, CentesimalAzimut, with the azimut received by the constructor of Observation2d. Observation2d also receives a distance in its constructor. You must perform also some checks: if the distance is bigger than 0, is correct, and initializes the distance. In case the distance be less than 0 you should raise an error. Again you can implement this distance checks directly in the constructor, or outside in a private method. This time that is up to you.

Create a public method called getDistance, which returns the distance property.

Create a public method called getDistanceWithOffset, that receives an offset, and returns the distance plus the offset. If there is a systematical error in the distance, positive or negative, with the offset you can get the proper distance value. I have to recognize that this is a very strange case, but it is useful to you to know that class methods can receive extra values, to make the class method work.

Why do not put the offset variable in the Observation2d constructor, and store it as a class property or variable?. Good question. It is normal at the beginning not to know where is the best place to put the things. It is a matter of practice. To create a class variable, instead of require a class method parameter, you must the following two criteria:

- The property must be required for all class methods. That is, it is a common required data for all methods.
- The property must be frequently used.

In this case the class Observation2d has three inherited methods and his own two ones, getDistance, and getDistanceWithOffset. So one out of five require the offset. First condition is not accomplished. But also the offset is going to be rarely used. So make sense not use it as property, and use it as parameter to get the corrected distance value with the getDistanceWithOffset method, if required.

Therefore, if you do not add the offset as class property, the initialization of the class will receive two parameters. e.g: o = Observation2d(distance=200, azimut=150).

But if you add the offset as class property, the initialization of the class will need three parameters: o = Observation2d(distance=200, azimut=150, offset=0 (or 0.05, or 0.025, whatever value)), although you can set the offset as optional value.

Create the class Radia2d. That class do not inherits anything from the other classes, because it is not going to extend their functionality, but is going to use objects of previous classes, in fact is going to use a Point2d object, and an Observation2d object. Define three public class variables outside of

the constructor, above, and set them to *None*: base: Point2d, obs: Observation2d and radiatedPt2d: Point2d.

Define the constructor that receives two parameters base: Point2d, obs: Observation2d. The constructor immediately initializes the class variables *base* and *obs*. To initialize the radiatedPt2d yo must calculate the x, and y coordinates and create a Point2d object, and set the radiatedPt2d to that object. I recommend you to do that in a private method, outside the constructor, e.g: _setRadiatedPt2d.

Note: Methods that gets property values are usually named getPropertyName. Methods that sets property values are commonly named setPropertyName. They are also named as *getters* and *setters* methods.

Define the following methods to get the results: getAsPoint2d (returns the radiated point), getAsList (returns the coordinates of the radiated point as list [x, y]), getAsDict(returns the resulting point as dict), printPoint (prints the radiated point in list form).

2.3.5.5. OOP proposal exercise solution

Here is the solution:

```
import math, typing
from pointOperationsObj import Point2d

PI = math.pi#global variable

class CentesimalAzimut():
    def __init__(self,centesimalAzimut:float):#constructor
        self.centesimalAzimut: float = None
        self.setCentesimalAzimut(centesimalAzimut)#setter de la
           propiedad centesimalAzimut

    def setCentesimalAzimut(self,value:float):
        """
        Setter de centesimalAzimut. Establece el valor correcto de
           la propiedad
        """
        if value > 800:
            raise Exception("Angulo mayor de 800")
        if float(value) < 0:
            self.centesimalAzimut = value+400
        else:
            if float(value) > 400:
                self.centesimalAzimut = value-400
            else:
                self.centesimalAzimut = value
    #getters
    def getAsSexagesimal(self):
        return (self.centesimalAzimut*180)/200

    def getAsRadians(self):
        return (self.centesimalAzimut*PI)/200

    def getAsCentesimal(self):
        return self.centesimalAzimut

class Observation2d(CentesimalAzimut):
    def __init__(self,centesimalAzimut:float,distance:float):
```

```
        self.distance2d : float = None
        CentesimalAzimut.__init__(self, centesimalAzimut)
        self._checkPositiveDistance(distance)
        self.distance2d = float(distance)

    def _checkPositiveDistance(self,value:float):
        if float(value) <= 0:
            mess="Distance must be bigger than 0"
            raise Exception(mess)

    def getDistance(self):
        return self.distance2d

    def getDistanceWithOffset(self,offset:float = 0):
        return self.distance2d+float(offset)

class Radia2d():


    def __init__(self,base:Point2d,obs:Observation2d):
        self.base:Point2d=None
        self.obs:Observation2d=None
        self.radiatedPt2d:Point2d=None

        self.base = base
        self.obs = obs
        self.setRadiatedPt2d()

    def setRadiatedPt2d(self):
        x=self.base.x + self.obs.distance2d*math.sin(self.obs.
            getAsRadians())
        y=self.base.y + self.obs.distance2d*math.cos(self.obs.
            getAsRadians())
        self.radiatedPt2d=Point2d(x,y)

    def getAsPoint2d(self):
        return self.radiatedPt2d
    def getAsList(self):
        return self.radiatedPt2d.getXYAsList()
    def getAsDict(self):
        return self.radiatedPt2d.getAsDict()
    def printPoint(self):
        self.radiatedPt2d.printAsList()

if __name__=="__main__":
    #Comprobacion de Observation2d
    obs = Observation2d(centesimalAzimut=500, distance=200)
    print("Angulo en centesimal: {0}".format(obs.getAsCentesimal()))
    print("Distancia: {0}".format(obs.getDistance()))
    print("Distancia con -0.025 de offset: {0}".format(obs.
        getDistanceWithOffset(offset=-0.025)))

    #Radiacion de un punto
    ptBase=Point2d(100,100)
```

```
rd=Radia2d(base=ptBase, obs=obs)
print("Radiacion primer punto:")
rd.printPoint()

ptBase2=Point2d(800,800)
obs2=Observation2d(centesimalAzimut=300, distance=450)
rd2=Radia2d(base=ptBase2, obs=obs2)
print("Radiacion segundo punto:")
rd2.printPoint()
```

The result of the before program is:

```
Angulo en centesimal: 100
Distancia: 200.0
Distancia con -0.025 de offset: 199.975
Radiación primer punto:
[300.0, 100.00000000000001, -1]
Radiación segundo punto:
[350.0, 799.9999999999999, -1]
```

2.4 Python string formatting

In this section you are going to learn an utility that you are going to use on making sql sentences dynamically. It is a way to substitute variable values inside a string. Very useful to introduce table names and field names into the sql sentences. Python give as two ways to to this:

2.4.1 Formatting a string using the order of the variables

In this way you put numbers between curly brackets into the string, an later you specify the variable values in order:

```
query1 = "select {0} from {1}".format("gid, description,st_area(geom
    )", "d.buildings")
print(query1) #print  --> select gid, description,st_area(geom) from
     d.buildings
```

2.4.2 Formatting a string using names in the formatted string

In this way you put names between curly brackets into the string, an later you specify the variable names and values:

```
query2 = "select {fieldNames} from {tableName}".format(fieldNames="
    gid, description,st_area(geom)", tableName="d.buildings")
print(query2) #print  --> select gid, description,st_area(geom) from
     d.buildings
```

2.5 Managing JSON strings

2.5.1 Communication client - server

A server only receives and send strings. It is widely used JSON strings to send data from the server to the user, and from the user to the server. Usually the server sends to the user json strings containing table rows and the user sends json strings containing form user data.

```
COMMUNICATION CLIENT --> SERVER

CLIENT: html form --> javascript --> json --> send to the server
    ||Internet|| SERVER: python WSGI --> json decode --> python
       dictionary --> send to the database

COMMUNICATION SERVER --> CLIENT
DATABASE --> PYTHON WSGI --> Python dictionary --> json --> send to
   the client
    ||Internet|| CLIENT: json --> js object --> html form
```

A json string is a string that contains fields names and values. It looks like a Python dictionary but wrapped by simple quotes. Json strings can easily be transformed into Python dictionaries and vice versa. In the following listing you can see a json string example. Pay attention in the external quotes and the internal quotes format.

```
'{"gid":"10","descripcion":"Hola","id_trabajo":"1","z_tapa":"10","
   profundidad":"10","diametro":"10","type":"Point",
"coordinates":"727763.05556,4372987.48466"}'
```

2.5.2 Creating JSON strings with Python

In the following listing you can find an example of how create a json string with Pyhton.

```
# -*- coding: utf-8 -*-

#imports the json librari to code and decode json strings
import json

#creates the dictionary d
d = {}

#introduces three keys and values
d["campo1"]="valor1"
d["campo2"]="valor2"
d["campo3"]="valor3"

#converts the dictionary to a json string
d_json = json.dumps(d)
print("First json: {0}".format(d_json))

#creates other dictionary
d2={}
#introduces some keys and values
d2["ok"]=True
d2["menssage"]="diccionariy 2"
#introduces the other json string as a value in this dictionary
d2["data"]=d
```

```
d2_json = json.dumps(d2)

print("Second json: {0}".format(d2_json))
```

The result is the following

```
First json: '{"campo1": "valor1", "campo2": "valor2", "campo3": "
    valor3"}'
Second json: '{"message": "diccionariy 2", "ok": true, "data": {"
    campo1": "valor1", "campo2": "valor2", "campo3": "valor3"}}'
```

As you can see in the in the second json of the above listing, you can put a dictionary into a dictionary an generate a json. This is what usually is done. In the second json there is a message, a variable to know if the request was *ok*, and a field called *data*. The data field contains, in this case, a row, but can contain several rows.

2.5.3 Decode a JSON string to obtain a Python dictionary

In the following listing you can find an example of how decode a json string with Pyhton.

```
# -*- coding: utf-8 -*-

#imports the json librari to code and decode json strings
import json

#the json string to decode
strJson = '{"message": "diccionariy 2", "ok": true, "data": {"campo1
    ": "valor1", "campo2": "valor2", "campo3": "valor3"}}'
#decodes the json string and creates a dictionary
d=json.loads(strJson)

#prints the keys of the dictionary
print(d.keys())
#print the values of the dictionary
print(d.values())
```

The result is the following. Pay attention in that the value for the *data* key is an other dictionary. That means that you can put several dictionaries inside other dictionaries, obtain a json string, and decoding the json string only once, you can retrieve all the dictionaries.

```
dict_keys(['message', 'ok', 'data'])
dict_values(['diccionariy 2', True, {'campo1': 'valor1', 'campo2': '
    valor2', 'campo3': 'valor3'}])
```

2.5.4 Decode a JSON string to obtain a JavaScript object

If a json string is received in the client, with JavaScript it is possible to convert it into an object. In the following code, a example is showed:

Listado 2.1: Convert json string to object in JavaScritp

```
function functionWichReceivesTheServerAnswer(resp_json){
    //resp_json is:
    //  '{"message": "diccionariy 2", "ok": true, "data": {"campo1":
        "valor1", "campo2": "valor2", "campo3": "valor3"}}'
    var obj_resp_json=$.parseJSON(resp_json);//$ mins JQUERY. It is
       necessary to load JQUERY library before
    alert(obj_resp_json.ok);//shows true
```

```
        alert(obj_resp_json.message);//shows the message "diccionariy 2"
        alert(obj_resp_json.data);//shows the json data, which will be a
            new json to decode again:
                                    "{"campo1": "valor1", "campo2": "
                                       valor2", "campo3": "valor3"}"
    }
```

CAPÍTULO 3

Database connection with Python. Use of the *psycopg* library

3.1 Introduction

In this chapter you are going to manually connect to Postgis, and manually perform SQL queries. Django will do it for you in the future, but you need to know this technique to be able to perform any query with Python.

3.2 Start the docker containers with the necessary services

We are going to use Docker to start the PostGIS service and the PgAdmin service. The PgAdmin service, and our small app are going to be clients of the PostGis service.

Follow the following steps:

- Create the folder *c:/docker*.
- Open a terminal, powersell o console window and clone the repository https://github.com/joamona/django-api-template:

```
git clone https://github.com/joamona/django-api-template.git
```

- Start Docker desktop to be able to use the Docker desktop service
- In the console, rename the folder django-api-template, to any other name related with your project.
- open Visual Studio Code (VS) and open the django-api-template folder by executing: code.
- Open a console in VS.
- Execute the script ./pgadmin_create_folders_windows.bat, to create some necessary folders.
- In Windows there is a problem
- Execute: docker compose up, to start the docker services.

The docker compose up command starts 4 services: postgreSQL + PostGIS, PgAdmin, geoserver, y desweb. Each one of them use a different port. You can see the ports of the services in the file .env. You can see the docker service definition in the file dockeer-compose.yml.

```
#.env file
DEVELOP_DOCKER_DJANGO_API_FORWARDED_PORT=8000
DEVELOP_DOCKER_POSTGIS_FORWARDED_PORT=8440
DEVELOP_DOCKER_PGADMIN_FORWARDED_PORT=8050
DEVELOP_DOCKER_GEOSERVER_FORWARDED_PORT=8080
```

PostgreSQL is accesible in the port 8440, PgAdmin in the port 8050 (http://localhost:8050), and geoserver in the port 8080 (http://localhost:8080/geoserver). The credentials for each one of these services are in the file .env.dev.

Open a web browser and check everything is running.

Open docker desktop and check the images and containers that you have just created. Note you can start - stop the containers from here.

Open a new terminal in VS and list the running containers with the command: docker ps

```
07f1bb22a62b   dpage/pgadmin4:8.3                  "/entrypoint.sh"
           44 minutes ago   Up 43 minutes             443/tcp,
   127.0.0.1:8050->80/tcp   django-api-template-pgadmin4-1
690b752f9615   django-api-template-deswebapi       "python manage.py
    ru?"   44 minutes ago   Up 43 minutes
   127.0.0.1:8000->8000/tcp          django-api-template-deswebapi-1
0038a73f24c9   docker.osgeo.org/geoserver:2.24.2   "/opt/startup.sh"
           44 minutes ago   Up 44 minutes (healthy)
   127.0.0.1:8080->8080/tcp          django-api-template-geoserver-1
6a70f2387735   django-api-template-postgis         "docker-
   entrypoint.s?"   44 minutes ago   Up 44 minutes (healthy)
   127.0.0.1:8440->5432/tcp          django-api-template-postgis-1
```

List the images with the command: docker image ls Check, in the project folder that some folders have been created to store the containers data.

When you have finished with the services, you can cancel with the key combination: control + c.

Next time you need the same services you have to do only two steps: start docker desktop and execute docker compose up.

3.3 Create a new database in the PostGIS docker container

The PostGis service cams with two databases: postgres and desweb. You are going to create a new one.

To create a database the docker services must be running. You need get into the postgis docker container. As you can see in the previous listing it is called django-api-template-postgis-1. To get into the container execute:

```
docker exec -it django-api-template-postgis-1 /bin/sh
```

Inside the container execute the following code:

```
\begin{lstlisting}
#createdb -U postgres desweb2  #create the database into the
   container
# psql -U postgres -d desweb2  #get into the database
psql (16.3 (Debian 16.3-1.pgdg110+1))
Type "help" for help.

desweb2=# create extension postgis;
CREATE EXTENSION
desweb2=# create schema d;
CREATE SCHEMA
desweb2=# create table d.buildings(id serial primary key,
   description text, area double precision, geom geometry("POLYGON",
    25830 ));
CREATE TABLE
desweb2=#
```

You can see the result in the Figura 3.1.

Note you could have done the same with the docker pgadmin service in http://localhost:8050, but in this way you have learned how get into a running container.

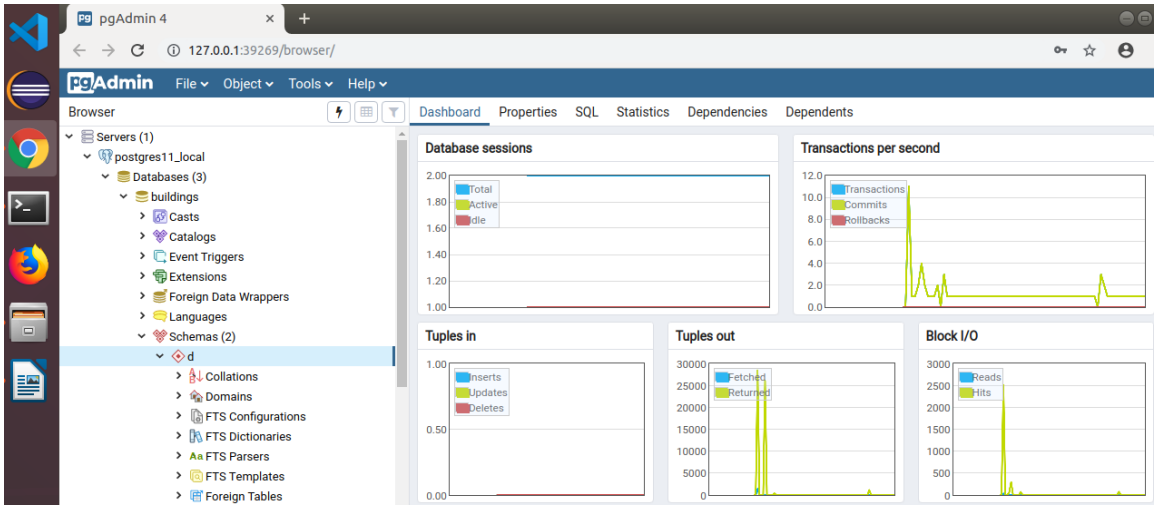

Figura 3.1: PgAdmin 4 showing the buildings table

3.4 Prepare the project folder and the Python

You are going to execute python scripts that execute SQL sentences into the database in the Docker *postgis* container, in the same stack (.yml). To do this you have to know:

- Thanks to the section volumes in the djangoapi service, in de docker-compose.yml, you have a shared folder between the container and the host (Windows). This folder is django-api-template/djangoapi/. See the volumes section in the .yml:

```
djangoapi:
  restart: "no"
  #the folder of the Django project
  build:
    context: ./djangoapi
    args:
      - USER_ID=${DJANGOAPI_USER_ID}
      - GROUP_ID=${DJANGOAPI_GROUP_ID}
      - USERNAME=${DJANGOAPI_USERNAME}

  command: python manage.py runserver 0.0.0.0:8000
  volumes:
    - ./djangoapi:/usr/src/app

  .....
```

- Any file/folder you create in the host inside the *djangoapi* folder is available inside the container in the folder /usr/src/app.
- You can get into the container and execute, with the container Python interpreter, the code you made in Windows with Visual Studio code, in the folder /djangoapi, py typing python myModule.py.
- So, you can create code in windows and execute it in Linux with the container intepreter.
- All the python libraries in the container are ready to use as you will execute the Python modules inside the container.

- You do not need python in Windows.
- The installed python libraries are in the file djangoapi/requirements.txt
- If you need to add another python library, you can add it to requirements.txt and rebuild the image: docker compose up –build.
- You execute always Docker and Docker Compose from windows.

Follow the following steps:

- Create the folder djangoapi/scripts/p1.
- Create the folder djangoapi/scripts/p1/myLib.
- Create the folder djangoapi/scripts/p1/buildings.

3.5 Execute SQL sentences with Python in the database

In this section you will learn how to connect with a database and how to perform any sql query. To make queries or changes in a PostgreSQL database with Python, we are going to use the *psycopg* library,*http://initd.org/psycopg/*.

3.5.1 Create the p1Settings.py file to store the database credentials

To connect to a database you need to know where is the the service and an user and a password. This information can change if you change the database location, or if you change the user password. It is good idea to keep all these information all together in a file. Usually this file is called settings.py and contains all the project global variables. Create a package called lib, and put the following code:

```
#p1/myLib/p1Settigns.py
POSTGRES_USER="postgres"
POSTGRES_PASSWORD="postgres"
POSTGRES_DB="buildings"
POSTGRES_HOST="localhost"
POSTGRES_PORT=5432  #the internal port of the postgis docker service
```

3.5.2 Execute any SQL sentence into the database

To execute any SQL sentence you have to follow the following steps:

- Connect to the database and get the object conn.
- Get the cursor with cur=conn.cursor() method.
- Create the SQL query string, avoiding to put the field values or where values. Instead use %s.
- Execute the cusor execute method to execute the sentence, passing the parameter values (%s) in a list, by order. The cur.execute method changes each *%s* by each parameter value. The first *%s* is replaced in the query for the first parameter value in the list, the second *%s* is replaced in the query for the second parameter value in the list, and so on.
- Get the results with cur.fetchall(), in case of insert or select, or cur.rowcount in case of update or delete sentences.

- Commit the results to persist the data into the database conn.commit(). The cursor.commit() function perform the changes into the database. Without the commit any change will be done into the database, despite the code be correct.
- Close the cursor and the connection cur.close() conn.close(). If you do not close the connection, the connection remains opened, and you can get out of connections, as the default maximum simultaneous connections is 100.

The following code shows an example of use by inserting a building:

```
#p1/buidings/insert.py
import psycopg
from myLib import p1Settings #I added EPSG_CODE=25830 to p1Settings.
   py
def connect():
    conn= psycopg.connect(
        dbname=p1Settings.POSTGRES_DB,
        user=p1Settings.POSTGRES_USER,
        password=p1Settings.POSTGRES_PASSWORD,
        host=p1Settings.POSTGRES_HOST,
        port=p1Settings.POSTGRES_PORT
        )
    print("Connected")
    return conn

def insert():
    conn=connect()
    cur=conn.cursor()
    cons="""
        INSERT INTO d.buildings
            (description, area,geom)
        VALUES
            (%s,%s,
            st_geometryFromText(%s,%s))
        RETURNING id
        """
    cur.execute(cons,
                ['My first building',
                 100,
                 'POLYGON((0 0, 100 0, 100 100, 0 100, 0 0))',
                 p1Settings.EPSG_CODE
                 ])
    conn.commit()
    l=cur.fetchall()
    #print(cur.fetchall()[0][0]) <-- ERROR. YOU ONLY CAN FECTH THE
       RESULTS ONCE
    print(l)
    print(l[0][0])
    cur.close()
    conn.close()
    print("Inserted")
```

3.5.3 How to get the results from the cursor

You already know how to execute any query with cursor.execute(RawQueryString, parametersList).

The cursor.execute() method does not returns anything. Instead it stores the results in the cursor objects. Depending on the sentence type the results in the cursor are stored and retrieved in a different way:

NSERT and SELECT: The results can be fetched with cursor fetchall(). This returns a list of rows. If the insert has not returning, or the select has not result fetchall() will return [].

SELECT: The results can be fetched with cursor fetchall(). This returns a list of rows. If not results you will get [].

PDATE and DELETE: The only result is the number of affected rows. This number can be retrieved with cursor.rowcount.

3.5.3.1. On insert

In the code of the section 3.5.2, add the following code before cur.close():

```
l=cur.fetchall()
#print(cur.fetchall()[0][0]) <-- ERROR. YOU ONLY CAN FECTH THE
   RESULTS ONCE
print(l)
print(l[0][0])
```

You will get something like [(25,)].

This is a list of rows. Each row is represented with a tuple. As only one row and the row only has one field (id), the result is list with one tuple with one value. The id can be fetched with cur.fetchall()[0][0].

3.5.3.2. On select:

You can get the results in the same way that you did in the case of insert with returning fields. Now you are going to need to connect again to the database. Each sql sentence needs connect - disconnect, so we are going to move the function insert.connect() to myLib/connect.py. So insert remain like follows:

```
from myLib.connect import connect

def insert():
    conn=connect()
    cur=conn.cursor()
    ... #the same
```

The myLib/connect.py module content is:

```
from myLib import p1Settings
import psycopg

def connect():
    conn= psycopg.connect(
        dbname=p1Settings.POSTGRES_DB,
        user=p1Settings.POSTGRES_USER,
        password=p1Settings.POSTGRES_PASSWORD,
        host=p1Settings.POSTGRES_HOST,
        port=p1Settings.POSTGRES_PORT
        )
    print("Connected")
    return conn
```

Now you can use the connect() function from any where in your project in the p1 folder.
To select all rows in the buildings table:

```
#buildings/select.py
from myLib.connect import connect

def select():
    conn=connect()
    cur=conn.cursor()
    cons="""
        SELECT
            (description, area,st_astext(geom))
        FROM
            d.buildings
        WHERE
            id>%s
        """
    cur.execute(cons, [0])
    l=cur.fetchall()
    print(l)
    print('First row:')
    print(l[0])
    conn.commit()
    cur.close()
    conn.close()
    print("Selected")
```

To execute the select function instead of execute the insert, you must change the main.py module:

```
from buildings.insert import insert
from buildings.select import select
select()
```

The result is the following:

```
/usr/src/app/scripts/p1# python main.py
Connected
[(('My first building', '100', 'POLYGON((0 0,100 0,100 100,0 100,0
   0))'),), (('My first building', '100', 'POLYGON((0 0,100 0,100
   100,0 100,0 0))'),), (('My first building', '100', 'POLYGON((0
   0,100 0,100 100,0 100,0 0))'),), (('My first building', '100', '
   POLYGON((0 0,100 0,100 100,0 100,0 0))'),)]
First row:
(('My first building', '100', 'POLYGON((0 0,100 0,100 100,0 100,0 0)
   )'),)
Selected
```

3.5.3.3. On update

Use the following code to update, and get the number of affected rows. We are going to use always the id to locate the row:

```
from myLib.connect import connect
from myLib.p1Settings import EPSG_CODE

def update():
    conn=connect()
    cur=conn.cursor()
    cons="""
        UPDATE
            d.buildings
        SET
            (description, area, geom) = ROW(%s, %s,
               st_geometryFromText(%s,%s))
        WHERE
            id=%s
        """
    # As there are 5 %s, you need a list with 5 values:
    #   [description, area, the_geom_wkt, the_epsg_code,
    #           the_id_to_select_the_row]
    valuesList=['New description',200,'POLYGON((0 0, 10 0, 10 10, 0
       10, 0 0))',EPSG_CODE, 1]
    cur.execute(cons, valuesList)
    print(cur.rowcount)
    conn.commit()
    cur.close()
    conn.close()
    print("Updated")
```

3.5.3.4. On delete

Use the following code to delete, and get the number of affected rows. We are going to use always the id to locate the row:

```
from myLib.connect import connect

def delete():
    conn=connect()
    cur=conn.cursor()
    cons="""
        DELETE FROM
            d.buildings
        WHERE
            id=%s
        """
    # As there are 5 %s, you need a list with 5 values:
    #   [description, area, the_geom_wkt, the_epsg_code,
    #           the_id_to_select_the_row]
    valuesList=[1]
    cur.execute(cons, valuesList)
    print(cur.rowcount)
    conn.commit()
    cur.close()
```

```
    conn.close()
    print("Deleted")
```

3.5.3.5. Get the cursor results as dictionaries

If you want to get each row as dictionary instead of a tuple, you can set a parameter on getting the cursor:

```
from psycopg.rows import dict_row
...

cur= conn.cursor(row_factory=dict_row)
...
```

Now the results on executing the select function are:

```
[{'description': 'My first building', 'area': 100.0, 'st_astext': '
   POLYGON((0 0,100 0,100 100,0 100,0 0))'}, {'description': 'My
   first building', 'area': 100.0, 'st_astext': 'POLYGON((0 0,100
   0,100 100,0 100,0 0))'}, {'description': 'My first building', '
   area': 100.0, 'st_astext': 'POLYGON((0 0,100 0,100 100,0 100,0 0)
   )'}, {'description': 'My first building', 'area': 100.0, '
   st_astext': 'POLYGON((0 0,100 0,100 100,0 100,0 0))'}]
First row:
{'description': 'My first building', 'area': 100.0, 'st_astext': '
   POLYGON((0 0,100 0,100 100,0 100,0 0))'}
Selected
```

3.5.3.6. Add system parameters to main.py to execute any function

Instead of modify the main.py code to execute the functions to act in your tables you can pass to the module two parameters: the table name to act, and the function name. The code is the following:

```
import sys

from buildings.insert import insert as insert_building
from buildings.select import select as select_building
from buildings.update import update as update_building
from buildings.delete import delete as delete_building

def main():
    # sys.argv[0] es siempre el nombre del archivo (main.py)
    # Por eso verificamos que haya al menos 3 elementos (nombre + p1
        + p2)
    if len(sys.argv) == 3:
        tableName = sys.argv[1]
        functionName = sys.argv[2]
    else:
        print("Error: You mus give two parameters tableName and
            functionName to execute the addecuate function.")
        sys.exit(0)


    if tableName not in ["buildings", "trees", "water"]:
        print("Error: The available table names are buildings, trees
            , water")
        sys.exit(0)
```

```
    if functionName not in ["insert", "select", "update", "delete"]:
        print("Error the available function names are insert, select
            , delete or update")
        sys.exit(0)

    if tableName == "buildings":
        if functionName=="insert":
            insert_building()
        elif functionName=="select":
            select_building()
        elif functionName=="update":
            update_building()
        elif functionName=="delete":
            delete_building()
    elif tableName=="trees":
        if functionName=="insert":
            pass
        elif functionName=="select":
            pass
        elif functionName=="update":
            pass
        elif functionName=="delete":
            pass

if __name__ == "__main__":
    main()
```

Now you can execute any function in the following way:

```
/usr/src/app/scripts/p1# python main.py buildings insert
Connected
[(5,)]
5
Inserted
```

3.5.4 Optimize the code with POO (optional)

You can create a class to automate the connect/disconnect/execute/fetch/rowcount/print operations. The following class automatizes all those operations:

```
#myLib\db.py

import psycopg
from psycopg.rows import dict_row

from myLib import p1Settings as settings
class Db:
    def __init__(self, autoCommit=True, autoPrintResults=True,
        getRowsAsDicts=True):
        """Create a connection to the database
        autoCommit: if True, the connection will commit the
        changes automatically. If False, the connection will
        need to be committed manually
        autoPrintResults: if True, the connection will print the
        results of the query automatically. If False, the
```

```
        results will not be printed
        """
        self.getRowsAsDicts=getRowsAsDicts
        self.conn = psycopg.connect(
            dbname=settings.POSTGRES_DB,
            user=settings.POSTGRES_USER,
            password=settings.POSTGRES_PASSWORD,
            host=settings.POSTGRES_HOST,
            port=settings.POSTGRES_PORT
        )
        self.autoPrintResults = autoPrintResults
        self.autoCommit = autoCommit

        if self.getRowsAsDicts:
            self.cursor = self.conn.cursor(row_factory=dict_row)
        else:
            self.cursor = self.conn.cursor()

        self.result = None
        print("Connected to database")

    def query(self, query, params=None):
        """
        Execute a query and return the results
            eg. query("SELECT * FROM table WHERE area > %s and owner
                like %s", [100], ['%Smith%']))

        query: the query to execute. Use %s as a placeholder for

        parameters. Eg. "SELECT * FROM table WHERE area > %s
                and owner like %s"

        params: the parameters to pass to the query in a list.
                Eg. [100, '%Smith%']

        return: the results of the query in a list. Eg.
                [(1, 'Smith', 200), (2, 'Smith', 300)]
        """
        self.cursor.execute(query, params)
        if self.autoCommit:
            self.conn.commit()
        #insert or select put result in cursor.fetchall(),
        # but update and delete do not.
        #update and delete put something in cursor.rowcount
        # but insert or select do not
        #So we have to manage this:
        if "insert" in query.lower() or "select" in query.lower():
            self.result = self.cursor.fetchall()
        else:
        #operations update and delete
            self.result = self.cursor.rowcount
        if self.autoPrintResults:
            self.printResult()
```

```
    def disconnect(self):
        """Disconnect from the database
        Disconnect from the database. This should be called when
        the connection is no longer needed.
        There is a maximum number of connections that can be
        made to the database, so it is important to
        disconnect when the connection is no longer needed.
        """
        self.cursor.close()
        self.conn.close()
        print("Disconnected from database")

    def printResult(self):
        print(self.result)

    def __del__(self):
        """ Destructor. Disconnect from the database.
        On destruction of the object, disconnect from the
        database.
        This is automatically called when the object is deleted,
        eg. when the program ends.
        """
        self.cursor.close()
        self.conn.close()
        print("Disconnected from database")
```

VERY, VERY, VERY IMPORTANT:

AFTER TO HAVE FINISHED WITH THE CURSOR AND CONNECTION, IT IS TOTALLY FUNDAMENTAL TO CLOSE THEM:

```
cursor.close()
conn.close()
```

IF YOU DO NOT CLOSE THE CONNECTION, THE CONNECTIONS CONTINUE OPENED, AND THERE ARE A DEFAULT NUMBER OF CONNECTION OF 100. IN FEW MINUTES YOU WILL DO POSTGRES UNUSABLE FOR ANY OTHER USER OR PROJECT. ANY MORE WILL BE ABLE TO CONNECT TO THE DATABASE, UNTIL THE POSTGRES SERVICE BE RESTARTED.

If the cursor, or the connection is closed, you can not send any sql query.

3.5.4.1. Insert rows with geometry

Here the code with the use of the Db class is used to insert. Note you need much less code:

```
from lib.db import Db

db=Db()
query="insert into d.buildings (description, geom) values (%s,
   st_geometryfromtext(%s,25830)) returning id"

values1=["edificio 1", "POLYGON((727844 4373183,727896
   4373187,727893 4373028,727873 4373018,727858 4372987,727796
   4372988,727782 4373008,727844 4373183, 727844 4373183))"]
values2=["edificio 2", "POLYGON((727988 4373188,728054
   4373192,728051 4373093,727983 4373093,727988 4373188))"]

db.query(query, values1)
db.query(query, values2)
```

The result of this program is:

```
Connected to database
[(1,)]
[(2,)]
Disconnected from database
```

Note the advantage of using OOP. Automatically the pgConn instance is:

- Importing the psycopg2 library.
- Connecting to the database.
- Commiting the query
- Printing the results.
- Disconnecting from the database.

INCREDIBLE!!!

The Db class automatically close the connection on finishing the program, as all the object instances are deleted on finishing the program. All the deleted objects call method __del__. This is because all classes inherit from the base class Object, and method __del__ is in the Object class, so all child classes have the same method. What we have done here is overwrite the father __del__ method. So the OOP theory is useful.

We use WKT format because OpenLayers is able to give its geometries in this forma. The WKT representation of a polygon geometry is:

```
"POLYGON((727844 4373183,727896 4373187,727893 4373028,727873
   4373018,727858 4372987,727796 4372988,727782 4373008,727844
   4373183, 727844 4373183))"
```

3.5.4.2. Update rows

Create a new module called *updateBuilding.py*, and paste the following code:

```
from lib.db import Db

db=Db()
query="update d.buildings set (description, geom) = row(%s,
   st_geometryfromtext(%s,25830)) where id=%s"

values=["Edificio actualizado",
        "POLYGON((727988 4373188, 728054 4373192,
           728095.84297791088465601 4373142.83781164418905973,
           728051 4373093, 727983 4373093, 727988 4373188))",
        5
        ]
db.query(query, values)
```

The result of the previous program is the following:

```
Connected to database
1
Disconnected from database
```

As you can see, the update query has three *%s*, so the values list has to have also three values, the last of which is taken for the where condition. Only the building who *id* is *5* is going to be update.

The number of rows affected is stored in the *cursor.rowcount* property.

3.5.4.3. Delete rows

Create a new module called *deleteBuilding.py*, and paste the following code:

```
from lib.db import Db
db=Db()
query_ins="delete from d.buildings where id=%s"
values=[5]
db.query(query_ins, values)
```

The result is the following:

```
Connected to database
1
Disconnected from database
```

As you can see, you do not repeat code unnecessarily.

3.5.4.4. Select rows

Create a new module called *selectBuildingAsTuples.py*, and paste the following code:

```
from lib.db import Db
db=Db()
query="select id, description, st_astext(geom) from d.buildings
   where id=%s"
values=[6]
db.query(query, values)
```

The result is the following:

```
Connected to database
[{'id':6, 'description':'edificio 1', 'geom':'POLYGON((727844
   4373183,727896 4373187,727893 4373028,727873 4373018,727858
   4372987,727796 4372988,727782 4373008,727844 4373183,727844
   4373183))'}]
Disconnected from database
```

e

As you can see, the result is a list of tuples with all the selected rows in a tuple. The order of the values is the order of the fields in the select query.

3.6 Topological geometry checks

The database checks in real time the field values, checking types and valid values. This is the normal bbdd operation, and mostly we use database because this characteristics, in addition to the select clause to filter the data.

But we are using postgis, which adds geomerty capabilities. Functions like st_intersects, st_covers, st_within, st_isvalid, st_snaptogrid, and st_snap are available to check the geometry validity before to perform the insert of the update of the row.

3.6.1 Round coordinates to be able to match vertices (st_snapToGrid)

This section is more important that you think. The geometrical engine compares the coordinate numbers to check if two vertex are the same, and if they do not, it is because the geometries intersects or are separated. This means 25.1234567, and 25.12314568, are no the same, therefore the geometries are not adjacent, intersects, or there is a hole between them. The difference is 0.0000001, the physical difference is absurd, but mathematically is not correct.

The goal of the command st_snaptogrid(geom, distance) is to round decimals. For example st_snaptogrid(geom, 0.005) the coordinates of the previous example, will be rounded to 25.125, and 25.125. This means they are exactly equal, therefore adjacent, which is the expected result. This operation must be done before to insert any geometry, and perform this operation also on the geometries already existing in the database, if they have not been adjusted to the grid, or rounded, before. Be careful with this operation, it can spoil all the geometries if you do not know what you are doing.

The grid must be at least 10 times more accurate than the data. If your data as an accuracy 0.05 m, you can use a grid of 0.005.

This is similar to the cluster tolerance applied in datasets in the ArgGis technology.

3.6.2 Check intersection on insert

Common geometry checks on insert are:
For example, a common task on inserting a geometry is:

- Round the coordinates of the geometry to be able to match vertices.
- Check if the geometry is valid (st_isvalid).
- Check if other geometry intersects with the new one (st_intersects).

Let see an example with the insert to add a new geometry:

```
import sys

from lib.db import Db
from lib.settings import SNAP_DISTANCE

db=Db()

geometry="POLYGON((0.00001 0.00001, 100.00001 0.00001, 100.00001
   100.00001, 0.00001 100.00001, 0.00001 0.00001))"

#check if the geometry is valid after having simplified it
query="""
    select ST_isvalid(
        st_snaptogrid(
            st_geomfromtext(%s, 25830),
            %s
        )
    ) as is_valid
"""
db.query(query, [geometry, SNAP_DISTANCE])

if db.result[0][0]:
    print("The geometry is valid")
else:
    print("The geometry is not valid")
```

```
    sys.exit()

#check if the geometry intersects any existing building
query="""
    select id from d.buildings where ST_intersects(
        geom,
        st_snaptogrid(
            st_geomfromtext(%s, 25830),
            %s
        )
    )
"""

db.query(query, [geometry, SNAP_DISTANCE])
if len(db.result) > 0:
    print("There are buildigs that intersect with the new geometry:
       {0}".format(db.result))
    sys.exit()
else:
    print("The new geometry does not intersect any existing building
       ")

#now we now that the geometry is valid and does not intersect any
   existing building
#we can insert the new building
query="insert into d.buildings (description, geom) values (%s,
   st_snaptogrid(st_geometryfromtext(%s,25830),%s)) returning id"
db.query(query, ["new building", geometry, SNAP_DISTANCE])

#Let's see if the original coordinates have been modified
query="select id, st_astext(geom) from d.buildings where id=%s"
values=[db.result[0][0]]
print("Original coordinates: {0}".format(geometry))
print("Note the coordinates have been modified to snap to the grid.
   They have been rounded")
db.query(query, values)
```

An the result, if the geometry is valid and do not intersects with any other in the table is:

```
Connected to database
[(True,)]
The geometry is valid
[]
The new geometry does not intersect any existing building
[(30,)]
Original coordinates: POLYGON((0.00001 0.00001, 100.00001 0.00001,
   100.00001 100.00001, 0.00001 100.00001, 0.00001 0.00001))
Note the coordinates have been modified to snap to the grid. They
   have been rounded
[(30, 'POLYGON((0 0,100 0,100 100,0 100,0 0))')]
Disconnected from database
```

If you try to insert the same geometry, as it intersects with the already inserted, the operation will be rejected.

3.6.3 Check intersection on update

The algorithm is the same except because we have to exclude, from the intersection check, the geometry that we are trying to update, as logically always most of times the new geometry is going to intersect with the old one.

```
import sys

from lib.db import Db
from lib.settings import SNAP_DISTANCE

db=Db()

geometry="POLYGON((0.00001 0.00001, 100.00001 0.00001, 100.00001
   200.1238567, 0.00001 100.00001, 0.00001 0.00001))"
id=30  #the id of the building we want to update

#check if the geometry is valid after having simplified it
query="""
    select ST_isvalid(
        st_snaptogrid(
            st_geomfromtext(%s, 25830),
            %s
        )
    ) as is_valid
"""
db.query(query, [geometry, SNAP_DISTANCE])

if db.result[0][0]:
    print("The geometry is valid")
else:
    print("The geometry is not valid")
    sys.exit()

#check if the geometry intersects any existing building, except the
   one we are updating
query="""
    select id from d.buildings where ST_intersects(
        geom,
        st_snaptogrid(
            st_geomfromtext(%s, 25830),
            %s
        )
    ) and id != %s
"""

db.query(query, [geometry, SNAP_DISTANCE, id])
if len(db.result) > 0:
    print("There are buildigs that intersect with the new geometry:
       {0}".format(db.result))
    sys.exit()
else:
    print("The new geometry does not intersect any existing building
         except the one we are updating")
```

```
#now we now that the geometry is valid and does not intersect any
   existing building
#we can insert the new building
query="update d.buildings set (description, geom) = row(%s,
   st_snaptogrid(st_geometryfromtext(%s,25830),%s)) where id=%s"
db.query(query, ["Building updated", geometry, SNAP_DISTANCE, id])

#Let's see if the description and coordinates have been modified
query="select id, description, st_astext(geom) from d.buildings
   where id=%s"
db.query(query, [id])
```

And the result is the following:

```
Connected to database
[(True,)]
The geometry is valid
[]
The new geometry does not intersect any existing building except the
    one we are updating
1
[(30, 'Building updated', 'POLYGON((0 0,100 0,100 200.124,0 100,0 0)
   )')]
Disconnected from database
```

3.6.4 You are not aware of something important. You made a mistake. Topological relations. 9IM matrix

The previous intersection checks are wrong. What happen if two geometries share part of the perimeter?. They do not intersects, but the st_intersects will return true. Why?. Because they intersects in the perimeter. Check if the following adjacent squares intersect or not:

```
select st_intersects(
    st_geometryfromtext('polygon((0 0, 1 0, 1 1, 0 1, 0 0))',25830),
    st_geometryfromtext('polygon((1 0, 2 0, 2 1, 1 1, 1 0))',25830)
)
```

The result is true.
Lets calculate the intersection between the two geometries:

```
select st_astext(
    st_intersection(
        st_geometryfromtext('polygon((0 0, 1 0, 1 1, 0 1, 0 0))
           ',25830),
        st_geometryfromtext('polygon((1 0, 2 0, 2 1, 1 1, 1 0))
           ',25830)
    )
);

      st_astext
--------------------
 LINESTRING(1 0,1 1)
(1 fila)
```

The intersection is the common perimeter. This is not you expected. To avoid this mistake you must use the st_relate function and the 9IM matrix, Figura 3.2.

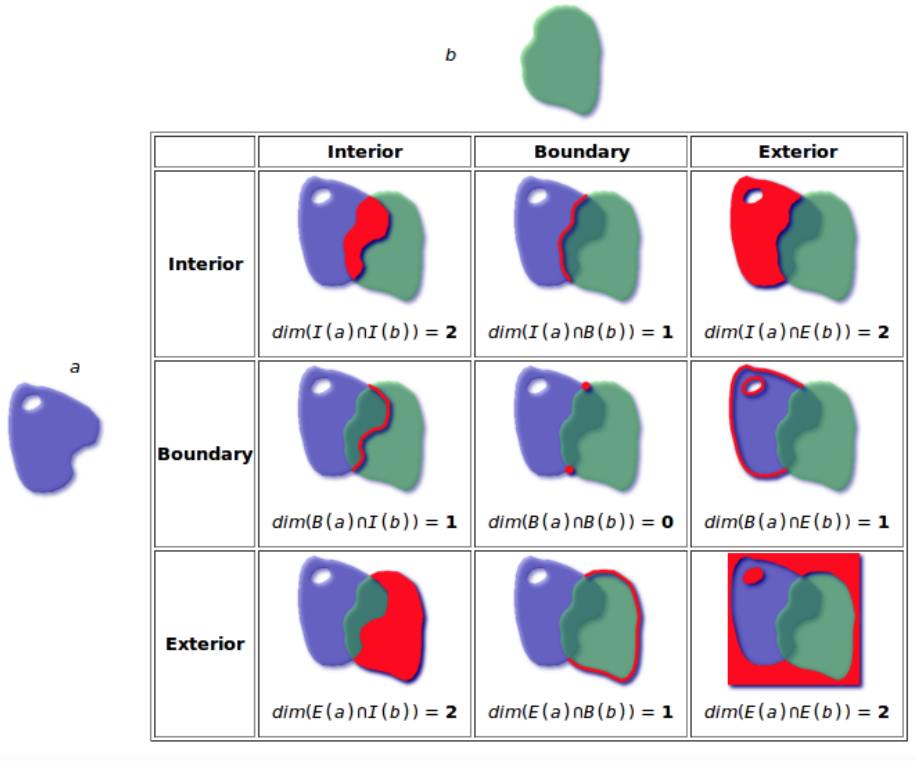

Figura 3.2: Topological relation between two geometries: 9IM matrix. Source: http://en.wikipedia.org/wiki/DE-9IM

Each geometry has three parts: Interior, perimeter and exterior. What you need is to check if the interior of the polygons intersects, and any other relation is indifferent. This is the relation T******** (see the figure). T means true, F means false, and * means any value.

Lets try if the two interiors of the squares intersects:

```
select st_relate(
    st_geometryfromtext('polygon((0 0, 1 0, 1 1, 0 1, 0 0))',25830),
    st_geometryfromtext('polygon((1 0, 2 0, 2 1, 1 1, 1 0))',25830),
    'T********'
)
```

The result is false, as you expected.

Let's check if the two perimeters intersects:

```
select st_relate(
    st_geometryfromtext('polygon((0 0, 1 0, 1 1, 0 1, 0 0))',25830),
    st_geometryfromtext('polygon((1 0, 2 0, 2 1, 1 1, 1 0))',25830),
    '****T****'
)
```

The result is true.

By the way, st_intersection can return any type of geometries. If you insert a vertex in the polygon 2, in the coordinates (0.5, 0.5), you will get a polygon with a triangle:

```
select st_astext(
    st_intersection(
        st_geometryfromtext('polygon((0 0, 1 0, 1 1, 0 1, 0 0))
            ',25830),
```

```
        st_geometryfromtext('polygon((1 0, 2 0, 2 1, 1 1, 0.5 0.5, 1
            0))',25830)
    )
);
           st_astext
-------------------------------
 POLYGON((1 0,0.5 0.5,1 1,1 0))
(1 fila)
```

3.6.5 Conclusions

About coding:

- OOP approach is very advantageous.
- The only difficult to change table values are the SQL sentences.
- We can check topology conditions in real time, avoiding inconsistent geometries.
- If the new geometry do not adjust with the existing ones we can calculate the intersection (st_intersection), or the holes between the geometries, and return them to the GIS or geoportal drawing them, to allow editors fix the errors easily. We did not do that because I do not want to complicate more the subject.

About geometry checks:

- Always use st_snapToGrid to snap the vertex to a grid. You can think in round the coordinates. Use always 1/10 times your accuracy as the step of the grid.
- To know if two polygons intersects you must not use st_intersects but st_relate, with the relation 'T********'.
- To know if two polygons are adjacent you can use st_relate, with the relation 'F***T****', which mean, the interior do not intersects, but the perimeters do.

CAPÍTULO 4

Database connection with Django

4.1 Introduction

In this chapter you are going to see how to modify a table using Django. It is a training to get the philosophy. You are going to learn the following:

- Create django projects.
- Create django apps.
- Connect the project to a database.
- Create a Django superuser.
- Access to the Django Admin Site.
- Create models.
- Use object models to insert, delete and update rows.
- Use the model class to filter rows.
- Evaluate the filter to get the model objects.
- Transform a model object into a dictionary.

The django-api-template runs in a linux container, and everything is installed, the project is created, the admin user is created, etc. In this chapter you do all this steps to learn how to do it.

In Windows you probably do not have GEOS installed, therefore geodjango, the django aplication to manage geometries, do not work. It can not import libraries, and the Django project can not run. Due to that, i have commented the lines where geodjango is used, and put, on the right of the code line, the comment:

```
#line commented  #deactivate in windows. You don't have GEOS
```

So if you can not run the project doe to import problems, comment that lines, and work with your tables, removing the geom field of your models.

In the next chapter you will use a Dockerized Django API everything ready. You will work with geometries there.

4.2 Create a Django project with your Python venv in the host (Windows or Linux)

Navigate where you are working with your database, activate the venv, who has all the dependencies installed, and create the Django project, form example *djdesweb*.

Now, with the virtualenv activated we are ready to start a Django project and an Django app. A Django project can contain several apps. Apps are ways of organize the code. You do not need any app, but I recommend you to have at least one.

With the virtualenv activated, type the following:

```
(env) vagrant@linux:~/apps$ django-admin startproject djdesweb
(env) vagrant@linux:~/apps$
```

The command creates a folder named djdesweb with the files showed in the Figura 4.1[(1)].

[(1)] Source https://docs.djangoproject.com/es/2.2/intro/tutorial01/

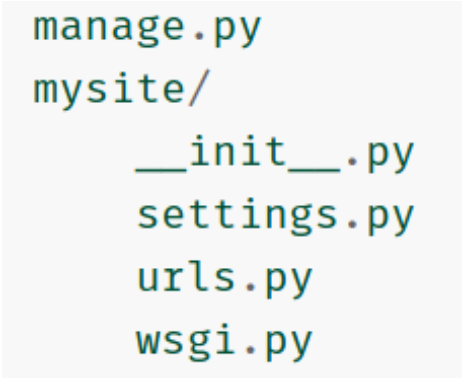

Figura 4.1: File structure created by the command django-admin startproject djdesweb

4.3 Create an app for the project

Apps are a way to organize the code. Thing about it as a way of classify the parts of your project: users, sells, map, ... Each part deserve an app to store there their urls, models, serializers, etc, to manage the tables separately.

Follow the following steps to create an app:

```
(env) vagrant@linux:~/apps$ cd djdesweb/
(env) vagrant@linux:~/apps/djdesweb$ python manage.py startapp
   appdesweb
```

The command creates the folder appdesweb with the files of the Figura 4.2.

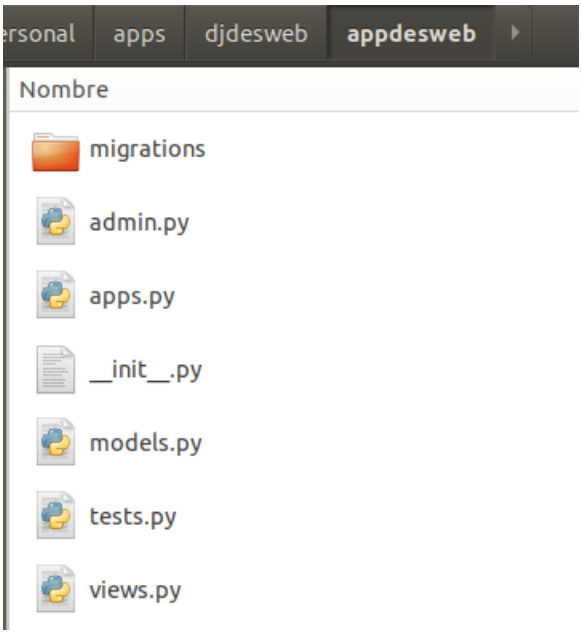

Figura 4.2: File structure created by the command python manage.py startapp appdesweb

4.4 Configure the Django project

We have to modify the files created automatically by Django. Open the project with VS: code .

Once the project and the app is created you need to configure the project. All the configurations are made in the module djdesweb/settings.py. This file configures the entire project. You have to change the following:

Change:

```
ALLOWED_HOSTS = []
```

by the following, to allow other ip addresses to use the app:

```
ALLOWED_HOSTS = ['*']
```

Add to the INSTALLED_APPS list the app *appdesweb*.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    #required by geodjango
    #'django.contrib.gis', # deactivate in windows. You don have
       GEOS
    #para el CORS
    'corsheaders',
    'rest_framework',
    #'rest_framework_gis', # deactivate in windows. You don have
       GEOS
    'django_filters',
    'drf_yasg',

    'appdesweb'
]
```

Add the following to the MIDDLEWARE section to avoid CORS, while debug:

```
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
#para el CORS
if DEBUG:
    CORS_ALLOW_ALL_ORIGINS=True
```

Comment the following lines

```
#DATABASES = {
#    'default': {
#        'ENGINE': 'django.db.backends.sqlite3',
```

```
#         'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
#     }
#}
```

Runt the django-api-template, in order to have postgres running in the port 8440. Use the PgAdmin service to:

- Create a new database called djdesweb.
- Create the extension postgis.

And add the bellow, to configure the database credentials, to connect with the before created database.

```
DATABASES = {
    'default': {
            #you must select only one ENGINE postgresql_psycopg2 or
               gis.db.backends.postgis.
            #Comment one of the followings two lines, and uncoment
               the other one
        ENGINE': 'django.db.backends.postgresql_psycopg2',
        #'ENGINE': 'django.contrib.gis.db.backends.postgis', #
           deactivate in windows. You don have GEOS
        'NAME': 'djdesweb',
        'USER': 'postgres',
        'PASSWORD': 'postgres',
        'HOST': 'localhost',
        'PORT': '8440',
    }
}
```

4.5 Create the Django administration tables: migrate

Now that the postgis database and credentials are set, you can create all the internal Django tables into desweb database, to allow Django to manage users and sessions. Type the following:

```
(env) vagrant@linux:~/apps/djdesweb$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
```

```
Applying auth.0011_update_proxy_permissions... OK
Applying sessions.0001_initial... OK
```

You can see the tables added to the *djdesweb* database, in the schema *public*, Figura 4.3.

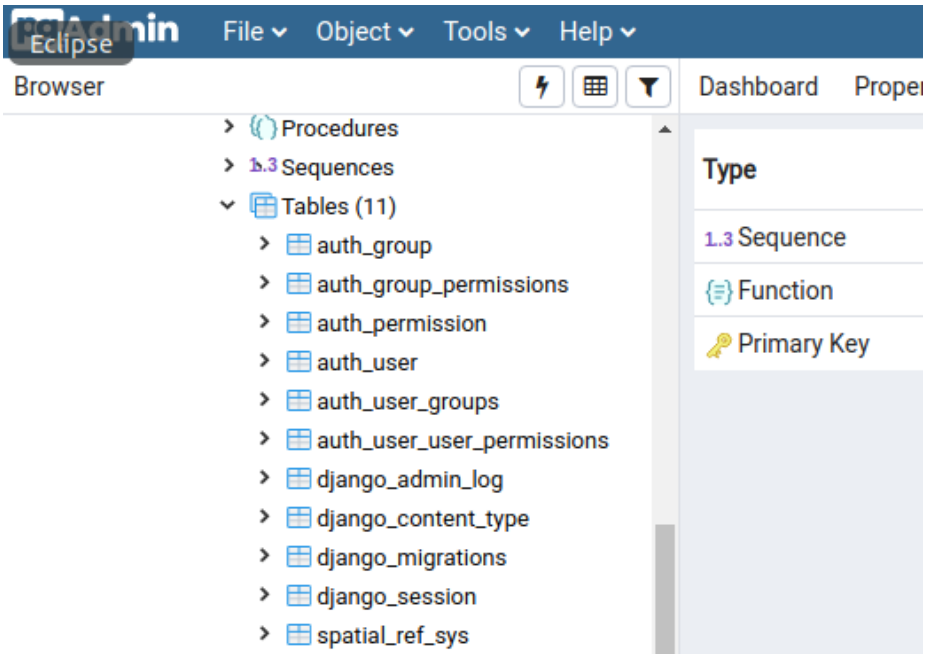

Figura 4.3: File structure created by the command python manage.py startapp appdesweb

4.6 Create the a superuser to access to the Django admin site

Start the development django service. Where the manage.py file is, the Django project folder, type the following:

```
python manage.py runserver
```

This will start the API service. Visit the default page at http://localhost:8000.

Remember a service is a program, always running, waiting for someone to ask something. In this case the service is running in the port 8000, and it is an http service. It is because this that the newly created Django API understand the browser request.

Try to access to http://localhost/admin/. Type user admin, password admin. You can not get in. This is because there is not still any user. Lets create a superuser for the API. Ctrl + C to stop the django service ant type (**Please note that we are using the default passwords in all the settings. This is potentially dangerous in a real deploy. In that case you mus change all the passwords**. This step is not necessary if you are not going to use the Django admin site.):

```
(env) vagrant@linux:~/apps/djdesweb$ python manage.py
   createsuperuser
Username (leave blank to use 'vagrant'): admin
Email address: admin@admin.com
Password:
Password (again):
The password is too similar to the username.
This password is too short. It must contain at least 8 characters.
```

```
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

You can run the Django server again:

```
(env) vagrant@linux:~/apps/djdesweb$ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 23, 2019 - 08:51:16
Django version 2.2, using settings 'djdesweb.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

You can visit the default page in http://localhost:8000. You can visit the Django admin page in http://localhost:8000/admin/ (user admin, psw admin).

Now you are ready to configure the models, urls and views of your app, in order to be able to manage your database through internet.

4.7 Create a model in the appdesweb app

See the documentation: https://docs.djangoproject.com/en/5.1/topics/db/models/.

Models are models of tables. Django is able to create the tables from the models (which are made in Python code) or use an existing table.

Each app has its own folder, and in each app folder there is a file called models.py. Put the following content:

```
from django.db import models
import django.utils.timezone as djangoTimezone
from django.contrib.gis.db import models as gis_models

# Create your models here.
class Buildings(models.Model):
    description = models.CharField(max_length=100, blank=True, null=
       True)
    area = models.FloatField(blank=True, null=True)
    perimeter = models.FloatField(blank=True, null=True)
    height = models.FloatField(blank=True, null=True)
    data_creation = models.DateTimeField(auto_now_add=True,blank =
       True, db_default=djangoTimezone.now())
    geom = gis_models.PolygonField(srid=25830,blank=True, null=True)
```

The data_creation field is an automatic field that stores the date of creation of every row. A very common task.

Now to create the table into the database djdesweb, type the following:

```
(venv) joamona@upv7cores:~/djdesweb$ python manage.py makemigrations
Migrations for 'appdesweb':
  appdesweb/migrations/0001_initial.py
    - Create model Buildings
(venv) joamona@upv7cores:~/djdesweb$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, appdesweb, auth, contenttypes,
     sessions
```

```
Running migrations:
  Applying appdesweb.0001_initial... OK
```

Createmigrations create the file 0001_initial.py in the appdesweb/migrations folder.

Migrate read the file appdesweb/migrations/0001_initial.py, and creates the table.

Other models will create other migrations files, even if you change a model other files will be created 0002..., 0003....

If you want to see which code sql is going to be execute, type the following, seen the migration file number

```
python manage.py sqlmigrate appdesweb 0001
BEGIN;
--
-- Create model Buildings
--
CREATE TABLE "appdesweb_buildings" ("id" integer NOT NULL PRIMARY
   KEY GENERATED BY DEFAULT AS IDENTITY, "description" varchar(100)
   NOT NULL, "area" double precision NOT NULL, "geom" geometry(
   POLYGON,25830) NOT NULL);
CREATE INDEX "appdesweb_buildings_geom_06e1b45f_id" ON "
   appdesweb_buildings" USING GIST ("geom");
COMMIT;
```

4.8 Using models to update the database

Models allow to insert, delete, update and select data from a table. Each table has an associated model to manage the table. Now we are going to create some Python modules to test the model.

Create the folder *scripts* in the Django project folder.

We are going to use the GEOS Library. Visit https://docs.djangoproject.com/en/5.1/ref/contrib/gis/geos/.

May be it is not installed in your host, so may be the examples are not going to work. Do not worry, the django-api-template has everything installed and ready.

4.8.1 Insert a row

Create the folder scripts/p1/buildingsDjangoModels to store the scripts to insert, delete, update, etc, using Django Models.

Create script insertDjango.py module (scripts.p1.buildingsDjangoModels.insertDjango).

This is the code:

```
from django.contrib.gis.geos import GEOSGeometry
from buildings2.models import Buildings
from scripts.p1.myLib import p1Settings

def insert(d:dict):
    #create the geometry with geos
    g=GEOSGeometry(d['geom'], srid=p1Settings.EPSG_CODE)
    #print the representation of the object
    if g.valid:
        print("Geometría válida")
    else:
        return {'ok': False, 'message':'Invalid geometry', 'data':
           None}
    print(g)
    #create a building object, from the model Buildings
```

```
        b=Buildings(description=d['description'], area=g.area, perimeter
            =g.length, height=d['height'], geom=g)

        #saves it into the database
        b.save()
        #prints the asigned id of the object in the database
        print(b.id)

        ###################################
        #another way to create the object with a dictionary

        d['geom']=g
        d['area']=g.area
        d['perimeter']=g.length

        #use the ** operator over a dictionary to automatically get the
        #   fieldname=fieldvalue parameters from the dictionary
        b2=Buildings(**d)
        b2.save()
        print(b2.id)

        return {'ok': True, 'message':'Building inserted', 'data': [{'id
            ':b.id}]}

    def run():
        d_of_values= {
            'description':'Edificio 1',
            'height':100,
            'geom':'POLYGON((0 0, 10 0, 10 10, 0 11, 0 0))'
        }

        print(insert(d_of_values))
```

To execute it open a terminal in the container and execute:
python manage.py runscript scripts.p1.buildingsDjangoModels.insertDjango:

```
{'ok': True, 'message': 'Building inserted', 'data': [{'id': 17}]}
```

You have to use manage.py and runscript, to execute scripts that use Django models. The scripts must be in the scripts folder.

The runscript utility executes the *run()* function inside the module, so that you have to create it in the module. You can use this scripts to read files with data and initialize the database.

The scripts can also receive parameters. You have an example in the file scripts/001_hello_script.py

You can also create the building object with dictionaries, by using the ** operator. The ** operator over a dictionary automatically gets the fieldname=fieldvalue constructor parameters from the dictionary.

```
        d['geom']=g
        d['area']=g.area
        d['perimeter']=g.length

        #use the ** operator over a dictionary to automatically get the
        #   fieldname=fieldvalue parameters from the dictionary
        b2=Buildings(**d)
        b2.save()
        print(b2.id)
```

As you can see you need replace the d['geom'] values, originally in WKT by its GEOSGeometry object version, and also set the area and perimeter parameters into the dictionary. You can automate this by overriding the save() method in the Building model:

```
#buildings2\models.py
class Buildings(models.Model):
    description = models.CharField(max_length=100, blank=True, null=
       True)
    area = models.FloatField(blank=True, null=True)
    perimeter = models.FloatField(blank=True, null=True)
    height = models.FloatField(blank=True, null=True)
    geom = gis_models.PolygonField(srid=25830,blank=True, null=True)
    def save(self, *args, **kwargs):
            # Calculate values from the geometry before saving
            if self.geom:
                self.area = self.geom.area
                self.perimeter = self.geom.length

            super().save(*args, **kwargs)
```

The new save() method gets the area and perimeter properties from the geometry, set them into the field values, and calls the super().save() to perform the original save() method of the model, but with the field values modified. Also the area and perimeter are calculated only if the geom field is present, so buildings without geometry do not raise errors.

Not only this, on edit a geometry, you do not need to worry about the area or perimeter. The new values are going to be automatically calculated, on execute save().

With this change in the Building model, the shortest code of the insert() function is:

```
#buildings2\models.py
def insert2(d:dict):
    g=GEOSGeometry(d['geom'], srid=p1Settings.EPSG_CODE)
    if g.valid:
        print("Geometría válida")
    else:
        return {'ok': False, 'message':'Invalid geometry', 'data':
           None}
    d['geom']=g
    b=Buildings(**d)
    b.save()
    return {'ok': True, 'message':'Building inserted', 'data': [{'id
       ':b.id}]}
```

It will be great, instead of returning only the id of the new building, to return the complete new row. This can be done in the following way, by using the Django model_to_dict function:

```
#buildings2\models.py
def insert2(d:dict):
    g=GEOSGeometry(d['geom'], srid=p1Settings.EPSG_CODE)
    if g.valid:
        print("Geometría válida")
    else:
        return {'ok': False, 'message':'Invalid geometry', 'data':
           None}
    d['geom']=g
    b=Buildings(**d)
    b.save()
```

```
    d=model_to_dict(b)
    return {'ok': True, 'message':'Building inserted', 'data': [d]}
```

The result is the following:

```
{'ok': True, 'message': 'Building inserted', 'data': [{'id': 21, '
   description': 'Edificio 1', 'area': 105.0, 'perimeter':
   41.04987562112089, 'height': 100, 'geom': <Polygon object at 0
   x78e40aa0cc00>, 'data_creation': datetime.datetime(2026, 3, 6,
   11, 20, 50, 474703, tzinfo=datetime.timezone.utc)}]}
```

As you can see area, perimeter and data_creation are automatically set. The problem now is that the geom and data_creation values in the dictionary are objects, datetime and a Polygon. We then need to modify the answer in the following way:

```
    ...
    d=model_to_dict(b)
    d['geom']=g.wkt
    d['data_creation']=d['data_creation'].strftime("%Y-%m-%d %H:%M:%
       S")
    return {'ok': True, 'message':'Building inserted', 'data': [d]}
```

And now the answer is the following:

```
{'ok': True, 'message': 'Building inserted', 'data': [{'id': 22, '
   description': 'Edificio 1', 'area': 105.0, 'perimeter':
   41.04987562112089, 'height': 100, 'geom': 'POLYGON ((0 0, 10 0,
   10 10, 0 11, 0 0))', 'data_creation': '2026-03-06 11:20:50'}]}
```

But wait, how can we use the functions st_snapToGrid and st_relate to properly check and store the geometries in the database?. You can do it with the connection of Django, that is a psycopg normal connection object. You can import the Django connection (from django.db import connection), that is opened with the database, use it as normal, but no close it. Django is in charged of close it for you.

The following code only inserts the geometry in case it is valid, it does not intersects with any other and when it is inserted, the geometry has been snapped to the grid. By the way, instead of using the settings in p1Settings.py, we must use the settings EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION in djangoapi/settings.py, that are set according the .env.dev environment variables.

```
from django.contrib.gis.geos import GEOSGeometry
from django.forms.models import model_to_dict
from django.db import connection

from buildings2.models import Buildings
from scripts.p1.myLib import p1Settings

from djangoapi.settings import EPSG_FOR_GEOMETRIES,
   ST_SNAP_PRECISION

def insert3(d:dict):
    #we first get the snapped wkb format for the geometry:
    cur=connection.cursor()
    query="select st_snaptogrid(st_geomfromtext(%s, %s),%s)"
    cur.execute(query, [d['geom'],EPSG_FOR_GEOMETRIES,
       ST_SNAP_PRECISION])
    snapped_wkb_geometry=cur.fetchall()[0][0]

    print(f'snapped_wkb_geometry: {snapped_wkb_geometry}')
```

```
        #now we can check if it is valid as before:
        g=GEOSGeometry(snapped_wkb_geometry, srid=EPSG_FOR_GEOMETRIES)
        if g.valid:
            print("Geometría válida")
        else:
            return {'ok': False, 'message':'Invalid geometry', 'data':
               None}

        #Now we can check if it intersects with another geomtry in the
           same layer
        #check if the geometry intersects any existing building
        query="""
            select id from buildings2_buildings where ST_relate(
                geom,
                %s,
                'T********'
            )
        """
        cur.execute(query, [snapped_wkb_geometry])
        r=cur.fetchall()

        if len(r)>0:
            return {'ok': False, 'message':'The geometry interior
               intersects with the following geometries id', 'data': r}

        d['geom']=g
        b=Buildings(**d)
        b.save()
        d=model_to_dict(b)
        d['geom']=g.wkt
        d['data_creation']=d['data_creation'].strftime("%Y-%m-%d %H:%M:%
           S")
        return {'ok': True, 'message':'Building inserted', 'data': [d]}

    def run():
        d_of_values= {
            'description':'Edificio 1',
            'height':100,
            'geom':'POLYGON((0 0, 10 0, 10 10, 0 11, 0 0))'
        }

        print(insert3(d_of_values))
```

The result is the following:

```
7f4fca71afb7:/usr/src/app# python manage.py runscript scripts.p1.
   buildingsDjangoModels.insertDjango
snapped_wkb_geometry: 0103000020
   E66400000100000005000000000000000000000000000000000000000000000000000000000000000000000000000000244000000

Geometría válida
{'ok': False, 'message': 'The geometry interior intersects with the
   following geometries id', 'data': [(1,), (2,), (3,), (4,), (5,),
   (6,), (7,), (8,), (9,), (10,), (11,), (12,), (13,), (14,), (15,),
```

```
 (16,), (17,), (18,), (19,), (20,), (21,), (22,), (23,), (24,),
(25,), (26,)]}
```

4.8.2 Using filters to get database data as Django model objects

You must use ModelName.objects.filter(conditions or lookoups) to get a filter. The filter do not extract anything from the database, do not hits the database. To force the extraction of objects, you must evaluate the filter, for example with list.

```
from appdesweb.models import Buildings  # Import the model
from django.forms.models import model_to_dict #to convert objects to
    dicts

filterById = Buildings.objects.filter(id=1) # Get the filter
l=list(filterById) # Evaluate the filter and gets a list of objects
b=l[0]
print(b) # Print the object representation string
print(b.description)
print(b.area)
print(model_to_dict(b)) # Print the object data as dictionary
```

You can perform much more complicated filter. In the following example you will get some examples:

```
from appdesweb.models import Buildings
from django.forms.models import model_to_dict #to convert objects to
    dicts
#from django.contrib.gis.geos import GEOSGeometry  # deactivate in
   windows. You don have GEOS
#from django.contrib.gis.db.models.functions import Distance #
   deactivate in windows. You don have GEOS

#using django lookups https://docs.djangoproject.com/en/5.1/ref/
   models/lookups/
#using spatila lookups
#https://docs.djangoproject.com/en/5.1/ref/contrib/gis/geoquerysets
   /#std-fieldlookup-contains_properly
#using geodjango lookups
# Filtrar edificios con área menor que 100 (less than --> lt,
   greater than --> gt)

filterById = Buildings.objects.filter(id=1)

smallBuildings = Buildings.objects.filter(area__lt=100)

filteringBytext = Buildings.objects.filter(description__icontains='
   edificio')

# deactivate in windows. You don have GEOS
#combinningFilters = Buildings.objects.filter(area__lt=100,
   description__icontains='edificio')
#filteringByGeometry = Buildings.objects.filter(geom__distance_lte=(
   GEOSGeometry('POINT(-74 38)'), 5))
#intersection = Buildings.objects.filter(geom__intersects=
   GEOSGeometry('POLYGON((0 0, 0 10, 10 10, 10 0, 0 0))', srid
   =25830))
```

```
#interiorIntersection = Buildings.objects.filter(
    geom__contains_properly=GEOSGeometry('POLYGON((0 0, 0 10, 10 10,
    10 0, 0 0))', srid=25830))

#all the results are querysets: https://docs.djangoproject.com/en
    /5.1/ref/models/querysets/
#to access to the results you can convert the queryset to a list
l=list(filterById)
print(l)

p=l[0]

print(p.id)
print(p.area)
#print(p.geom)

#from django.contrib.gis.geos import GEOSGeometry

#create the geometry with geos to be able to calculate the area and
    length
#g=GEOSGeometry(p.geom, srid=25830)
#print(g.area)
#print(g.length)
print(model_to_dict(p)) #Print a dict
```

4.8.3 Update

To update, the properties of an object of the model are the field names of the table. You simply have to change the values of the properties of the object and call to save():

```
from appdesweb.models import Buildings  # Asegúrate de importar el
    modelo correcto

filterById = Buildings.objects.filter(id=1)

l=list(filterById)
print(l)

p=l[0]

print(p.id)
print(p.area)
#print(p.geom)

p.area=800
p.description='Edificio 1'
p.save()
print(p.description)
```

4.8.4 Delete

To delete an object model, get it by evaluating a filter, and call the method delete():

```
from appdesweb.models import Buildings  # Asegúrate de importar el
   modelo correcto

filterById = Buildings.objects.filter(id=1)

#all the results are querysets: https://docs.djangoproject.com/en
   /5.1/ref/models/querysets/
#to access to the results you can convert the queryset to a list
l=list(filterById)
print(l)

p=l[0]

print(p.id)
print(p.area)
#print(p.geom)

p.delete()
```

4.9 Evaluation 1

4.9.1 Part 1. Theorical written exam. Value 2 points

In this exercise you can be asked about the following:

- How to import packages and modules.
- Create and use Python venvs.
- How to manipulate Python lists, dictionaries and json strings.
- How to connect with Docker Containers and execute commands inside.
- Create tables and fields.
- Create classes to insert, delete, update or select in a given table in PostGIS with Python
- Any theoretical concept explained.

4.9.2 Part 2. Practical project. Create functions to modify your own Postgis database. Value 1 point

The goal of this exercise is to create classes with methods to insert, delete, update and select data from your project tables. You have to think in which PostGIS tables are you going to manage in your own project, and create them in a new database, in a Docker PostgreSQL service.

You can use any code from your teacher or other sources, but you have to understand it.

You have to create at least three tables, with at least 5 fields plus the geom field. You must use a different geometry type in each table: point, linestring and polygon:

- Create a new database in the docker container, and at least three tables.

- Keep all the sql code in a file iniddb.sql. You will learn where to put this code in order to create it to create everything on start the postgres container first time.
- Create a package for each table into the folder djangoapi/p1. A package is a folder with the __init__.py file.
- Create a module for each operation over the table: insert, delete, update, selectAsTuples, selectAsDicts. Select as tuples returns, given an id, a list with a tuple with the row that match with the id. selectAsDicts the same but returns a list with a dictionary.
- Optionally you can put all the functions of the same table in the same module. Optionally, and even better, you can put all these operations in a class.
- You must execute all functions with the container interpreter.
- All operations must return a Python dictionary. With the fields ok, message and data fields:

  ok: True or False, depending if you found a problem in the operation or not.

  message: A message description.

  data: A list with the operation data results. If no results None. If results, is a list of dictionaries.

  For example: {ok:true, message: Data inserted, data: [{id:253}]}.

- All the methods, or functions, must receive a Python dictionary with the necessary values to perform the operation:

  insert: Receives a dictionary with the name and values of the fields. Must return, in the *data* field of the returned dictionary, a list with a dictionary with the id of the new row. Eg. {ok:true, message: Data inserted, data: [{id:253}]}.

  selectAsTuples, selectAsDicts: Receives a dictionary with the id to select: {id:25}. Must return, in the *data* field of the returned dictionary, a list with a dictionary, or tuple with the row data. Eg. {ok:true, message: Data retrieved, data: [{field: value, field: value, ...}]}.

  delete: Receives a dictionary with id to delete: {id:25}. Must return, in the *data* field of the returned dictionary, a list with a dictionary with the number of deleted rows: {ok:true, message: Data deleted, data: [{rows_deleted:1}]}

  update: Receives a dictionary with with the name and values of the fields. In the dictionary must be the id of the row to update. Must return, in the *data* field of the returned dictionary, a list with a dictionary with the number of updated rows: {ok:true, message: Data updated, data: [{rows_updated:1}]}.

- You must round all the geometry coordinates to 0.0001 decimals in every insert or update.
- You must check if the geometries are valid before to insert them.
- Reject polygons that intersects with other polygons in the same layer.
- Reject linestrings than intersects with other linestrings in the same layer
- All points must be inside of a polygon of the polygon layers. Reject points outside any polygon (st_within).

- In the p1 folder, create a main.py moduile to execute with parameters any function or method you have.

Do the same with Django Models:

- Create a Django app for your project inside the django-api-template project.
- Create the models for your app.
- Migrate to create the tables into the database.
- Create one model for each table you have.
- Create a class for each table with the methods: insert, delete, update, selectAsTuples (optional), and selectAsDicts. It is the same that you already did with the psycopg library but with the Django models. For example, in the case of the Buildings model, you must have have a buildings.py module, with the Buildings class. The Building class will have the methods insert, delete, update, selectAsTuples, and selectAsDicts. All them have to receive and return the same that for the psycopg case: dicts. Example:

```
djangoapi/scripts/p1/buildigns.py
from myDjangoApp import Buildings as BuildingsModel
class Buildigns:
    def insert(self, d):
        b = BuildignsModel()
        b.description = d['description']
        ...
        ...
        b=save()
        return {ok:True, message:'Todi ok', data:[{id:b.id]}}

    def update(self, d):
        id=d['id']
        b=list(BuildingsModel.objects.filter(id=id))[0]
        b.description = d['description']
        ....
        b.save()
        {ok:true, message: Data updated, data:[{rows_updated
            :1}]}

    ...
```

- In the case of the method selectAsTuples (optional), you can easily manually to build a tuple with the model instance (a Python object) like this:

```
b=list(Buildings.objects.filter(id=id))[0]
tup=(b.description, b.area, ..., rest of the field values)

And return this tuple in a list

return JsonResponse({'ok':True, 'message': 'Building Retriewed',
    'data': [tup]}, status=200)
```

- To implement selectAsDicts, you need to convert a model instance (a Python object) to a dictionary. You can also to do it manually, similar to the above example:

```
b=list(Buildings.objects.filter(id=id))[0]
di={'description':b.description, 'area':b.area, ..., rest of the
    field and values}

And return this dictionary in a list

return JsonResponse({'ok':True, 'message': 'Building Retriewed',
    'data': [di]}, status=200)
```

But to convert a model to a dict you can use the Django function model_to_dict. Here you have an example of how to use it, to get one building, or all of them, convert the objects to dict and return them in a list. For example:

```
from django.forms.models import model_to_dict

class Buildings:
    def insert(self, d):
        ...

    def update(self, d):
        ...
    #GET OPERATIONS
    def selectAsDicts(self, d):
        id=d['id']
        l=list(Buildings.objects.filter(id=id))
        if len(l)==0:
            return JsonResponse({'ok':False, "message": f"The
               building id {id} does not exist", "data":[]},
               status=400)
        b=l[0]

        d=model_to_dict(b)
        return JsonResponse({'ok':True, 'message': 'Building
           Retriewed', 'data': [d]}, status=200)

    def selectallAsDicts(self):
        l=Buildings.objects.all()
        data=[]
        for b in l:
            d=model_to_dict(b)
            data.append(d)
        return JsonResponse({'ok':True, 'message': 'Data
           retrieved', 'data': data}, status=200)
```

- Use the folder djangoapi/scripts/p1 to create all the code.
- You can not use the Django models without to initialize Django. To do this you must execute python manage.py.
- To execute an script you have to use a Django installed app, called runscript:

```
python manage.py runscript script_without_extension --script-
   args param1 param2
```

The script must have a function called run, with the code to execute. You have an example in djangoapi/scripts/001_hello_script.py.

You can also pass parameters to the script, so you can create a main2.py file to load an execute any file from one point, as you did with main.py. But this is optional.

- To test the operations over a table you could have a script similar to the following:

```
djangoapi/scripts/p1/testBuildings.py
from scripts.p1.buildings import Building
d={'id':10,'descripcion':'aa, ....}
b=Building()
print(b.insert(d)) #We pass the id but it is not used in the
    method
print(b.update(d))  #We pass more data than the id but they are
    not used in the method
print(b.selectAsTuples(d)) #Optional. We pass more data than the
     id but they are not used in the method
print(b.selectAsDicts(d)) #We pass more data than the id but
    they are not used in the method
```

Delivery:

- Upload the complete project, compressed as *exercise1.zip*, to the subject shared space in Poliformat.
- Upload a file called *groupMembers.txt*, where you specify name and surname of all the members of the group.
- All the members have to upload the same two files: *groupMembers.txt* and *exercise1.zip*.
- You will show the project to the teacher. You have to have prepared a demonstration to execute all the functions, that is function calls with all the parameters ready. Of course all the functions have to work.
- You will have to answer correctly the teacher questions about the code to obtain the whole note of the exercise.

CAPÍTULO 5

Setup a Django API with Docker

5.1 Introduction

All the code in this section is in the repo: https://github.com/joamona/django-api-template.git, and all the steps done bellow have been done for you. All you have to do is to start the services and start modifying the django project that is al ready working in the repository. In the Geospatial Content Managers subject, you will see which are the steps to dockerize a Django API, here you only have to use it.

Dockerize a Django APi consists in the steps described bellow. All this steps are commands that are written in a file called Dockerfile. The Dockerfile instructions allways do the same: downloads an existing template image, install the required dependencies, and copy the Django API code.

1. Download a Docker Python image from https://hub.docker.com/_/python. There you have many versions, we are going to use the alpine versions, known because they are smaller. This is done with the command *FROM python:3.12-alpine*.
2. Install the dependencies of the system. For example Psycopg2 and GeoDjango need some libraries be installed y the Linux system of the image. This is done with the command *apk*. It is the equivalent to *apt-get* in a Ubuntu Linux system.
3. Install the Python dependencies with pip and the requirements.txt file.
4. Copy the Django API code.

So to dockerize a Django API you need:

1. A Django API.
2. Where the manage.py file is, of the Django Project, put the files: Dockerfile, requirements.txt and initdb.sh. This files are given by the teacher.

We are going to build the image with the *docker compose* command. Up to now we have used the docker hub images as they are in the docker-compose.yml file. For example with the following:

```
services:
  pgadmin4:
    image: dpage/pgadmin4:8.3
    ...

  geoserver:
    image: docker.osgeo.org/geoserver:2.24.2
    ...
```

The previous listing downloads the images, create a container and starts the service. But now we want download an image, install requirements, and put into it the Django API code. But this is not enough, we need to specify the command to start the service into the container, this can be one of the following options:

```
# For developing
python manage.py runserver

# For production
gunicorn django-project-name.wsgi:application --bind 0.0.0.0:8000
```

The first option is only to develop, but is not multi-user, and gives the error traces to the user. The second option is the one to use in production. gunicorn is a Python application to serve Django APIs in a professional way. It must be installed with pip in the image, that is, it must appear in the requirements.txt file.

Instead of using the Python image we are going to build it. To do this instead of define the service like normal, because we only get an image with python:

```
services:
  djangoapi:
    image: python:3.12-alpine
```

You must specify that you want to build it, in the following way:

```
services:
  djangoapi:
    build:
            context: ./deswebapi
```

deswebapi must be a folder where the docker-compose.yml file is, in order to be able to find it. The content of the folder is the Django project plus three files: the Dockerfile (with the instructions to build the image), the requierements.txt, and the initdb.sh.

The complete code of the Django API service in the docker-compose.ylm is, for developing purposes:

```
services:
  ...
  djangoapi:
    restart: "no"
    #the folder of the Django project
    build:
      context: ./djangoapi
      args:
        - USER_ID=${DJANGOAPI_USER_ID}
        - GROUP_ID=${DJANGOAPI_GROUP_ID}
        - USERNAME=${DJANGOAPI_USERNAME}

    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./djangoapi:/home/${DJANGOAPI_USERNAME}
    ports:
      - 127.0.0.1:${DEVELOP_DOCKER_DJANGO_API_FORWARDED_PORT}:8000
    env_file:
      - .env.dev
    networks:
      - postgis
```

For production purposes:

```
services:
  ...
djangoapi:
    restart: unless-stopped
    #the folder of the Django project
    build:
      context: ./djangoapi
      args:
        - USER_ID=${DJANGOAPI_USER_ID}
```

```
        - GROUP_ID=${DJANGOAPI_GROUP_ID}
        - USERNAME=${DJANGOAPI_USERNAME}
    command: gunicorn deswebapi.wsgi:application --bind 0.0.0.0:8000
    volumes:
      - ./deswebapi:/usr/src/app
    ports:
      - 127.0.0.1:${UPVUSIG_DOCKER_DJANGO_API_FORWARDED_PORT}:8000
    env_file:
      - .env.prod
    networks:
      - postgis

    #to ckeck if postgres is already ready. This service
    #will not start up to the posgis service is ready
    depends_on:
      postgis:
        condition: service_healthy
```

On executing docker compose up, Docker will need the folder ./djangoapi with the files Dockerfile, the requierements.txt, and the initdb.sh. The content of these files are:

./deswebapi/Dockerfile: Please read de comments:

```
# pull official base image
FROM python:3.12-alpine

# ARG USER_ID sets the USER_ID variable in the Dockerfile,
#    reading the value from the USER_ID arg variable.
# This arg variable is set in the djangoapi->build->args section
#    of the docker-compose.yml file
ARG USER_ID
ARG GROUP_ID
ARG USERNAME

# Crear grupo y usuario con los mismos UID y GID que el host
RUN addgroup -g $GROUP_ID $USERNAME && \
    adduser -D -u $USER_ID -G $USERNAME $USERNAME

# set work directory
WORKDIR /home/$USERNAME

# set environment variables
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1

# install psycopg2, and geodjango dependencies
RUN apk update && apk add postgresql-dev g++ gcc musl-dev libpq-dev
   wget pango gdal-dev geos-dev proj-dev gcc musl-dev \
     libffi-dev python3-dev
RUN apk add --update --no-cache --virtual .tmp-build-deps \
     libc-dev linux-headers

# install dependencies
RUN pip install --upgrade pip

# Set the user to copy the files to the container,
```

```
# so the owner of the files is the user and not root
# and the user in the host match with the user in the container
USER $USERNAME
COPY . .
# Set the user root to install the dependencies. Otherwise the
# the python dependencies are not found latter in the container
USER root
RUN pip3 install -r requirements.txt
# Set the user again to be the defaut user in the container
USER $USERNAME
```

./deswebapi/requirements.txt:

```
asgiref==3.8.1
click==8.1.7
Django==5.0.4
gunicorn==22.0.0
h11==0.14.0
packaging==24.0
psycopg2==2.9.9
sqlparse==0.5.0
typing_extensions==4.11.0
requests
django-cors-headers
djangorestframework
six
debugpy
cryptography
django-rest-knox
pgOperations
datetime
django-filter
drf-access-policy
```

./deswebapi/initdb.sh:

```
#/bin/sh

#create the initial Django tables
python manage.py makemigrations
python manage.py migrate
#create the Django superuser
DJANGO_SUPERUSER_PASSWORD=${DJANGO_SUPERUSER_PASSWORD} python manage
    .py createsuperuser --noinput --username ${
    DJANGO_SUPERUSER_USERNAME} --email ${DJANGO_SUPERUSER_EMAIL}
```

The initdb.sh migrates the database and creates de *admin* user. This last step need some environment variables set in the image. This is done by docker compose up command, thanks to the environment variable files, that are loaded because the lines in the .yml file:

In develop environment:

```
...
env_file:
  - .env.dev
...
```

In production environment:

```
...
env_file:
  - .env.prod
...
```

5.2 Environment variables

Environment variables are variables that are set in the operative system. Set an environment variable in Linux is very easy:

```
# Set the variable A to 20
joamona@upv7cores:~/$ export A=20

# The value of a variable is retrieved with $variable
joamona@upv7cores:~/$ echo $A
20
```

Environment variables are commonly used to no write some values as username, password, database, etc, directly in the source code. This has two advantages: you set all the variables in a file, and you can hide this file. All the configuration variables are in the same point, easy to locate and change.

In Python, to access to an environment variable:

```
import os
A=os.getenv('A','Not found')
print(A)
```

El comando *os.getenv* will search for the A variable in the SO, and if it is not found will return *Not found*. The second parameter is optional.

The repo sets the following environment variables.

The variables in the file .env only are used in the .yml file, only contains ports, and allow to change the ports of the services without modifying the .yml file:

```
#User used to create files in the djangoapi container
#This is used to match the user in the host machine with the user in
    the container
#This is use to avoid permission issues when creating files in the
   container
#This permissions are only applicable if you develop in Linux
#You can get your user id in linux with: id -u
#You can get your group id in linux with: id -g
DJANGOAPI_USER_ID=1000
DJANGOAPI_GROUP_ID=1000
DJANGOAPI_USERNAME=joamona

DEVELOP_DOCKER_DJANGO_API_FORWARDED_PORT=8000
DEVELOP_DOCKER_POSTGIS_FORWARDED_PORT=8440
DEVELOP_DOCKER_PGADMIN_FORWARDED_PORT=8050
DEVELOP_DOCKER_GEOSERVER_FORWARDED_PORT=8080

UPVUSIG_DOCKER_DJANGO_API_FORWARDED_PORT=71001
UPVUSIG_DOCKER_POSTGIS_FORWARDED_PORT=7102
UPVUSIG_DOCKER_PGADMIN_FORWARDED_PORT=7103
UPVUSIG_DOCKER_GEOSERVER_FORWARDED_PORT=7104

GISSERVER_DOCKER_DJANGO_API_FORWARDED_PORT=8901
```

```
GISSEVER_DOCKER_POSTGIS_FORWARDED_PORT=5440
GISSEVER_DOCKER_PGADMIN_FORWARDED_PORT=5441
GISSEVER_DOCKER_GEOSERVER_FORWARDED_PORT=8080
```

The file .env.dev and the file .env.prod are used in developing or production environment. They set the environment variables in the containers. The containers need some environment variables to work, this is explained in each image documentation, for example for the postgis service we use the image postgis/postgis:16-3.4, and you can see what variables it needs in https://hub.docker.com/r/postgis/postgis/. The same for the rest of images.

The content of the .env.dev file is the following:

```
#DEVELOPMENT ENVIRONMENT VARIABLES FOR DOCKER CONTAINERS

### SERVICE POSTGRES ###
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=desweb
POSTGRES_HOST=postgis
POSTGRES_PORT=5432

### SERVICE PGADMIN4 ###
PGADMIN_DEFAULT_EMAIL=vagrant@vagrant.com
PGADMIN_DEFAULT_PASSWORD=vagrant

### SERVICE GEOSERVER ###
GEOSERVER_CSRF_DISABLE=false
GEOSERVER_CSRF_WHITELIST='myserver.com,myserver.com/geoserver/,
   localhost:7104'
CORS_ENABLED=true
CORS_ALLOWED_ORIGINS=*
CORS_ALLOWED_HEADERS=*
CORS_ALLOWED_METHODS=GET,POST,PUT,HEAD,OPTIONS
PROXY_BASE_URL=https:myserver.com/geoserver/
INSTALL_EXTENSIONS=true
STABLE_EXTENSIONS=wps,csw,dxf,importer,inspire
EXTRA_JAVA_OPTS=-Xms1G -Xmx2G
GEOSERVER_ADMIN_USER=admin
GEOSERVER_ADMIN_PASSWORD=geoserver

### DJANGO API SERVICE
DJANGO_SECRET_KEY='django-insecure-g49)#p#gugd(p6v*@a$l(d#^
   rd2wdtdmfl@cuwch^(!yzw)qj*'
DJANGO_SUPERUSER_USERNAME=admin
DJANGO_SUPERUSER_PASSWORD=admin
DJANGO_SUPERUSER_EMAIL=joamona@cgf.upv.es
DJANGO_ALLOWED_HOSTS=*
DEBUG=1
CORS_ALLOWED_ORIGINS=*
```

The .env.dev and de .env.prod files are divided in four sections to define the variables needed for each container.

The last section, *DJANGO API SERVICE*, is used to set those variables in the settings.py file. The variables in the *SERVICE POSTGRES* section are also used in the settings.py, to connect with the database. Pay attention that the host is *postgis*, which is the service name of the postgis service in the

.yml file. In the next listing you will see how this variables are used in the settings.py file to connect with the database:

```
import os
...

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql_psycopg2',
        'NAME': os.getenv('POSTGRES_DB'),
        'USER': os.getenv('POSTGRES_USER'),
        'PASSWORD': os.getenv('POSTGRES_PASSWORD'),
        'HOST': os.getenv('POSTGRES_HOST'),
        'PORT': os.getenv('POSTGRES_PORT'),
        'OPTIONS': {
            'options': '-c search_path=public',
        }
    }
}

...
```

5.3 Use the django-api-template repository to create an API

Mostly the goal of using Docker is not to have to install anything. You only have to install Docker, clone a repository, where all the code is: the docker-compose.yml, and the Python project, change the environment variables, and execute docker compose up. Docker is going to download all the images, modify them with the environment variables, create the volumes, initialize databases, copy code to images, create containers and start the services. You will have everything ready in minutes, to use the services, to start, or continue, coding an API, or making a web.

You even do not need Python, neither to create a Python venv, everything is inside the djangoapi container.

To create a Django API with docker we are going clone the following repo: git clone https://github.com/joamona/django-api-template.git. It has postgis, geoserver, pgadmin and a django project ready to start coding. All the environment variables of the file .env and .env.dev are used to start the container services. All you have to do to customize that services is to change the environment variables in that files.

Once cloned the repo, the django-api-template folder will be created with all the files. You can rename this folder to a more appropriate name for your project, for example flowers-api, if your project is going to manage flowers.

```
joamona@upv7cores:~/temp$ git clone https://github.com/joamona/
   django-api-template.git
joamona@upv7cores:~/temp$ mv django-api-template/ flowers-api
joamona@upv7cores:~/temp$ cd flowers-api/
```

Execute one of the following files to create the folders that pgadmin4 needs:

```
# for linux
./pgadmin_create_folders_linux.sh

#for Windows
.\pgadmin_create_folders_windows.sh
```

You will see the folders pgadmin4, and pgadmin4_prod in the project folder, where the .yml is.

Now get into the project folder and execute docker compose up (remember in Windows you first have to start Docker Desktop).

Now you can check the services:

- pgadmin: http://localhost:8050.
- geoserver: http://localhost:8080.
- Django API: http://localhost:8000/buildings/hello_world/.

Usually first time you start the services, the volumes are created, and the djangoapi service fails because it start before the database is ready. Wait to all the services finish to start, cancel and start them again.

Next step is to migrate the database. Enter into the container, in the folder /home/$yorusername and type ./initdb.sh. This will create the tables in the database that django needs to manage users, and create the admin user.

Now you have everything working and ready to code, but you need to know:

- The command python manage.py runserver is automatically executed, and you can modify the Django project in the folder djangoapi. Each time you create, or modify file of the project, the service reloads the django application, as if you have Pyhton in your host. But it is not, at least it is not necessary, and if you have one, it is not been used. You are using the interpreter in /usr/local/bin/python in the container. There all the packages in requirements.txt have been installed.
- You can not execute python manage.py command, from the host as you are using the pyhton in the container. To use this command, you have to enter into the container.
- The Visual Studio Code do not understand the imports as the Pyhton libraries are in the container. This is not necessary as the project works, but, VS is not going to help you on coding.

To solve the last issue you have to open the code of the Django project inside the container with Visual Studio Code. In this way, as the python interpreter is in the same container VS is going to understand the Django imports.

5.3.1 Open the Django project code, inside the container with Visual Studio Code

We do this to VS understand the project Python imports, as we do not have a venv in the host. Do the following steps:

- Start the services: docker compose up.
- Open VS.
- Press Ctrl + Shift + p.
- Paste the following: Dev Containers: Attach to Running Container.
- VB will ask you to install some extensions. Say yes.
- Select the container yourProjectName-djangoapi-1.
- A new VS code window is started.
- In the terminal, change to the folder /home/$username, where the source code is.

- Select the interpreter: Ctrl + Shift + p, and type python select interpreter, and select the interpreter int he container /usr/local/bin/python. In this way VS will help you to code.
- Now, you can modify the source files with this VS, and create new Django apps from the VS connected to the container.
- To create a new app, in the terminal, in the VS connected to the container, type: python manage.py startapp mynewapp. Something you could not do before.

5.3.2 Installed apps

The django-api-template has the following apps installed:

admin : The Django admin site: http://localhost:8000/admin/. User admin, psw: admin. Check that you can add a building by drawing it in the site.

swagger : http://localhost:8000/swagger/. User admin, psw: admin.

core: for the user management and the common libraries.

codelist: for the codelist, or domain tables: http://localhost:8000/codelist/hello_world/

buildings: demo app. It has a model, a serializer and a viewset to manage the buildings. The model includes the *geom* field which is of polygon type. http://localhost:8000/buildings/hello_world/.

5.4 Understanding the names of containers, volumes and networks

If you clone the django-api-template folder, rename the folder to other-api, and perform the same operations. Docker is going to create new images and containers. Docker names the containers as the project name folder + the service name + n, being the n the number of the container of that project, usually, only 1. The names will be:

- For flowers-api:flowers-api-pgadmin4-1, flowers-api-flowersapi-1, flowers-api-postgis-1, flowers-api-geoserver-1.
- For other-api: other-api‘-pgadmin4-1, other-api-flowersapi-1, other-api-postgis-1, other-api-geoserver-1

The same is applied for the names of Docker volumes (managed by docker, not folder volumes), and networks.

In older versions of docker the names are formed slightly different: project-folder_servcie-name_n.

Respect the volumes. You can have several containers of Postgis, for example, connected each one to its volumes, so the data is independent for all the volumes, or connect all the containers to the same volume, In this case all the containers have access to the same databases. If I create a database from one volume, this database is going to be available in all the volumes.

If the volume is managed by Docker like: postgis-data:/var/lib/postgresql/data, it will have a name that you can know with docker volume ls. Latter you can use it with its name from other container other-project-postgis-data:/var/lib/postgresql/data.

If the volume is in this way ./postgis-data:/var/lib/postgresql/data, this is a folder, and you can connect to it by specifying the absolute path from other service definition: ./home/user/otherproject/postgis-data:/var/lib/postgresql/data

Respect pgadmin4. First you must remember you have to create some folders before starting a container for first time. I always forget it, and you too. If this happens you need to remove the pgadmin4 folder created automatically by the container and create it with the given script. Also you have to remove the pgadmin container, and recreate it again to it to connect to the new volume. Respect the connection, you can connect pgadmin4 to any postgis container, the problem is the host. If the pgadmin service is in the same .yml than the postgis service, the host is the service name: postgis. If you want to connect to other service, you have to connect the pgadmin container to a docker external network.

5.5 Publish your Docker Django API in a server

The normal way of doing this es by using git and github. Your team will use git to create branches (vesions) join the code (merge) and, at the end, you will have a new version on github of the code of the API every week, for example. This code will include the .yml to deploy the services needed for the API as the django-api-template. What you have to do in this scenario are things you already know:

- First time: clone the repo.
- First time: create the docker-compose.prod.yml and the .env.prod files to deploy the services. These files are not in the repo as they are secret. Thy are only in the developer computer and the server.
- Start the services. They are served in several ports.
- Create a virtualhost for Apache.
- Enable the virtualhost.
- Install the SSL certificates for the virtualhost.
- Create the apache proxies in the SSL virtualhost to publish your services with aliases: /geoserver/, /api/, etc.
- Tests

CAPÍTULO 6

Django urls and views

6.1 Introduction

In this chapter you are going to understand Django philosophy. You have to learn where the code goes, and from which Django classes to inherit in order to get the starting functionality. You also will get some libraries, and base classes, to be faster in coding. Everything has been included in the django-api-template project. Also you have a working example in the app *buildings* of the project.

At the moment you are able to modify a PostGIS database using your functions, and the Django models, but you need to be able to execute these functions through Internet. This is done using the HTTP protocol and HTTP requests. To manage HTTP requests you need a web server. This web server listen HTTP requests from internet, on the 80 port by default, and is able to execute scripts. The most used HTTP servers at this moment are Apache2 and Nginx. We are going to use Apache2, but not at this stage. You are going to create an HTTP API service. This is a service which is always listening in a port the http requests. At the beginning you will use the Django development server, to serve your Django API, but in production you have to replace it by Apache or Nginx. The request are sent to the API in two ways: GET and POST. You can perform GET requests with a web browser, by simply visiting the API site, but you need a program to perform POST requests (Postman, Python with requests, or JavaScript in a web browser. See the section 6.9, page 83. GET gets information form the server, and POST is used to send form data to the server to change the database. POST is dangerous as it send the cookie data.

You need a program always running, a service, who be able to receive HTTP requests and execute one function or an other, the functions you have already done. Django does that and much more. We are going to use Django to execute one function or other, depending of the requested url.

In a real case you will have a server linked to a domain, with the Apache2 HTTP server installed. The domain will bring the request to your server by typing *http://myDomain.com/*. In your server you probably have several apps and web pages, each of one has an Apache alias. By typing the Apache alias after the domain Apache will execute the appropriate script. For example */api/* will execute a Python Django app. But the Django app also needs to know which function to execute, so it is necessary to write something more, for example *building/insert/* could execute the *insert* function in the app *buildings*. See some more examples:

- http://myDomain.com/api/buildings/insert/ –>Will execute the function buildings.insert
- http://myDomain.com/api/buildings/select/125/ –>Will execute the function buildings.select selecting the building 125
- http://myDomain.com/api/buildings/delete/ –>Will execute the function buildings.delete
- http://myDomain.com/api/buildings/update/ –>Will execute the function buildings.update

You will configure a real server in the subject Content Managers and Smart Cities, but this is the last step on the developing process. To develop you will use the Django server in first place. Your API will be available in https://localhost:8000.

- http://localhost:8000/buildings/insert/ –>Will execute the function buildings.insert
- http://localhost:8000/buildings/select/125/ –>Will execute the function buildings.select selecting the building 125
- http://localhost:8000/buildings/delete/ –>Will execute the function buildings.delete
- http://localhost:8000/buildings/update/ –>Will execute the function buildings.update

The Django server serves and listen by the port 8000, with the command python manage.py runserver. You must not use the alias */api/* in the url because that is only for production. If you want to change the port 8000, you can do so with: python manage.py runserver 0.0.0.0:newPort.

To link urls to Python code two new type of files came to play: the urls.py and the views.py files, Figura 6.1.

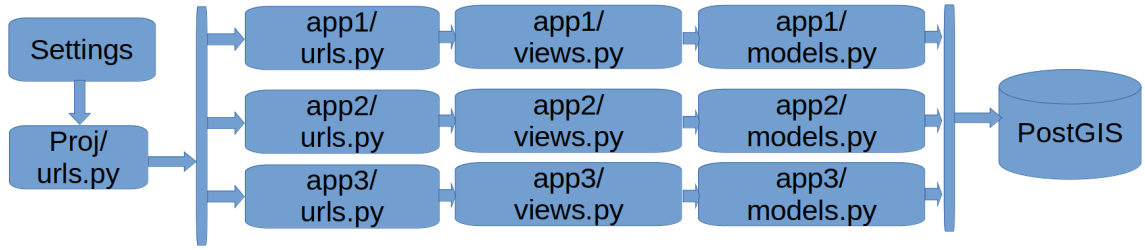

Figura 6.1: Django data flow schema

6.2 Django apps

As it is said, Django apps are thought to organize the data management. Each app can manage as many tables as you need. Each app is a folder, created by Django with python manage.py startapp appname. All apps have the files models.py, views.py and urls.py. You only have to add the models in the models.py, the views classes to the views.py and the urls to the urls.py of the app. Remember to add the app/urls.py to to the djangoapi/djangoapi/urls.py in order they be loaded. Also remember to add the appname to the INSTALLED_APPS list in the djangoapi/djangoapi/settings.py module.

All Django projects have a core app, to manage users, authentication, and common libraries.

6.3 Create an app for your project

The django-api-template cams with tree apps:

core: With the co.mmon libraries and for the users management.

codelist: Empty but thought to manage the coded lists for all the tables.

buildings: As an example of use.

To use django-api-template project you must follow the following steps:

1. Clone the django-api-template project, and setup it by following the steps in chapter 5.
2. Create an app for your code: python manange.py startapp myapp
3. Add your app name to the settings.py file, into the INSTALLED_APPS list.
4. Create the urls.py for the app, and add the urlpatterns list.
5. Add your app urls to the djangoapi/urls.py
6. Add your models to the models.py of your app.

7. Migrate the models to create the tables into the database: python manange.py makemigrations, and python manage.py migrate.
8. Create the views. One view for each model, and inherit from BaseDjangoView.
9. Define the methods insert, update, delete, selectone, selectall in all of your child classes.
10. Add the two urls patterns to the urlpatterns list of your app urls.py.
11. Test your views.

As a example the *buildings* app is presented. All the code presented in this document is in the django-api-template project. I am not going to copy much code from the django-api-template project here, because I do not want to maintain two versions of the code, and also it is too long. You will be redirect to the source code.

6.4 Urls

To link urls with python code there are the files urls.py. Each url in the urls.py file will execute a method in a View class in the views.py file. Urls are the intermediaries between Views and Internet. Views are who do the work in the database.

All Django projects have a main urls.py. The content is the following:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

This means to access to the admin site urls, you must type http://localhost:8000/admin/, or https://myserver.com/api/a in production, where *api* is an alias for an Apache proxy that redirects the request to the Django api in the server.

On create a new Django app, a new folder is created, with the files models.py and views.py, but not a urls.py. Each app must have a urls.py, with the urls that that app are going to manage. The urls to manage must be in a list called *urlpatterns*. The urls.py file has to be created manually in each app. For example, the django-api-template has the buildings app. I created the urls.py file and added the following:

```
from django.urls import path, include
urlpatterns = [
]
```

Once created the app (buildings), and the file urls.py (buildings/urls.py), you must tell Django to load it, in the main urls.py located in the same place where the settings.py file is (djangoProject/urls.py):

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('buildings/', include('buildings.urls')),
]
```

The previous code will give you access to all the urls in the buildings app, through the *buildings* word in the url. You must type http://localhost:8000/buildings/, or https://myserver.com/api/buildings/ in production, to access to the urls stored in the buildings/urls.py module, now empty.

Each url in the buildings/urls.py will be linked to a view,

6.5 Views

Django views (https://docs.djangoproject.com/en/5.1/topics/http/views/) are classes that inherit from the Django View Class. They perform the operations in the database through the Django models. Django Views classes have the following characteristics:

- Have two methods: post and get that you must override, at least one.
- If you call the url by POST the view will execute its *post* method, and the the GET call will execute the *get* method.
- *post* and *get* Views methods receive a request parameter that is automatically passed by Django. This give you the request objet inside the method implementation.
- The request object have the user objet request.user (an instance of User), who sent the request. If the user is not login jet, you will get an anonymous user.
- The request object has the POST and GET properties with the information sent by the user to the view. Both properties are a dictionary key, value.

Lets see an easy example with a view that is going to be called by GET and is going to respond a json with a message. Our APIs always will respond a json. Add the following to the buildings/views.py file:

```
# Create your views here.
#Django imports
from django.http import JsonResponse
from django.views import View

from .models import Buildings

class HelloWord(View):
    def get(self, request):
        return JsonResponse({"ok":True,"message": "Buildings. Hello
            world", "data":[]})
```

Now link this view to the url /buildings/hello_world/. In the buildings/urls.py add the following:

```
from django.urls import path, include
from . import views

urlpatterns = [
    path("hello_world/", views.HelloWord.as_view(),name="hello_world
        "),
]
```

Now you can test the view with http://localhost:8000/buildings/hello_world/. Note all the urls in the buildings/urls.py have the /buildings/ in the url due to the djangoapi/settings/urls.py, the main urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('buildings/', include('buildings.urls')),   #includes
        buildings.urls, and to access to them you have to put before
         buildings/.
]
```

6.6 Base class for views: BaseDjangoView

Here we are going to create a class that inherits from the Django View class, and it is going to be our base class for all our classes.

I am not going to copy the code here. See the code in the django-api-template/djangoapi/core/myLib/baseDjangoVie
The django-api-template folder is where you have the .yml services to manage them. The djangoapi is the folder where the Django project is. The *core* folder is a Django app. The *myLib* is the folder where I put the common code of all apps.

Read the comments in the django-api-template/djangoapi/core/myLib/baseDjangoView.py module. By inheriting from it you only have to redefine, in the child class, the methods: insert, delete, update, selectone, and selectall. See an example in django-api-template/djangoapi/core/buildings/views.py. The class *BuildigsView* is a child of the class BaseDjangoView.

Once redefined you can call them by adding two lines into the urls.py of your app. See the example in the file: django-api-template/djangoapi/buildings/urls.py

The BaseDjangoView class only have an algorithm to call one of the class methods, depending of the url. The url have to follow the two patterns in urls.py. If the call is GET the get method of the BaseDjangoView class will call a class method depending on the *action* specified in the url. The same by POST.

By inheriting you save to define all times the post and get methods to forward the calls to the class methods, You only have to redefine the methods: insert, delete, update, selectone, and selectall methods directly in the child class. You also can add new methods, adding new actions, as is said in the comments of the baseDjangoView.py module. In the class methods you can use the Django models to manage the tables, as explained in chapter 4.

6.7 Example of view based in BaseDjangoView

See an example of use in django-api-template/djangoapi/core/buildings/views.py. The class *BuildigsView* is a child of the class BaseDjangoView. The BuildingsView redefine the the methods selectone, selectall, insert, update, and delete of the BaseDjangoView class, and also the post method, to add a new action, insert2.

The *insert* method uses the geodjango to create a Geos geometry, and snap to a grid the geometry, and check if the geometry is valid, and check if the interior of the geometry intersects with an other geometry in the same table.

The *insert2* method do the same that *insert*, but by using the core/mylib/geometryTools.py module. This has been coded by the teacher and has been created because I realized that using the Geoss library is a bit complicated, overall to update, due to the geometry checks we do.

The *update* method also uses the geometryTools.py module.

The methods selectone and selectall use the Building model to filter the rows and the model_to_dict function to convert the instances to dictionaries, that is the only you can send to the client. Currently, you do not send dictionaries but jsons, but Django is in charged of convert it for you with the JsonResponse class.

Pay attention that thanks to GeoDjango, the geom field contains a geos geometry, and we can get the wkt representation of the geometry with: b.geom.wkt.

6.8 Geometry checks with geometryTools.py

It has two classes:

WkbConversor: Is able to get a binary representation, wkb, of a snaped to grid geometry from a wkt, or geojson string.

GeometryChecks: Is able to check:

- If the geometry is valid.
- Check the relation from the geometry in wkb, and the elements in any table, by using the using the 9IM matrix.
- Check any condition fo the wkb geometry and the geometries in any table. The conditions must return true false, and receive two geometries. Eg: st_intersects, st_contains, st_within, ...

Examples of use in the view BuildingsView, in the file buildings/views.py.

6.9 Sending data to the views: POST or GET

To test your APIs you need to perform POST or GET requests. You can manage it by using Postman, or Python request library.

6.9.1 Postman

Postman (https://www.postman.com/) allow you to store the requests in a *collection* to use them again and again, until you be sure the back-end works properly. Thanks to it you can check you API endpoints without to have the front-end finished.

Fortunately Visual Studio Code has a plugin for it. You will need a Postman account to use it.

6.9.2 Python requests

You can also use Python requests to send post and get data. Here an example to send the username and password to intis a session, and latter how to perform an insert.

```
import requests

def initSession(username, password):
    s = requests.Session()
    URL_BASE = 'http://localhost:8000/'
    d={"username":username, "password":password}
    r = s.post(url=URL_BASE + 'app_login/', json = d)
    print(r.text)
    return s
```

```
URL_BASE = "http://localhost:8000/"
d={"descripcion":"edificio 1", "geomWkt":"POLYGON((727844
    4373183,727896 4373187,727893 4373028,727873 4373018,727858
    4372987,727796 4372988,727782 4373008,727844 4373183, 727844
    4373183))"}

s=initSession(username="editor", password="2023master")

r = s.post(url=URL_BASE + "buildings_insert/", json = d)
print(r.text)
```

6.10 GIS QuerySet API Reference

To use geodjango to check spatial relationships, you must query the official documentation: https://docs.djangoproject.com/en/5.1/ref/contrib/gis/geoquerysets/.

CAPÍTULO 7

Users management and authentication (Optional)

7.1 Introduction

In a real project you only will allow authenticated users to perform some API operations. In our Buildings project, for example, the usual way would be to allow all users to select buildings, but only authenticated users to insert, delete or update buildings, that is to modify the database.

Django provides very useful tools to manage and authenticate users. Usually you will have three types of user: administrators, editors and users:

- administrators: they can get into the administration site and manage other users.
- editors: they can modify data in the database.
- users: they can see the data.

But real word is complex, you probably will have still more groups to classify better your users.

7.2 Create users and groups with the Django admin site

You can create and manage users and groups with Python and with the Django administration site. This is a pre-installed application in all the Django projects. You can access to the admin site by typing http://your-server/admin. In the case of the development server http://localhost:8000/admin Figura 7.1. You can authenticate with the user *admin*, password *admin*. You created this user in a previous step with the command *python manage.py createsuperuser*.

To use Django users framework, you should have:

- Migrated the database first, executing *python manage.py migrate*, only once. See the section 4.4.
- In the INSTALLED_APPS variable, in teh module *settigns.py*, the following apps *django.contrib.auth*, and *django.contrib.admin*. These apps are installed by default.
- In the urls.py of the project folders have imported *admin* from django.contrib, and to add *path('admin/', admin.site.urls)* to the *urlpatterns* list. This is also done by default.

Select the *Groups* link and crate the groups needed, by clicking in the button *Add ... +*, Figura 7.2.

Return to the main page and select the link *Users*. In the new window add a user by clicking in the button *Add ... +*. Add all the necessary user details, Figura 7.3. In the example, the username is *desweb@desweb.com* and the password is *adminadmin*.

To add a user to a group, go back to the main page and select the user, select the group on the left list and pass it to the right list, Figura 7.4.

A user can belong to several groups.

7.3 Create super-user from command

You can create a super-user by typing:

```
python manage.py createsuperuser
```

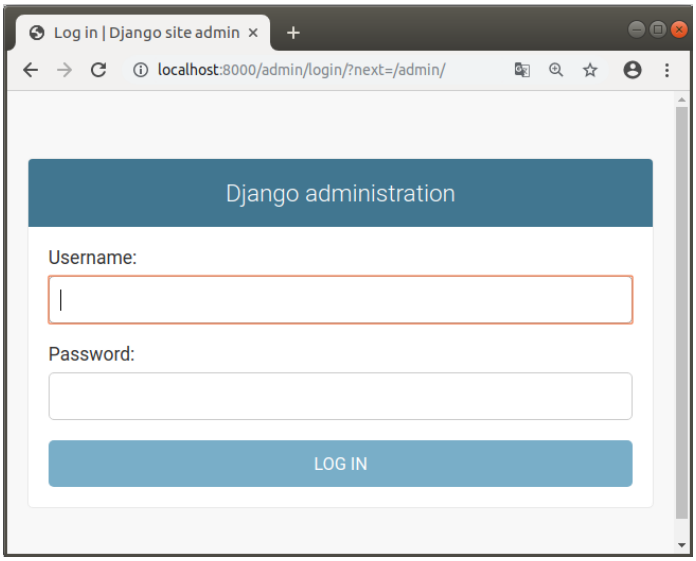

Figura 7.1: Django admin site. Login

7.4 Create normal users from command

You can not add a normal user by executing any shell command. To create normal users, you have to use the Django Admin Site, see the section 7.2, or with Python, see the section 7.5.

7.4.1 Change users password from command

You can change user password swith the following command:

```
python manage.py changepassword <user_name>
```

7.5 Manage users with Python

You probably will need to create new users from a view, without using the Django administration site. You can create a new user by using the Django *User* model

```
from django.contrib.auth.models import User

    ...
    #checks whether or not the user exists
    if User.objects.filter(username=d['username']).exists():
        return {"ok":"false","message": "The user {0} already exists
           ".format(d["username"]), "data":[]}

    #creates the user and returns a User object to manipulate the
       user
    user = User.objects.create_user(email=d["username"], username=d
       ["username"], password=d["password"])
    ...
```

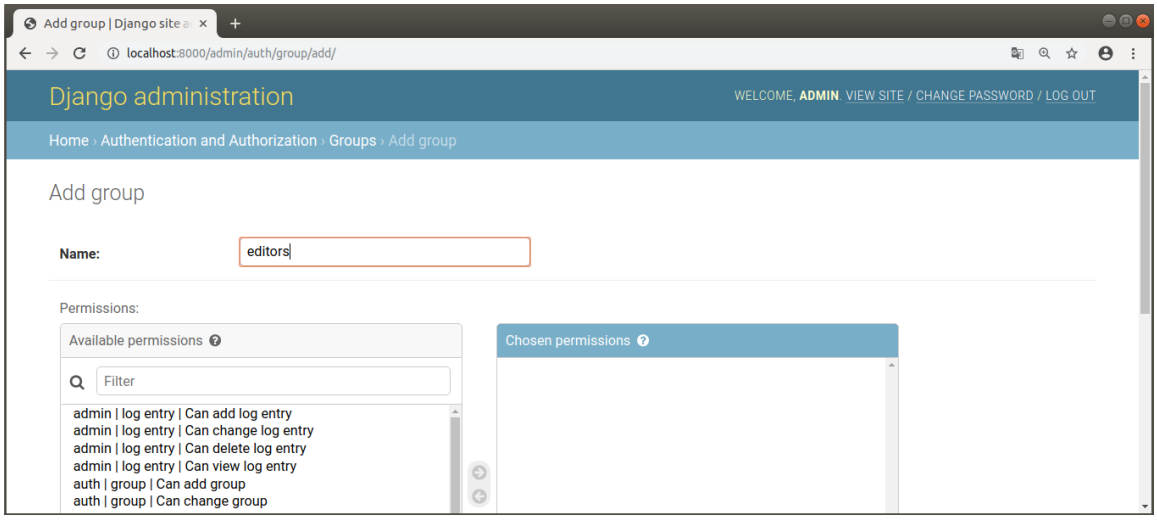

Figura 7.2: Django admin site. Add group

7.5.1 Add a user to a group with Python

You usually will create the users groups manually with the Django administration site, but, if you create users from Internet, you probably will need to classify them in the already created Django groups. In the next listing you have an example of how to do it:

```
from django.contrib.auth.models import Group
from django.contrib.auth.models import User

    ...
    #user = request.user
    #or
    #user = User.objects.create_user(email=d["username"], username=d
        ["username"], password=d["password"])
    #or
    #user = User.objects.get(username='the_username')

    normal_users = Group.objects.get(name='normal_users')
    normal_users.user_set.add(user)
    ...
```

7.5.2 Active - deactive users with Python

If you want temporally disable the login for a user, the best way is to de-active him. By default all new users are active, see the selected check in the Figura 7.4, in the *Active* field. By unselecting this check, the user is inactive. This means that the user, and all his data, exists but he is unable to login. The user will be able to login again at the moment an administrator actives the user.

You can active-deactive users also with Python. In the next listing you have an example of how to deactive an user:

```
from django.contrib.auth.models import User

    ...
    #user = request.user
    #or
```

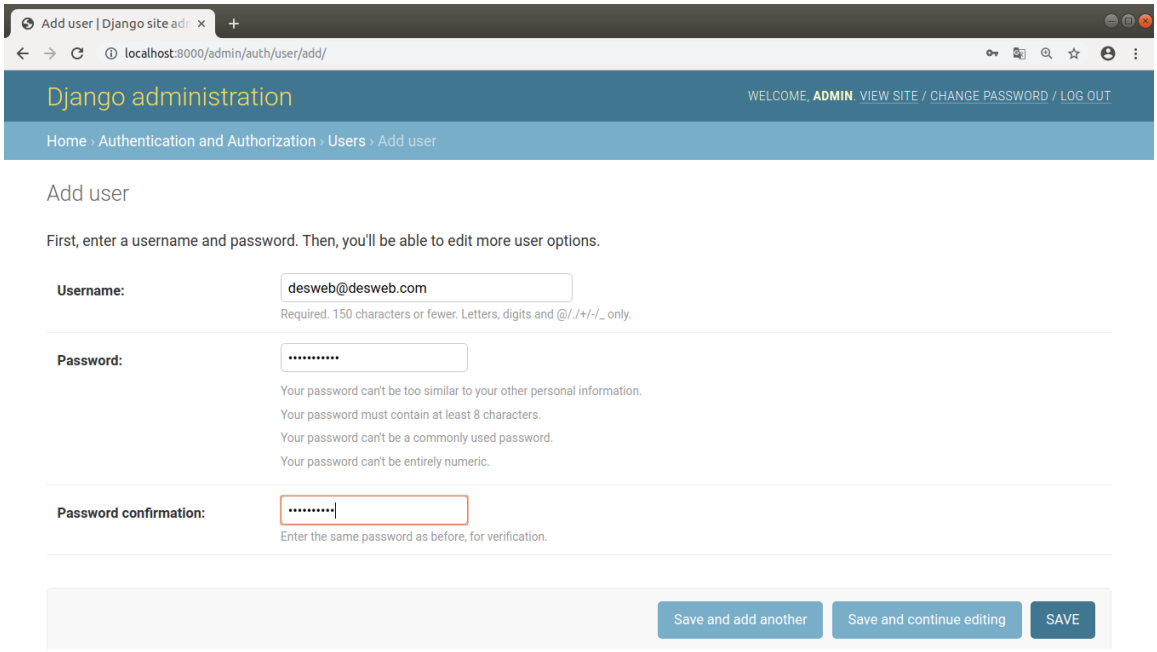

Figura 7.3: Django admin site. Add user

```
#user = User.objects.create_user(email=d["username"], username=d
    ["username"], password=d["password"])
#or
#user = User.objects.get(username='the_username')

user.is_active=False
user.save()
...
```

7.5.3 Change users password with Python

You can change the user password like in the following example:

```
from django.contrib.auth.models import User
u = User.objects.get(username='the_username')
u.set_password('new password')
u.save()
```

7.6 Protect some views from unauthenticated users

Now you have some users you can check if the user is authenticated or not, before to use any view.

To forbid not authenticated users to access to a view you have to import the class *LoginRequiredMixin* from *django.contrib.auth.mixins*, and make your views to inherit from it. In python you can inherit from several classes, and the order is important. You must inherit from LoginRequiredMixin in first time. Lets change the code of the module buildings/views.py to forbid non authenticated users to the view BuildingsView:

```
#djangoapi/buildings/views.py

...
```

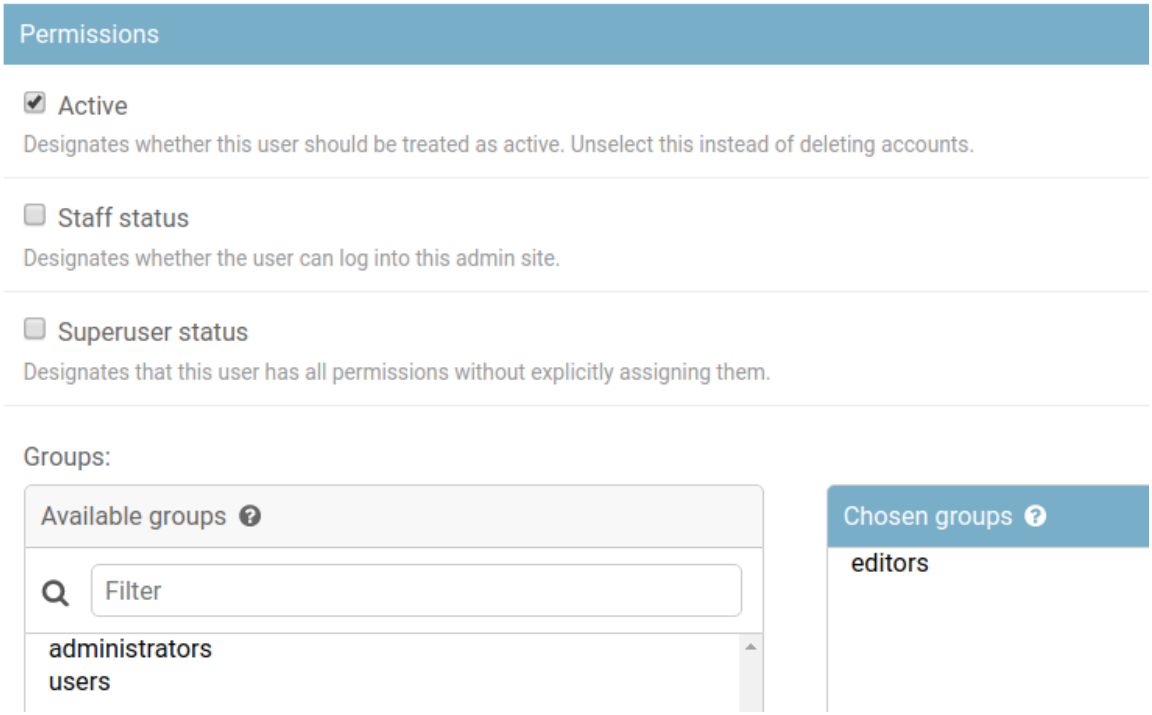

Figura 7.4: Django admin site. Add user to a group

```
from django.contrib.auth.mixins import LoginRequiredMixin
...
class BuildigsView(LoginRequiredMixin, BaseDjangoView):
    ...
```

Now any unauthenticated user is able to use the view. Lets try to get a building: http://localhost:8000/buildings/buildin
You will get the following answer:

```
{"ok": "false", "message": "You are not logged in", "data": []}
```

Django forbids the access and redirects the request to an other view to manage not authenticated requests. This view is called automatically each time a non authenticated user tries to use a protected view. The view return a json. This has been made in three steps:

1. Add the following function view to the file *core/views.py* module.

```
#core/views.py
...
from django.http import JsonResponse

def notLoggedIn(request):
    return JsonResponse({"ok":"false","message": "You are not
        logged in", "data":[]})
...
```

2. Add a url to the file *core/urls.py* to execute the notLoggedIn view function:

```
#core/urls.py

from django.urls import path, include
```

```
from . import views


router = routers.DefaultRouter()
#router.register(r'core', views.BuildingsModelViewSet)

urlpatterns = [
    path("hello_world/", views.HelloWord.as_view(),name="
        hello_world"),
    ...
    path('not_loggedin/', views.notLoggedIn, name="not_loggedin
        "),
]
```

3. Set the variable *LOGIN_URL* to '/not_loggedin/' in the *djangoapi/djangoapi/settings.py* file. This is the url to change when a user tries to do something that requires to be logged in, and he is not. That variable is not set by default:

```
#if you try to use a view without being logged in, redirect to
   the following URL
LOGIN_URL = "/core/not_loggedin/"
```

   All those configurations are already done. If you try to use any of the protected views you will get the following message, and all you have to do is to inherit from LoginRequiredMixin in the view you want to protect.

```
{
    "ok": "false",
    "message": "You are not logged in",
    "data": ""
}
```

   Try it with Postman.

7.7 Authenticate users

To authenticate users we are going to use the Django User model, and the functions authenticate, login and logout. Ant to use those functions we are going to use the core views: LoginView, LogoutView, and IsLoggedIn.

7.7.1 Settings.py configuration

To authenticate users you need the cookies to be sent to the API on making a POST. To achieve this you need to configure Angular and Django. You can see the Angular configuration in the section 9.11.2, page 115. In the API, in the djangoapi/settigns.py file, you need the following:

- Install corsheaders (aready installed).
- Add corsheaders to the installed apps (already added).
- Add to the MIDDLEWARE list, at first position, the midleware corsheaders.middleware.CorsMiddleware. This is the in charged of set the proper headers in the requests. A middleware is in the middle of Django and the request, and allow modify the request headers.

- Add the following:

```
#para el CORS
if DEBUG:
    #CORS_ALLOW_ALL_ORIGINS = True   <-- Not allowed any more
       for chrome
    #You need to specify the allowed origins
    CORS_ALLOWED_ORIGINS=['http://localhost:4200']

#necressary to allow the cookies to be sent in the header of the
    request
CORS_ALLOW_CREDENTIALS = True
```

7.7.2 Login

To authenticate users you have to have a non protected view. This view will receive the user and password by POST. Add the following to the core/views.py module:

```
class LoginView(View):
    def post(self, request, *args, **kwargs):
            # The request object has the user information
        if request.user.is_authenticated:
            username=request.user.username
            return JsonResponse({"ok":"true","message": "The user
               {0} already is authenticated".format(username), "data
               ":[]})

            username=request.POST.get('username')
            password=request.POST.get('password')
            user = authenticate(username=username, password=password)
            if user:
                login(request,user)#introduce into the request cookies
                   the session_id,
                        # and in the auth_sessions the session data.
                           This way,
                        # in followoing requests, know who is the user
                           and if
                        # he is already authenticated.
                        # The coockies are sent in the response header
                           on POST requests
                return JsonResponse({"ok":"true","message": "User {0}
                   logged in".format(username), "data":[{"userame":
                   username}]})
            else:
                # To make thinks difficult to hackers, you make a random
                    delay,
                # between 0 and 1 second
                seconds=random.uniform(0, 1)
                time.sleep(seconds)
                return JsonResponse({"ok":"false","message": "Wrong user
                    or password", "data":[]})
```

7.7.3 Logout

And to logout add the following to the core/views.py module:

```
class LogoutView(LoginRequiredMixin, View):
    def post(self, request, *args, **kwargs):
        username=request.user.username
        logout(request) #removes from the header of the request
                        #the the session_id, stored in a cookie
        return JsonResponse({"ok":"true","message": "The user {0} is
            now logged out".format(username), "data":[]})
```

7.7.4 Is the user loggedin?

A common practice is, on loading a web page, automatically send a request to the API to know if the user is already loggedin. So we need an endpoint to return to the request if the user is logged in, and its username:

```
#core/views.py

class LogoutView(LoginRequiredMixin, View):
    def post(self, request, *args, **kwargs):
        username=request.user.username
        logout(request) #removes from the header of the request
                        #the the session_id, stored in a cookie
        return JsonResponse({"ok":"true","message": "The user {0} is
            now logged out".format(username), "data":[]})
```

7.7.5 Update the core urls

Now you need to update the core/urls.py to expose the new views to internet:

```
#core/urls.py
...
urlpatterns = [
    path("hello_world/", views.HelloWord.as_view(),name="hello_world
        "),
    path('', include(router.urls)),
    path('not_loggedin/', views.notLoggedIn, name="not_loggedin"),
    path('login/', views.LoginView.as_view(),name="login"),
    path('logout/', views.LogoutView.as_view(),name="login"),
    path('isloggedin/', views.IsLoggedIn.as_view(),name="isloggedin
        "),
]
```

Use the PostMan application to execute POST requests to the urls http://localhost:8000/core/login/, and http://localhost:8000/core/logout/.

.

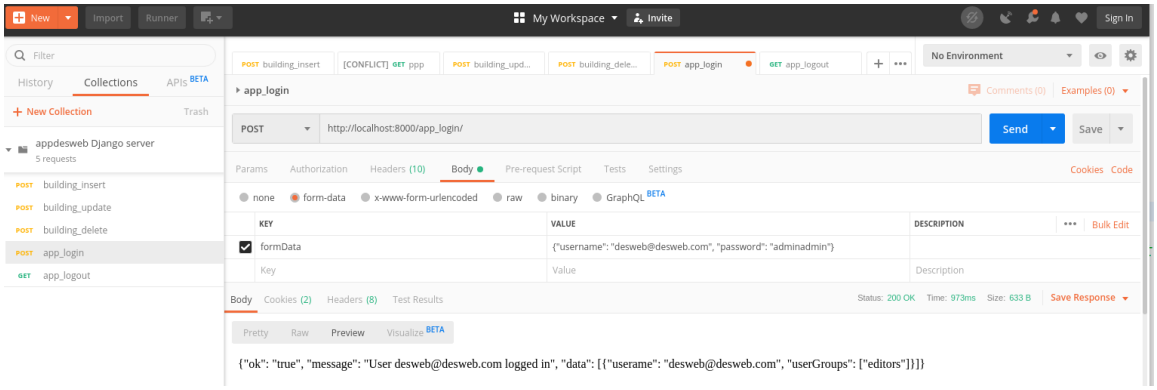

Figura 7.5: Login with PostMan

7.7.6 How does Django remember we where already authenticated

If you try to login again you will receive the message:

```
{"ok": "true", "message": "You where already authenticated", "data":
    [{"username": "desweb@desweb.com"}]}
```

This means the system remembers that you where already logged in. But, how?. Internet is an stateless system. This means that once the server sends the answer forgets everything about the user. If we where not using Django, next request of the authenticated user will be rejected, as the server forgets that the user where already authenticated. What Django does to remember the user?. When you call the function *login(request,user)*, the *login* function creates a random session ID and it is stored in the database, together the encrypted session data (field *session_data*). But this is not enough, the *login* function also stores the session ID in the request, in a part called *headers*, in something called cookie (Figura 7.6). Wen the server answer is received by the client, usually a web browser, but in this case PostMan, the cookie is stored in the client. The cookies are automatically sent to the server in every request, so Django gets the session ID, search for it in the database and, if it exists, retrieves from the database the information. These information is encrypted, but Django is able to decrypt it. In this way, only with the session ID Django can retrieve the username, if he where logged in, etc.

Try to insert a building, in order to check that now, as you are logged in, you are able to insert.

7.7.7 Session expiration time

Although the user close the client (the web browser, or PostMan), the session ID is stored, and each time the user visits our page, the cookie is sent to the server, so Django is able to retrieve the user data: if he is authenticated or not, the username, etc. You can test this by closing PostMan, restarting it again, and sending an insert request. You will be able to do it because the cookie is sent in the header of the request, and the session expire date is not expired, see the field *expire_date* in the Figura 7.6.

By default the session expire date is set 14 days latter of the session initialization. You can change this. For example you can force session expiration on closing the client (the web browser, with PostMan it does not work). To do that you have to set the variable *SESSION_EXPIRE_AT_BROWSER_CLOSE* to *True*, in the *settings.py* module.

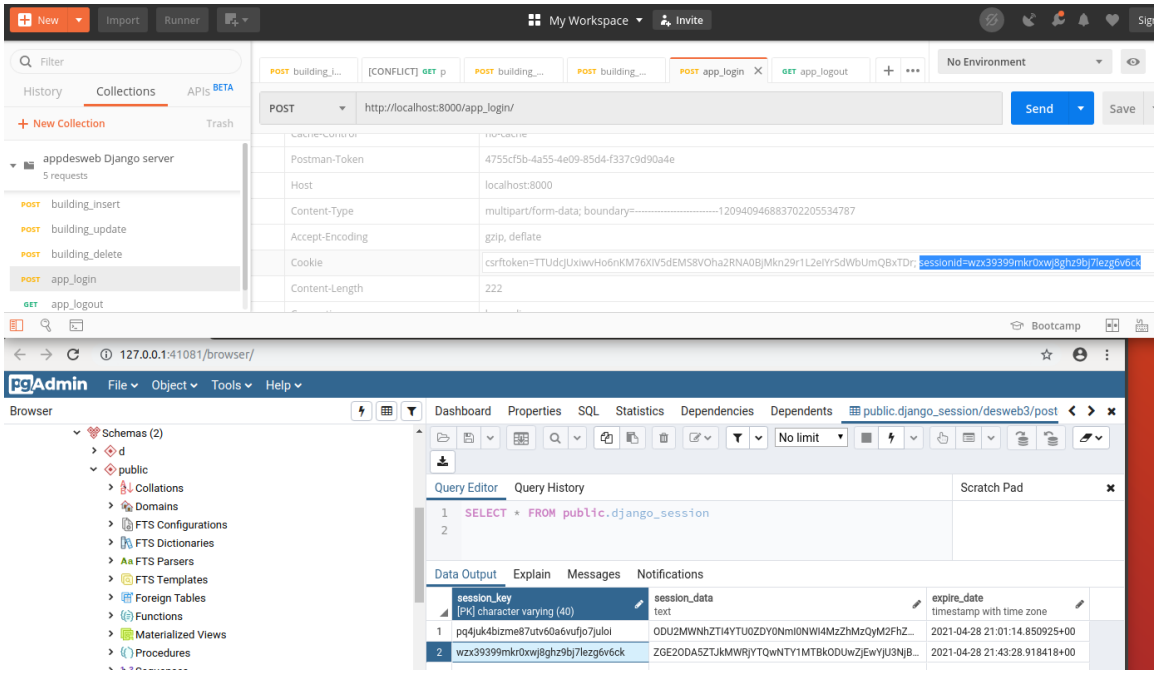

Figura 7.6: Session ID stored in a cookie, in the headers of the request, and in the database, in the django_session table

7.7.8 Limit the access to views to users that belongs to some groups

LoginRequired mixin protects all the methods oa View. But a view can have many methods: insert, delete, update, selectone, ..., and some authenticated users must have access to some of this methods, and other authenticated users must have access to other methods in the same view. In most cases it is not enough that a user be authenticated to allow him to use all methods of a view. For example *users*, and *editors* should not be able to perform the same operations, despite both are be authenticated. For example an editor, belonging to the *editor* group, must have access to the *insert* method of a view, but not a normal user, belonging to the *user* group.

You already know you can create groups and users with the admin site of Django, or with Python. In the latter case you must create specific views to receive the user data bu POST, create the user and add it to a group. Usually groups are created with the Django admin site.

So it is not enough to be authenticated to be able to use some views, also it is necessary, in most cases, to check the group that the user belongs.

This is easy to check as you have the function *core.myLibs.users.manageUsers.getUserGroups*, that receives the User object which is always stored in the request object, in *request.user*. This function returns a list with the name of the groups that the user belongs. To check if a group name is in that list is very easy. For example, in the following listing is checked if the logged in user belongs to the editor group. You can put this code in the *insert* method to avoid a normal user, despite to be authenticated, could insert:

```
...
l=general.getUserGroups(request.user)
if not 'editor' in l:
    #is not able to continue
    return JsonResponse({"ok": "false", "message": "You have to be
        administrator to use this view", "data": []})
```

```
#the logic of the method continues
...
```

CAPÍTULO 8

Django REST Framework (DRF) (Optional)

8.1 Introduction

Django Rest Framework simplifies API development in Django by providing serialization, authentication, and easy-to-use views. By leveraging DRF, developers can create powerful and scalable RESTful APIs efficiently. For further reading, refer to the official documentation:
https://www.django-rest-framework.org/.

Django REST Framework is an APP for Django that add a new component to Django: serializers Figura 8.1.

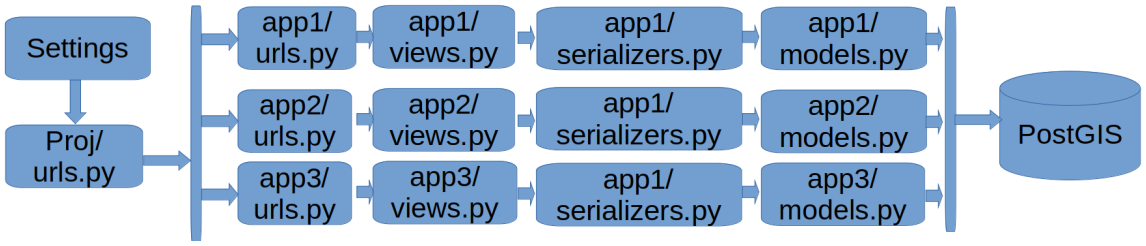

Figura 8.1: DRF data flow schema

Serializers convert dictionaries to model objects and vice versa, but in the process check all the field data, one by one, based in the field definition, in the model definition, and if something is not right, it raise automatic message errors. Also you can write your own validation methods.

Also it provides the viewsets.ModelViewSet class that provides all the standard operations by default, cero code: insert, delete, update, partial update (this is something we do not have in our views), selectone, and selectall. You only have to inherit from viewsets.ModelViewSet.

I have created an example of serializer in buildings/serializers.py, and it is used in a viewsets.ModelViewSet child class in buildings/views.py module. You can check the standard methods with the following urls:

```
The ModelViewSet class is a special view that Django Rest
   Framework
    provides to handle the CRUD operations of a model

The actions provided by the ModelViewSet class are:
    -list()  -> GET operation over /buildings/buildings/. It
       will return all reccords
    -retrieve() ->GET operation over /buildings/buildings/<id>/.
                 It will return the record with the id.
    -create() -> POST operation over /buildings/buildings/. It
       will insert a new record
    -update() -> PUT operation over /buildings/buildings/<id>/.
                 It will update the record with the id.
    -partial_update() -> PATCH operation over /buildings/
       buildings/<id>/.
            It will update partially the record with the id.
            The difference between update and partial_update is
               that the first one
            will update all the fields of the record, while the
               second one will update
            only the fields that are present in the request.
    -destroy() -> DELETE operation over /buildings/buildings/<id
       >/.
            It will delete the record with the id.
```

The calls are different that we used in normal Django views. Standard Django views only have the post and get methods, and by using the *action* parameter in the url, we managed to send the requests to other class methods: insert, delete, ... DRF use more send methods (GET, POST, PUT, PATCH and DELETE) to call the proper method (list, retrieve, create, update, partial_update and destroy). As with the BaseDjangoView class, you can override this methods to adjust behavior.

DRF is also better to control the user access to the class methods. I for example use *Django REST - Access Policy*: https://rsinger86.github.io/drf-access-policy/.

8.2 Set the environment variables to work with

You must set in the .env.dev file the variables:

```
EPSG_FOR_GEOMETRIES=25830
ST_SNAP_PRECISION=0.0001
MAX_NUMBER_OF_RETRIEVED_ROWS=1000
```

This variables are loaded in djangoapi/settings.py, from where you can import them in other apps.

8.3 Manage models without geometry with serializers and ModelViewSet

We are going to manage the owners of the buildings.

8.3.1 Create the model Owners

```
#buildings/models
class Owners(models.Model):
    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100, blank=True, null=True)#
        optional
    dni = models.CharField(max_length=100, unique=True)#mandatory
        and unique
```

8.3.2 Migrate model Owners

Migrate the model. To do this get into the Django container

```
docker exec -it buildings-djangoapi-1 /bin/sh
python manage.py makemigrations
python manage.py migrate
```

8.3.3 Create the serializer OwnersSerializer

```
from django.db import connection
from rest_framework import serializers
from .models import Owners

class OwnersSerializer(serializers.ModelSerializer):
    class Meta:
        model = Owners # The model to work with
        fields = ['id', 'name', 'dni'] # The fields of the model to
            work with
                            # name is mandatory and unique. This
                                checks are automatically
```

```
                                # defined by the serializer by reading
                                    the characteristics of the
                                # field in the Owners serializer


    #you can validate the fields of the model using the validate_<
        field_name> method
    #def validate_<field_name>(self, value):
    def validate_name(self, value):
        if 'bad' in value:
            raise serializers.ValidationError("The name can't
                contain 'bad'.")
        return value
```

8.3.4 Create the view OwnersModelViewSet

Create a view inheriting of the DRF ModelViewSet class. It has the standard views for a model: list(), retrieve(), create(), update(), partial_update() and destroy(). If you need extra functionality you can override any of them.

```
#rest_framework imports
from rest_framework import viewsets
from rest_framework import permissions

class OwnersModelViewSet(viewsets.ModelViewSet):
    """
    DJANGO REST FRAMEWORK VIEWSET.

    The ModelViewSet class is a special view that Django Rest
        Framework
        provides to handle the CRUD operations of a model

    The actions provided by the ModelViewSet class are:
        -list()  -> GET operation over /buildings/owners/. It will
            return all reccords
        -retrieve() ->GET operation over /buildings/owners/<id>/.
                    It will return the record with the id.
        -create() -> POST operation over /buildings/owners/. It will
            insert a new record
        -update() -> PUT operation over /buildings/owners/<id>/.
                    It will update the record with the id.
        -partial_update() -> PATCH operation over /buildings/owners
            /<id>/.
                It will update partially the record with the id.
                The difference between update and partial_update is
                    that the first one
                will update all the fields of the record, while the
                    second one will update
                only the fields that are present in the request.
        -destroy() -> DELETE operation over /buildings/owners/<id>/.
                It will delete the record with the id.
    """
    queryset = Owners.objects.all()#The opeartions will be done over
        all the records of the Owners model
```

```
    serializer_class = OwnersSerializer#The serializer that will be
        used to serialize
                            #the data. and check the data that is
                                sent in the request.
    permission_classes = [permissions.AllowAny]#Any can use it.
                                # Use https://rsinger86.github.io/
                                    drf-access-policy/
                                # to more advanced permissions
                                    management
```

8.3.5 Register the view endpoints in the urls.py

```
from django.urls import path, include
from . import views
from rest_framework import routers
from . import views

router = routers.DefaultRouter()
router.register(r'owners', views.OwnersModelViewSet)

urlpatterns = [
    path("hello_world/", views.HelloWord.as_view(),name="hello_world
        "),
    path('', include(router.urls)),
    path('buildings_view/<str:action>/', views.BuildigsView.as_view
        (), name='buildings_views'),  # POST requests
    path('buildings_view/<str:action>/<int:id>/', views.BuildigsView
        .as_view(), name='buildings_views'),  # POST requests
]
```

The endpoints will be available in http://server.com/buildings/owners/

8.3.6 Check the endpoints in Swagger

Visit the url http://localhost:8000/swagger/, and deploy the Owners app, Figura 8.2.

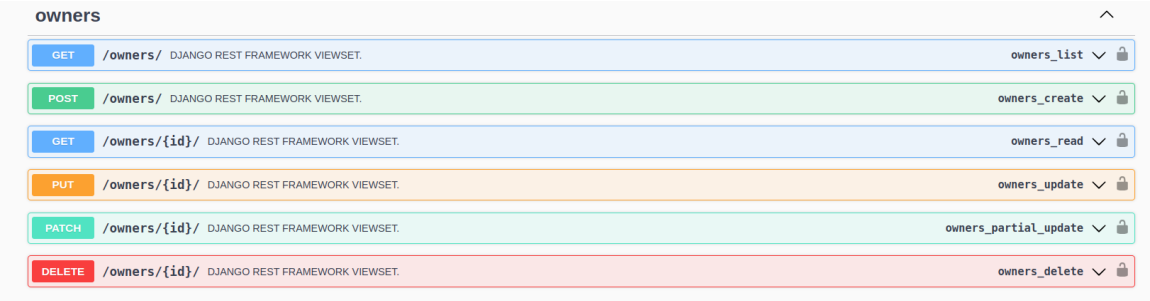

Figura 8.2: DRF. Owners endpoints

Deploy the POST operation to add a Owner. Press the button 'Try it out'. Put the data in the json, and press 'Execute'. After several inserts, if you put 'bad' in the name, and a dni that already exist you will get the following response:

```
Error: Bad Request
Code: 400
```

```
Response body
{
  "name": [
    "The name can't contain 'bad'."
  ],
  "dni": [
    "owners with this dni already exists."
  ]
}
```

As you can see DRF help us facilitating the entry data checks.

8.3.7 Register the model Owners in the Django Admin Site

Add into the file buildings/admin.py the following:

```
from django.contrib import admin
from .models import Owners

admin.site.register(Owners, admin.ModelAdmin)
```

Check the Owners table appears in the admin site: http://localhost:8000/admin, and you can add/edit/delete/select owners.

8.4 Manage models with geometry with serializers

We are going to manage the buildings model, which have a geometry field.

8.4.1 Create the model Owners

It is already created, but I copy the code again

```
#buildings/models
from django.db import models
from django.contrib.gis.db import models as gis_models
from djangoapi.settings import EPSG_FOR_GEOMETRIES

class Buildings(models.Model):
    id = models.AutoField(primary_key=True)
    description = models.CharField(max_length=100, blank=True, null=
        True)
    area = models.FloatField(blank=True, null=True)
    geom = gis_models.PolygonField(srid=int(EPSG_FOR_GEOMETRIES),
        blank=True, null=True)
```

8.4.2 Migrate the model Buildings

It has been already migrated, but I do it again here:

```
docker exec -it buildings-djangoapi-1 /bin/sh
python manage.py makemigrations
python manage.py migrate
```

8.4.3 Create the serializer BuildingsSerializer

The next serializer is a bit more complicated because it has a geometry field. First of all it inherits from GeoModelSerializer, that id a child class of DRF serializers. I have added to it some more class properties to configure the method validate_geom who validated the geom by checking:

- It snap the geometry, before the geometry checks, and also before insert or update. The inserted geometry will be snaped to the grid.
- If *check_geometry_is_valid* = True. Rejects not valid geometries.
- If *check_st_relation = True*, checks the relation of the new geometry with the existing ones in the same layer. The relation checked is the set in the property *matrix9IM*.

The serializer assumes that the model has the geometry field *geom*. You must add to the Meta.fields property the rest of the fields of the model that you want to serialize, and that are not in the GeoModelSerializer.

```
#buildings/serializers
from django.db import connection

from rest_framework import serializers

from core.myLib.geoModelSerializer import GeoModelSerializer
from .models import Buildings, Owners

class BuildingsSerializer(GeoModelSerializer):
    check_geometry_is_valid = True #if true it will check if the
       geometry is valid: not self-intersecting and closed
    check_st_relation = True #if true it will chck the relation of
       the geometry with the other geometries
    matrix9IM = 'T********' #matrix 9IM for the relation of the
       geometries: 'T********' = interiors intersects
    geoms_as_wkt = True #if true the serializer expects the
       geometries in WKT format. If false, in geojson format
    check_st_relation = True #if the new geometry must be checked
       against
            #the other geometries in the table according to the
               matrix9IM. If any geometry
            #has the relation with the new geometry, the new
               geometry is not saved
            #an a validation error is raised, with the ids of the
               geometries that have the relation

    class Meta:
        model = Buildings
        fields = GeoModelSerializer.Meta.fields + ['description', '
           area']

    def validate_geom(self, value):
        """Validates if a geometry is valid.
            Do not do anythin special. Simple is an example of how
               to override the father method
        """
        print('validate_geom, child')
        return super().validate_geom(value)
```

Have a look to the GeoModelSerializer implementation in core/myLib/geoModelSerializer.py. It is quite well commented.

8.4.4 Create the view BuildingsModelViewSet

Create a view inheriting of the DRF ModelViewSet class. It has the standard views for a model: list(), retrieve(), create(), update(), partial_update() and destroy(). If you need extra functionality you can override any of them.

```
#rest_framework imports
from rest_framework import viewsets
from rest_framework import permissions

class BuildingsModelViewSet(viewsets.ModelViewSet):
    """
    DJANGO REST FRAMEWORK VIEWSET.

    The ModelViewSet class is a special view that Django Rest
       Framework
     provides to handle the CRUD operations of a model

    The actions provided by the ModelViewSet class are:
     -list()  -> GET operation over /buildings/buildings/. It
        will return all reccords
     -retrieve() ->GET operation over /buildings/buildings/<id>/.
                  It will return the record with the id.
     -create() -> POST operation over /buildings/buildings/. It
        will insert a new record
     -update() -> PUT operation over /buildings/buildings/<id>/.
                  It will update the record with the id.
     -partial_update() -> PATCH operation over /buildings/
        buildings/<id>/.
              It will update partially the record with the id.
              The difference between update and partial_update is
                 that the first one
              will update all the fields of the record, while the
                 second one will update
              only the fields that are present in the request.
     -destroy() -> DELETE operation over /buildings/buildings/<id
        >/.
              It will delete the record with the id.
    """
    queryset = Buildings.objects.all()[:MAX_NUMBER_OF_RETRIEVED_ROWS
       ]#The opeartions will be done over all the records of the
       Buildings model
    serializer_class = BuildingsSerializer#The serializer that will
       be used to serialize
                           #the data. and check the data that is
                              sent in the request.
    permission_classes = [permissions.AllowAny]#Any can use it.
                               # Use https://rsinger86.github.io/
                                  drf-access-policy/
                               # to more advanced permissions
                                  management
```

8.4.5 Register the view endpoints in the urls.py

```
from django.urls import path, include
from . import views
from rest_framework import routers
from . import views

router = routers.DefaultRouter()
router.register(r'owners', views.OwnersModelViewSet)
router.register(r'buildings', views.BuildingsModelViewSet)

urlpatterns = [
    path("hello_world/", views.HelloWord.as_view(),name="hello_world
        "),
    path('', include(router.urls)),
    path('buildings_view/<str:action>/', views.BuildigsView.as_view
        (), name='buildings_views'),  # POST requests
    path('buildings_view/<str:action>/<int:id>/', views.BuildigsView
        .as_view(), name='buildings_views'),  # POST requests
]
```

The endpoints will be available in http://server.com/buildings/buildigns/

8.4.6 Check the endpoints in Swagger

Visit the url http://localhost:8000/swagger/, and deploy the Buildings app, Figura 8.2.
Deploy the POST operation to add a Owner. Press the button 'Try it out'. Put the data in the json:

```
{
  "geom": "POLYGON((0 0, 10 0, 10 10, 0 10, 0 0))",
  "description": "Poligono",
  "area": 200
}
```

Press 'Execute'. The result will be the following:

```
{
  "id": 26,
  "geom_geojson": "{ \"type\": \"Polygon\", \"coordinates\": [ [ [
     0.0, 0.0 ], [ 10.0, 0.0 ], [ 10.0, 10.0 ], [ 0.0, 10.0 ], [
     0.0, 0.0 ] ] ] }",
  "geom_wkt": "POLYGON ((0 0, 10 0, 10 10, 0 10, 0 0))",
  "description": "Poligono",
  "area": 200
}
```

If you try to insert it again, as the interiors intersects you receive the following answer:

```
{
  "geom": [
    "The following ids of the table buildings_buildings have the
       requested  relation (ST_relate, matrix: T********), with the
       given geometry: [(26,)]"
  ]
}
```

You can also test the endpoints in Postman.

8.4.7 Register the model Buildings in the Django Admin Site

Add into the file buildings/admin.py the following:

```
from django.contrib import admin
from django.contrib.gis import admin
from .models import Buildings, Owners

admin.site.register(Owners, admin.ModelAdmin)
admin.site.register(Buildings, admin.GISModelAdmin)
```

Check the Buildings table appears in the admin site: http://localhost:8000/admin, and you can add/edit/delete/select Buildings.

CAPÍTULO 9

Making a web with Angular

9.1 Introduction

Angular (https://angular.dev/) is a web develop framework.

Angular does not use regular JavaScript but TypeScript. Neither it uses pure HTML but the Angular template language, which understand regular HTML, but it is not exactly the same. CSS is the same.

Angular divides the web page in components, and provides ways to include one component (father) inside other (child), by using the component label (like a new html label). For example, a web page has usually four parts: menu, header, body and footer. Make sens to create a component for each part and include them in the main component, called app-component. By default all components names start by *app-*, so the labels for the components will be: app-menu, app-header, app-body and app-footer. A to combine them in the app-component, you only have to open the app.component.html file, and type the following:

```
<app-menu></app-menu>
<app-header></app-header>
<router-outlet></router-outlet>
<app-footer></app-footer>
```

Well strictly speaking, you need to import the components menu, header, body and footer, in the app component to be able to use them, but in this case I only am showing the philosophy.

Each child component can be composed by other child components, so you can subdivide presentation and functionality, which is the secret to success: to treat each problem separately.

Each component have three parts:

- The template. An .html file with the html elements to show.
- The CSS file. A CSS file to define the component appearance: alignment, colors, fonts, etc. It is important to remark child components do not inherit father CSS rules. There is a common CSS file with affect all components (src/styles.scss), but component styles are independent.
- The .ts file. It contains the functionality of the component. It is coded in TypeScript, similar to JavaScript, but it is a typed language. This means you have to define variables and types before to use them.

Up to here the nice side of the front developing. To divide functionality has a drawback: you need to pass data between components, and communicate events from father to child and vice-versa. And this is the most difficult part. This things have its own technique. We are not communicate events between components, but inevitably we have to pass data between components. Angular have three mechanism to do this:

- To pass variables from father to child. This is very easy, but we are not going to see it.
- To pass variables from child to father. A bit strange technique. We are not going to see it.
- Services. Services are .ts files, usually stored in src/services. Services are classes that can be used by get data from the server, or a place to store data for components. Unlike components, services are never destroyed once loaded, so one component can store data there and recover it from the same component, if it is created again, or by other components.

Also to organize data properly, it is recommendable to define structures of data that be a mirror of the tables of the database. They are classes with properties with the same field names than the database tables. They are the same that models in Django, in fact, they usually are stored in the src/models folder.

9.2 Setup the development environment

Clone the angular-template repository and start the container:

- (c:/docker/): git clone https://github.com/joamona/angular-template.git
- Open the repository with Visual Studio Code (VS).
- Open a terminal in VS.
- Start Docker Desktop.
- Start the container: docker compose up.
- Visit the url http://localhost:4200.

9.3 Create an angular project

You do not need to do this because it is already created in the angular-template repo. You only have to modify it, but to create an Angular project you need to perform the following steps:

- Install a supported version of Node.
- Install angular: npm install -g @angular/cli .
- Create a new project: ng new <project-name>.
- Install the project dependencies in packages.json: cd <project-name>, npm install. This will create the node_modules folder to store the packages used in the project.
- Serve the app. ng serve
- Visit the url http://localhost:4200.

9.4 Add new packages to the project

If you need any other package, execute npm install <package>. This will install the package in the node_modules folder, but also update the packages.json file. The goal is to be able to put all the project source code in a repo, but not the node_modules folder. This is automatically created on executing npm install. This command will read the packages.json file and will create the node_modules folder, which usually takes a lot of space.

We will need the ol, and angular material packages. Which deliberately have not been installed to you to see the process.

9.5 Angular project structure

Like Django, an Angular project have a strict structure, and a place to put anything. Check the following files in the project:

```
project\.env --> Environment variables. Not used in this case
project\docker-compose.yml --> Docker compose project

project\web\Dockerfile --> Image creation for docker
```

```
project\web\package.json  --> List of software needed for the
   project.
                        It is like python manage.py > freeze.
                        To install the software you must execute npm
                            install
                        from the web folder. It will create the
                        node_modules folder where all the
                        node packages are installed.
project\web\angular.json  --> Angular configuration to compile the
   project, etc.
project\web\src  --> Source code of the project
project\web\src\index.html  --> Main page where the app-component is
    inserted.
project\web\src\main.ts  --> Functionality for the index.html
project\web\src\styles.css  --> Common styles for all components.
                    If a component redefines a style, the component
                    style takes preference

project\web\src\app\app.component.html --> Elements of the main
   component
project\web\src\app\app.component.css  --> Styles for the elements
project\web\src\app\app.component.ts   --> Functionality for the
   component
project\web\src\app\app.routes.ts   --> Here is where you specify,
   if the
                            url changes, which component to show
                            in the <router-outlet></router-outlet>
                            component. It is similar to the urls.py
                            of Django.

project\web\src\app\components  --> folder for all components.
            Each component is a folder called with the name
            of the component. Each folder have three files:
                    name\name.component.html --> Called template,
                       where the html
                                                elements go
                    name\name.component.scss  --> Styles for the
                       html elements
                    name\name.component.ts  --> Functionality for
                       the html
                                                    elements

project\web\src\app\services  --> folder for all services (.ts files
   )

project\web\src\app\models  --> folder for all models (.ts files)

project\web\src\app\commonLibs  --> folder for all common
   functionality
                                      (.ts files)

project\web\dist  --> Compiled web. To compile enter into the
   container
            and execute: ng build
```

```
                This will generate index.html and .css, and .js for
                the web. Copy this code into the production server
                to publish it, into /var/www/html/mydomain.com/

project\web\public  --> Place to put the additional static content
                    for example images. These images are referenced
                    form the html templates. For example:
                    <img class="fff" src="images/forjandoFuturos.png
                        "
                         alt="Forjando Futuros"></a>
                    will search the image forjandoFuturos.png in the
                    folder
                        project\web\public\images\
                    ng build will get the image and will put it into
                    the dist folder. You do not need to publish the
                    public folder in the production server
```

9.6 Practical exercise. Publish the project

Follow the following steps:

- Create a component. To do this you must get into the container.

```
ng generate component components/footer
```

- Put a title in the components/footer/footer.component.html and add a link to an image:

```
<img class="fff" src="images/example.png" alt="Example" ></a>
```

- Take a screen shot and put it in web/public/images/example.png
- Import the component from web/src/app/app.component.ts.

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { FooterComponent} from './components/footer/footer.
   component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, MenuComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.scss'
})
export class AppComponent {
  title = 'web';
}
```

- Add the component label <app-footer><app-footer>to the main component template web/src/app/app.component.html.

```
<h1>It will work</h1>
<app-footer></app-footer>
```

- See the result in http://localhost:4200
- Compile the project: get into the container, and execute:

```
ng build
```

- Copy the content of the folder web/dist/web/browser into the VPS geomaticaupv.es, in the folder /var/www/html/yoursubdomain.es/
- Visit your subdomain.
- Congratulations, you have created a web with Angular, created a component, added static content to it, compiled it and published it. No bad for first session.

9.7 Create the menu, header, footer, home, map, help and building-form component

To create the componentes, get into the container an write:

```
ng g c components/menu
ng g c components/header
ng g c components/footer
ng g c components/home
ng g c components/help
ng g c components/map
ng g c components/forms/building-form
```

9.8 Set the menu component

The goal is to have some buttons and, on click, show in the router outlet a specific component. In the components/menu/menu.component.ts, import all the dependencies you need to use in the template (.html):

```
import { Component } from '@angular/core';
import {MatButtonModule} from '@angular/material/button';
import { RouterLink } from '@angular/router';
import {MatIconModule} from '@angular/material/icon';
import { MatTooltip } from '@angular/material/tooltip';

@Component({
  selector: 'app-menu',
  standalone: true,
  imports: [MatButtonModule, RouterLink, MatIconModule, MatTooltip],
  templateUrl: './menu.component.html',
  styleUrl: './menu.component.scss'
})
export class MenuComponent {
  constructor() { } // Inject the service to use it in the template
}
```

In the components/menu/menu.component.html, create some buttons and link the click to the urls with *routerLink*.

```
<button mat-button routerLink='home'>Home</button>
<button mat-button routerLink='map'>Map</button>
<button mat-button routerLink='help'>Help</button>
<button mat-button routerLink='about'>About</button>
<button mat-button routerLink='building-form'>Building form</button>
```

9.9 Configure the routes for the menu component in the router outlet: app.routes.ts

In the app.routes.ts file you set which component must be showed when a route is in the path of the browser. These paths are set with routerLink directive, in the template of the menu component:

```
import { Routes } from '@angular/router';
import { MapComponent } from './components/map/map.component';
import { AboutComponent } from './components/about/about.component';
import { HelpComponent } from './components/help/help.component';
import { HomeComponent } from './components/home/home.component';
import { BuildingFormComponent } from './components/forms/building-
    form/building-form.component';

export const routes: Routes = [
    {path: '', redirectTo: '/home', pathMatch: 'full'},
    {path: 'home', component: HomeComponent},
    {path: 'help', component:HelpComponent},
    {path: 'about', component:AboutComponent},
    {path: 'map', component:MapComponent},
    {path: 'building-form', component:BuildingFormComponent},
];
```

Visit the web in and check de buttons links.

9.10 Models

A model is a model of data. As in Django, it is recommended to have a model for each row data, or each group of data. A model is like a structure of data. It is useful in Angular because you can manage them easily in the .ts and in the template (.html).

Models must be created in the models folder. They are regular .ts files with an exported class. Exported classes, or function are the only things available in a .ts file. You must export anything you want to be importable from the .ts.

9.10.1 Server answer model

All the answers of the server follow the same structure, the fields ok, message and data. So we can define a model for it. Create the file app/models/server-answer.model.ts, with the following content:

```
export class ServerAnswerModel {
    message: string='';
    ok: boolean = false;
    data: {
      [key: string]: any; // Permite otras propiedades dinámicas si
         las hay
    }[]=[];
  }
```

9.10.2 Building model

To manage each row data of the table buildings, we can create the file: buiding.model.ts, with the following content:

```
export class BuildingModel {
    public id: number=-1;
    public description: string='';
    public area: number=-1;
    public geom:string='';
  }
```

9.11 Services

Services must be created in app/services.
Services are useful for:

- Put shared functionality between components. For example the ApiService has the method post and get available for any form which need to send requests to the server.
- Share data between components. For example the url of an API.
- As components are constantly seen changes in the instance variables, from one component you can change other component in real time, id you change variable values in a service shared by the components. An example is the AuthService, where when from the login component, a user is authenticated by using the ApiService, and change the status from the authService.loggedin from false to true, and AuthService.username from nothing to the username string. As the AuthUser is a shared service with the menu component, the menu component, can show the authenticated user data, in real time, when the user initializes the session from the login component. The same when a user logges out.

You have to know about services:

- Services must not be imported in the component.
- Services must be initialized in the component constructor. This is called service injection.
- Once initialized in one component, it is never initialized again, so it keeps its data. Components are initialized every time they are required in the router outlet.

9.11.1 The settings service

This service contains a variable with the api url to send all requests. May components need this url, so make sense to have as service with this variable.

Create the service. To create the service, get into the container an write:

```
ng g s services/settings
```

Next paste the following:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
```

```
export class SettingsService {
  public API_URL='http://localhost:8888/'
  public GEOSERVER_URL='http://localhost:8080/geoserver/'
  constructor() { }
}
```

As you can see a service is an injectable therefore it should injected in a constructor, where needed. If the service has been already initialized, it will be only injected. If not, it will be initialized and injected. Do not import services in components.

9.11.2 The API service

This service is used to send requests to the server. The form data must suffer a transformation, and the headers of the request must be configured, so it make sense to have a shared service between component and avoid making this configurations in all components.

Create the service. To create the service, get into the container an write:

```
ng g s services/api
```

Next paste the following:

```
import { Injectable } from '@angular/core';

//To be able to set http requests

import { HttpClient, HttpHeaders, HttpParams} from '@angular/common/
   http';

import { SettingsService } from './settings.service';

@Injectable({
  providedIn: 'root'
})
export class ApiService {

  headers = new HttpHeaders({
    'Content-Type': 'application/x-www-form-urlencoded'
  })

  constructor(public settingsService:SettingsService, private
     httpClient:HttpClient) { }

  get(endPointUrl:string, getParams:HttpParams=new HttpParams({})){
    return this.httpClient.get<any>(this.settingsService.API_URL +
       endPointUrl,
      {
        headers: this.headers, responseType : 'json', reportProgress
           : false,
        params: getParams,
      })
  }

  post(endPointUrl:string, postParams:{}={}){
    var postData = this.generarHttpParamsDesdeObjeto(postParams);
    console.log('postParams',postParams);
    console.log('postData',postData);
```

```
        return this.httpClient.post<any>(
          this.settingsService.API_URL + endPointUrl,
          postData,
          { headers: this.headers,
            responseType : 'json',
            reportProgress: false,
            withCredentials: true, //withCredentials. Necessary to
                send cookies: sessionid, csrf, ...
          }
        )
      }

      private generarHttpParamsDesdeObjeto(data: { [key: string]: string
          | number }): string {
        /**
         * Gets a string of HttpParams from an object.
         * By default angular sends the data in request.body
         *    in this way it senfs the data in the body of the request
            request.BODY, as
         *    django expects.
         * You need also to set the headers in the request
         *    'Content-Type': 'application/x-www-form-urlencoded'
         * @param data: an object with key-value pairs {'key': 'value',
            'key2': 'value2', ...}
         * @example {id: '', description: 'gg', area: '236', geom: '
            polygon((0 0, 1 0, 1 1, 0 0))'}
         * @returns {string}: id=&description=gg&area=236&geom=polygon
            ((0%200,%201%200,%201%201,%200%200))
         * @description
         * This function takes an object and converts it into a string
            of HttpParams.
         */
        let params = new HttpParams();
        for (const key in data) {
          if (data.hasOwnProperty(key)) {
            params = params.set(key, data[key].toString()); //
                Convertimos el valor a string
          }
        }
        return params.toString();
      }
    }
```

As you can see in the code, the service exposes the methods post and get, to be used where needed.

9.12 Create a reactive form, in building-form component

To create a reactive form, in the *.component.ts file:

- Import the necessary classes.
- Create the form controls, one for each form field, and with the same name. Set the validators in the control.
- Create a form group to group all the form controls. This is useful to get all the control values together, and to know if all the validators are accomplished.import Injectable from '@angular/-core';
- Create the controls in the template (.html), and link the form and controls with controls in the .ts.
- Inject the ApiService.
- Create the method to execute on click over the form buttons.
- Send the data and manage the server answer in the .subscribe method of the observable.
- Use the ServerAnswerModel and the model for the table to cast the data.
- Inform in the web page to the user the server answers.
- Create the buttons to execute insert, update, ... in the template.
- Do not allow users to send the form if all controls are not valid.
- Try the buttons.

9.12.1 Import the necessary classes

In the .ts file paste the following:

```
import { Component } from '@angular/core';

//To use the template syntax @if, @for, ...
import { CommonModule } from '@angular/common';

//To use forms
//  Import in the imports on the component the following
import { ReactiveFormsModule } from '@angular/forms';
import {MatInputModule} from "@angular/material/input";//angular
   material must be installed before
import { MatTooltip } from '@angular/material/tooltip';
import {MatCardModule} from '@angular/material/card';
import {MatButtonModule} from '@angular/material/button';

//To use the controls in the component
//  Import in the imports on the component the following
import {FormControl} from '@angular/forms';
import {FormGroup, Validators} from '@angular/forms';
```

```
import { ApiService } from '../../../services/api.service';
import { ServerAnswerModel } from '../../../models/server-answer.
   model';
import { BuildingModel } from '../../../models/building.model';

@Component({
  selector: 'app-building',
  standalone: true,
  imports: [CommonModule, MatInputModule, ReactiveFormsModule,
     MatTooltip, MatButtonModule,
    MatCardModule
  ]
  ...
```

9.12.2 Create a form group to group all the form controls

Under the controls paste the following:

```
  //Create a form group to eval the data at once
  controlsGroup = new FormGroup({
    id: this.id,
    description: this.description,
    area: this.area,
    geom: this.geom
  })
```

9.12.3 Create the controls in the template

Create the form and the controls and link them to the controls in the .ts:

```
<!--Form start-->
<form [formGroup]="controlsGroup">
    <p>Specify the id only in case of select, update or delete</p>
    <mat-form-field>
        <input matInput placeholder="id" [formControl]="id" name="id
            ">
    </mat-form-field>
    <br>
    <mat-form-field>
        <input matInput placeholder="Description" [formControl]="
            description" name="description">
    </mat-form-field>
    <br>
    <mat-form-field>
        <input matInput placeholder="Area" [formControl]="area" name
            ="area">
    </mat-form-field>
    <br>
    <mat-form-field>
        <input matInput placeholder="Geometry WKT" [formControl]="
            geom" name="geom">
    </mat-form-field>
</form>
```

9.12.4 Inject the ApiService

To send requests you need to inject the ApiService into the component. This is done declaring it in the constructor:

```
//Pay attention to::
//  - Services must be injected in the constructor
//  - Services are not imported in the component, in the imports
   array
constructor(private apiService:ApiService){}
```

9.12.5 Create the method to execute on click over the form buttons

Add a method in the .ts for each action you need:

```
insert(){}
delete(){}
...
```

9.12.6 Send the data and manage the server answer in the .subscribe method of the observable

Get the form data with this.controlsGroup.value, and send it to the API endpoint by using the ApiService:

```
insert(){
    this.apiService.post('buildings/buildings_view/insert/',this.
       controlsGroup.value).subscribe({
      next: (response: ServerAnswerModel) => {
        console.log('response',response)
        this.selectAll()
      },
      error:error=>{
        console.log(error.description)
      }
    })//subscribe
 }
```

next will be executed if no internal errors are in the server, that is the code is 200, 201, ... 210. **error** will be executed if the answer does not have a 200, 201, ... 210 code.

9.12.7 Use the ServerAnswerModel and the model for the table to cast the data

This is done in:

```
next: (response: ServerAnswerModel) => {
..
}
```

This is indicating the response is an instance of ServerAnswerModel, and put this instance into the arrow function (=>{ arrow function code}).

Thanks to this you can write: answer.ok, answer.message, answer.data, inside the arrow function.

9.12.8 Inform in the web page to the user the server answers

You can do this by creating an instance variable, in the .ts file:

```
this.serverMessage='';
```

showing this instance variable in the template:

```
<mat-card-content>{{serverMessage}}</mat-card-content>
```

Set the instance variable value on receiving a server answer, to inform the user:

```
this.serverMessage=response.message;
```

Te web will reflect any change on any instance variable, or control value, in real time. To delete the message in the web you only have to empty the instance variables in the .ts code:

```
this.serverMessage='';
this.listOfRows=[];
...
```

9.12.9 Create the buttons to execute the actions

In the .html, create buttons and set the (click) property to the method you ant to execute on click:

```
<button mat-button matTooltip="Insert" (click)="insert()">Insert
   </button>
<button mat-button matTooltip="Delete" (click)="deleteRow()">
   Delete</button>
<button mat-button matTooltip="Update" (click)="update()">Update
   </button>
<button mat-button matTooltip="Select" (click)="select()">Select
   </button>
<button mat-button matTooltip="Select all" (click)="selectAll()
   ">Select all</button>
<button mat-button matTooltip="Clear form" (click)="clearForm()
   ">Clear form</button>
```

9.12.10 Do not allow users to send the form if all controls are not valid

To do this you must disable the buttons if any of the validators has an error. This is done with:

```
<button mat-button matTooltip="Insert" [disabled]="!
   controlsGroup.valid" (click)="insert()">Insert</button>
...
```

The property disabled will be true if not all form controls have valid values according the validators.

9.13 What to do in case of error in the front-end

You are going to start coding in JavaScript. Apart of the console messages, you have many other ways of seen what is happening in case of error. You have to learn how to investigate the problem. **Do not read your code, read the reports Django, Angular or the console says**. Also you can stop the code and execute it step by step. This section is to show you how to know what is happening.

9.13.1 See the console messages

First step do not panic. You are not blind. The web browser, or Django, will tell you what is exactly happening. Open the developer tools, and go to the *Console* tab. You will see the error and the line where it happened (Figura 9.1).

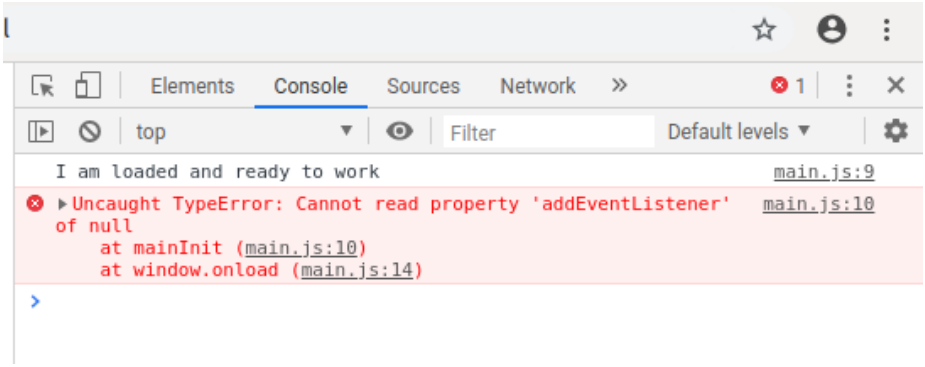

Figura 9.1: Console with the error message and the lines where the error was triggered

If you click over the first line number indication you will see the js code and the first line that triggered the error, Figura 9.2.

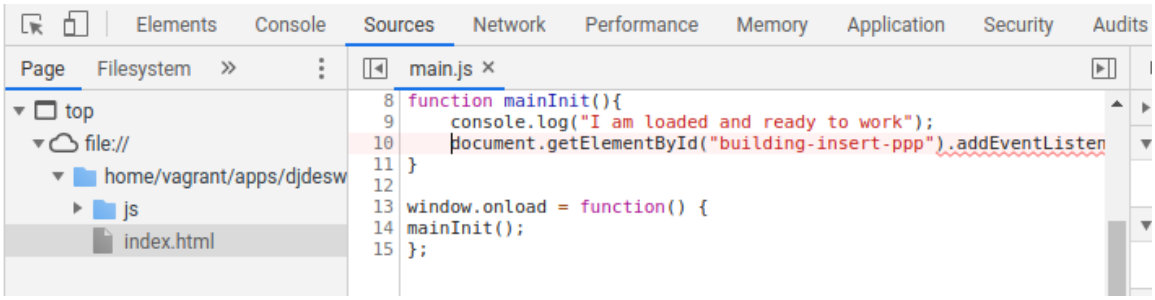

Figura 9.2: First line where the error was triggered

In this case the error *Cannot read property addEventListener of null* is a very common mistake. This means that the object before *addEventListener* is null. Like null objects do not have the method *addEventListener* the execution can not continue. In other words *document.getElementById(building-insert-ppp)* is null, what means there is not any html object with the id *building-insert-ppp*. The 80% of the times that you call the teacher is because this error. Fix the error typing the id of an existing element in your page.

9.13.2 Stop the JavaScript execution

You can also stop the execution whatever you want, execute the code line by line and see the local variable values. In the next listing we are going to make some calculations to show you how to stop the code and see the local variable values. Change the *buildingInsert* function code for the following:

```
function buildingInsert(){
    var a=5;
    var b=10;
    var c;
    c=a+b; //this will trigger an error
```

```
        console.log("The result is" + d)
        console.log("I promess. I will send the form data");
    }
```

If you reload the page you will see that the message *I am loaded and ready to work* appears in the console. So the page is properly loaded. But on click on the insert button, you will see the following error on the console window Figura 9.3.

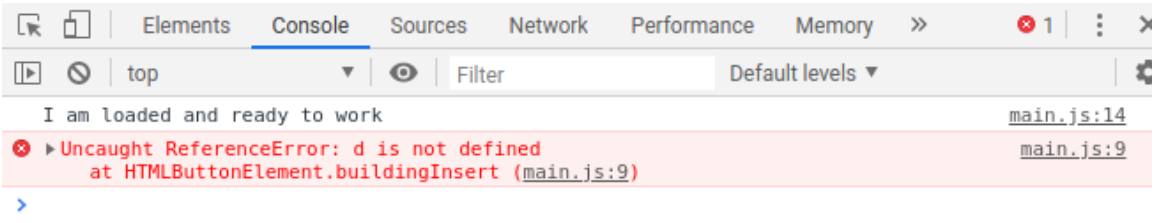

Figura 9.3: Reference error

The error is telling you that d is not defined. That means that there is not any d=something before the line 9 in the main.js file. Lets stop the execution a little bit before to see the local variables and run the code step by step. Click on the error line number (Figura 9.4).

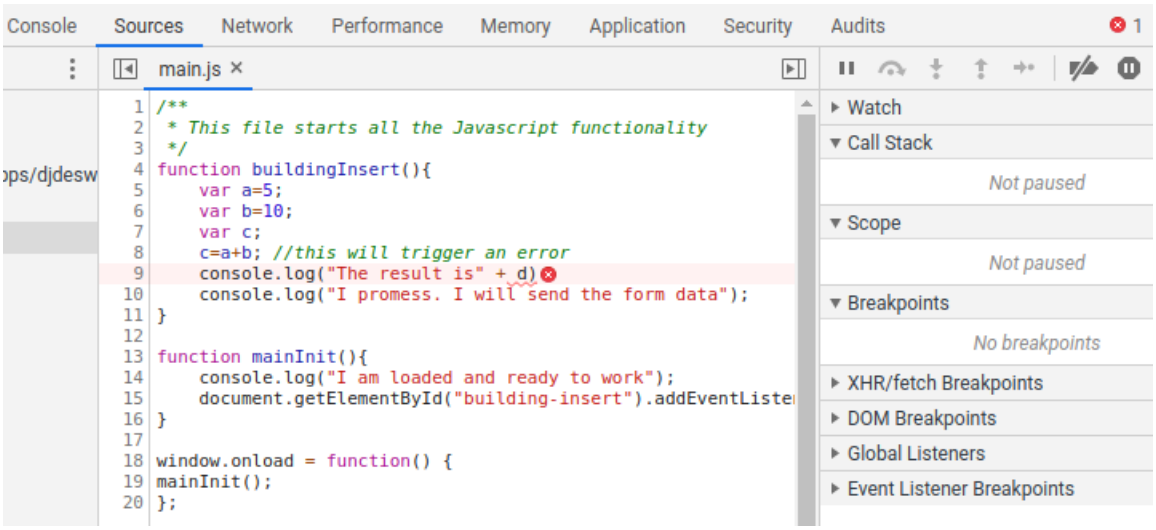

Figura 9.4: Reference error

Click on the left side of the code window some lines before the error line, but inside the the function who made the error, in the figure the fifth line.

Now you can click on the insert button to trigger the function again an to see how the execution is stopped. Yon can advance the execution by pressing one of the buttons in the control bar Figura 9.6. To execute line by line press the curved arrow.

After three clicks on the curved arrow you can see the local variable values in the *Local* section Figura 9.7.

Once solved the problem, remove the stop you must click on the same line number where you clicked to stop the execution.

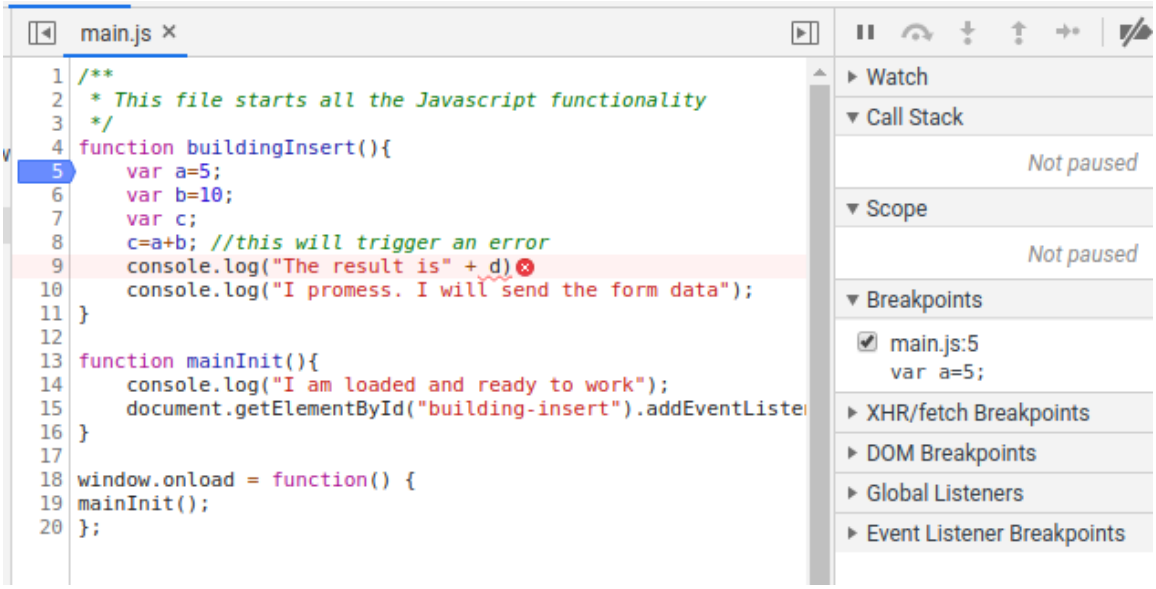

Figura 9.5: Stop the code execution in a line

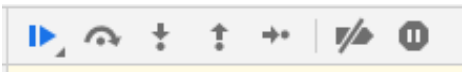

Figura 9.6: Buttons to advance the execution

9.13.3 Where I am sending the data

Now you are sending data to the server, you need to know how to check where are you sending the data, what are you sending and what is the server responding. All the answer to these questions are in the Web developer tools of Chrome.

If the page does not work as you expect, you must extract the Web developer tools of Chrome an to see the messages in the *Console* tab.

You also must see, in all moment, the messages of the backend in the console. If there is an error in the backend, you will see the error trace. If no error, you will see the code 200.

To know **where I am sending the data** you must open the tab *Network*, Figura 9.8. In the figure you can see three requests. The second request is in red. That means that there was an error.

To know where I sent the request click over the request and go to the *Headers* tab. In the above part you will see the *Request url*. In the Figura 9.9, *http://localhost:8000/building_insert/*. There is where you are sending the data. You also can see the method, in this case *POST*.

9.13.4 What I am sending to the server

Many times you will get an error because you are sending to the server something that the server is not expecting. You first have to know at which view of your API are you sending the data. After that what is your view is expecting, and later to check what are you sending. This last step is done, in Chrome developer tools, click into the *Network* tab, click into the request you want to check, and later into the *Headers* tab. At the bottom of that tab you have the *Request Payload* field, Figura 9.10. There you can see what are you sending. In the figure a json with two fields: *descripcion* and *geomWkt*.

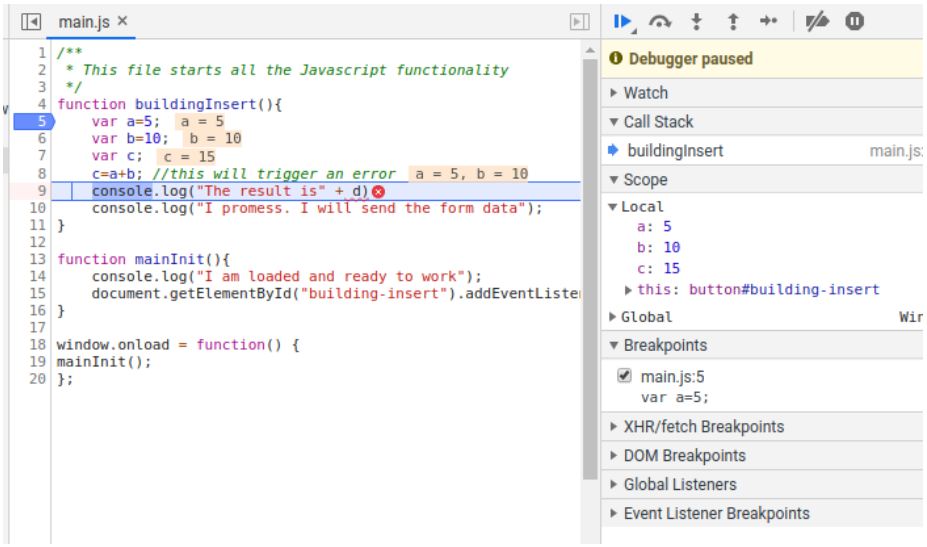

Figura 9.7: Local vars values

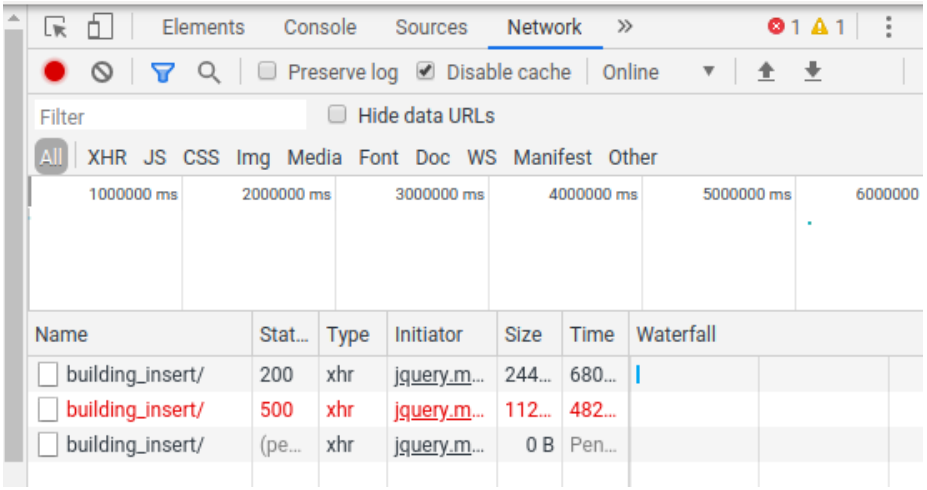

Figura 9.8: How to see the requests that my web does

9.13.5 What is the server responding

As your js code uses the server responses, in case of error, you must check if the server is answering what are you expecting.

In Chrome developer tools, click into the *Network* tab, click into the request you want to check, and later into the *Preview* or the *Response* tab.

In this case, the problem is that, for whatever reason the server is not running. You have to run python manage.py runserver again. Remember, to run your Django application you have first to activate the Pyhton virtual environment.

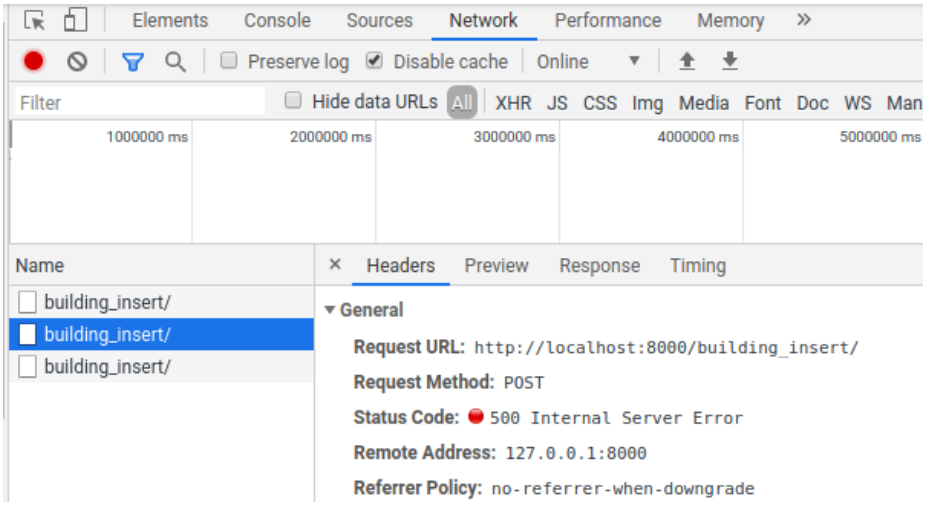

Figura 9.9: How to see the where I am sending the data

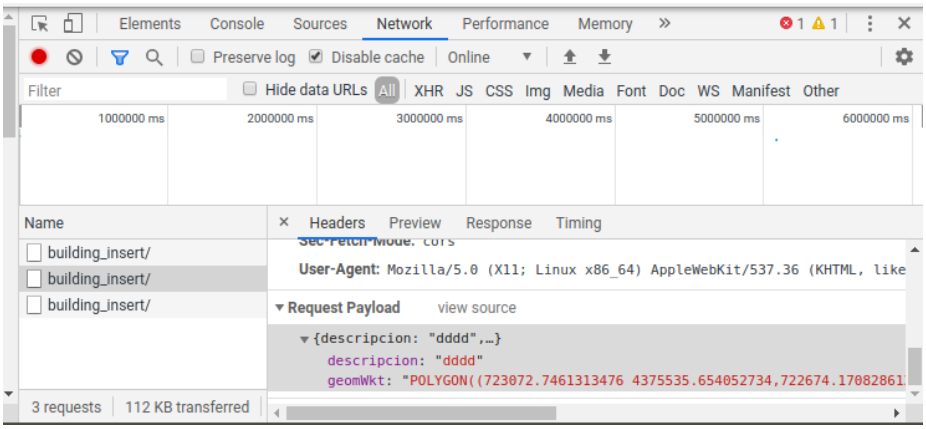

Figura 9.10: How to see what I am sending to the server

9.14 Evaluation 2

9.14.1 Part 1. Theorical written exam. Value 2 points

In this test you can be asked about:

- About Django: urls, views, and models.
- Angular: models, services, components, reactive forms, and making requests catching the answers.
- To link click events on buttons with component methods.
- To change the value of form controls.
- To read the value of form controls.

For example you can be asked for create a web page with a form and, in the ts code you will have to calculate something, getting the information from the form and putting the result in a paragraph or an other control. Also you can be asked to send information to an API and manage the answer, by changing something in the web, showing the server message, for example.

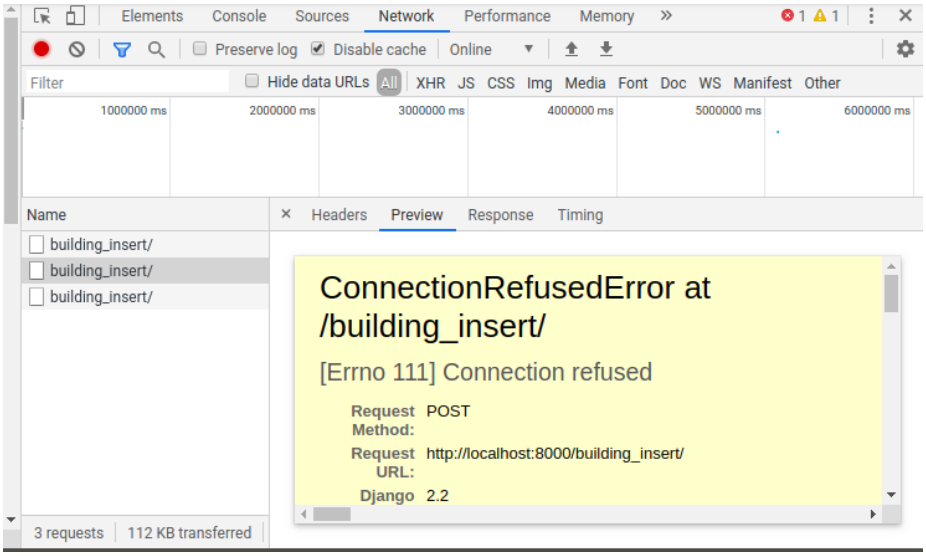

Figura 9.11: How to see the server response

9.14.2 Part 2. Practical project. Create a web page to update the tables of your database. Value 1 point

The goal of this exercise is to create a web page to insert rows in the tables of your database, with Angular, and using your Django API. You have to manage at least three tables. All operations are mandarory in all tables: insert, selectone, selectall, update and delete. You must build tree forms with all the form control to manage the fields of your tables, create 5 buttons, with the 5 operations, and make them to work. On selectone, you must show the field values into the fom controls. Perform the following steps:

- Create the 5 operations for each of your tables in the API insert, selectone, selectall, update and delete: create the models, views, and the urls. Everything must be in a new app created by you.
- Clone the angular-template-result repo, and rename the angular-template-result folder according your project.
- Create three components to manage your three forms.
- Create in each component a reactive form, according your table fields.
- Add the components to the app.router.ts file.
- In the menu component, create three new buttons, and link the buttons to the routes in app.routes.ts, with the routerLink property, to show in the router outlet your forms, on clicking on the menu buttons.
- IDs of html form controls must match with the field names of the corresponding database tables.
- Create 5 buttons to perform the 5 operations, link each (click) event to a method in the component to perform the request.
- Create one model to manage a row of each table.
- Use the ApiService service to send the request to the server and manage the answer. Catch the answer into the ServerAnswer model and, the data property, into your table model.

- On deleting a record, you must empty the form data, that is to empty form text boxes.
- You must give a message, changing a paragraph content, to alert the user that all was ok or not. If all was ok you must to show the gid of the new row inserted, updated, or deleted. Each form must have its message paragraph.
- Everything must run in local: your API and your front-end.
- Every form must have a Clean button to empty the form.

Delivery:

- Upload the complete projects, front-end and back-end, compressed as exercise2.zip, to the subject shared space in Poliformat. Please only upload obly source code. **Avoid to upload the node_modules folder**.
- Upload a file called groupMembers.txt, where you specify name and surname of all the members of the group.
- All the members have to upload the same two files: groupMembers.txt and exercise2.zip.
- You will show the project to the teacher. You have to have prepared a demonstration.
- You will have to answer correctly the teacher questions about the code to obtain the whole note of the exercise.

General evalutation criteria: Every situation is different, and many students do not do some exercise parts, but do altenative and valuable functionalities: css, multiple pages, extra functionalities. Never the less here you have hoy I start evaliating the exercise:

- Complete Django backend: 0.4.
- Front-end: each table with all functionalities 0.2. Three tables 0.6.
- Not using Docker to API developmen: -0.1. This feature adds a layer of complexity.
- Not showing the database data into the form controls: -0.1.
- Not showing server messages into a paragraph: -0.1.

9.15 User authentication (optional)

9.15.1 API endpoints

To authenticate users, you need the following endpoints: login (section 7.7.2, page 92), logout (section 7.7.3, page 93), isloggedin (section 7.7.4, page 93). They are available in the urls: core/login/, core/logout/, core/isloggedin/.

9.15.2 The auth-user.service

You need a service to store the current username, if the user is loggedin (true/false). Also in the service we are going to have a method (isloggedin) to check were or not the user is already authenticated.

To create the component, get into the angular container and type:

```
import { Injectable } from '@angular/core';
import { ApiService } from './api.service';
import { ServerAnswerModel } from '../models/server-answer.model';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  public username: string = '';
  public isAuthenticated: boolean = false;
  public userGroups: string[] = [];
  constructor(public apiService: ApiService) {
    this.checkIsLoggedInInServer();
  }
  checkIsLoggedInInServer() {
      this.apiService.post('core/isloggedin/', {}).subscribe({
              next: (response: ServerAnswerModel) => {
                if (response.ok){
                  this.username = response.data[0]['username'];
                  this.isAuthenticated = true;
                }
              },
              error: (error:any)=>{
                console.log(error.description)
              }
            })//subscribe
  }
}
```

9.15.3 React in the menu component to the auth-user.service changes

We can show a different icon in the menu component depending on if the user has been authenticated or not. To know this you need the service auth-user to be injected in this component. Use the constructor for this:

```
...

@Component({
  selector: 'app-menu',
  standalone: true,
  imports: [MatButtonModule, RouterLink, MatIconModule, MatTooltip],
  templateUrl: './menu.component.html',
  styleUrl: './menu.component.scss'
})
export class MenuComponent {
  constructor(public authService: AuthService) { } // Inject the
      service to use it in the template

}
```

If instead of public authService, you use private authService, in the constructor, you could not use the service in the component template.

Now you can use the Angular template directives to show one icon or an other in template:

```
@if(authService.isAuthenticated){
    <span>{{authService.username}}</span>
    <button mat-button matTooltip="Close session" routerLink='logout
        -form'><mat-icon style="transform:rotate(180deg);">logout</
        mat-icon></button>
} @else {
    <span>Not logged in</span>
    <button mat-button matTooltip="Init session" routerLink='login-
        form'><mat-icon routerLink='login-form'>login</mat-icon></
        button>
}
```

I use span label because it is an inline html object.

9.15.4 Automatically check if the user is authenticated

On sending a POST to the sever, despite not to send anything, the sessionid is sent to the server in the request headers. This is received by the core/isloggedin/ endpoint and it checks if the user has initialized already a session or not, returning the username, or an empty list in the data section of the server answer. We can then save in the service the result of the request.

To automatically send the request the isloggedin method must be executed in the service constructor. Due to services are executed only once, the constructor is executed only once, therefore the isloggedin method is also executed only once.

To ensure the auth-service service is executed you must put in in a component that is loaded always and only once. Three components accomplish this condition app (the main component), menu, header and footer. I put it in the menu component. We saw this in the previous section, because the service is used in it, but I usually initialize this kind of services in app component.

9.15.5 Login form

Now you have to create a login form to allow users to authenticate. This, if OK, will create the sessionid cookie.

Create the component:

```
ng g c components/forms/login
```

Puth the following in the .ts:

```
import { Component } from '@angular/core';

//To use forms
//  Import in the imports on the component the following
import { ReactiveFormsModule } from '@angular/forms';
import {MatInputModule} from "@angular/material/input";//angular
   material must be installed before
import { MatTooltip } from '@angular/material/tooltip';
import {MatCardModule} from '@angular/material/card';
import { CommonModule } from '@angular/common';

//To use the controls in the component
//  Import in the imports on the component the following
import {FormControl} from '@angular/forms';
```

```
import {FormGroup, Validators} from '@angular/forms';
import { ServerAnswerModel } from '../../../models/server-answer.
   model';
import { ApiService } from '../../../services/api.service';
import { MatButtonModule } from '@angular/material/button';
import { AuthService } from '../../../services/auth.service';


@Component({
  selector: 'app-login.form',
  standalone: true,
  imports: [MatInputModule, ReactiveFormsModule, MatTooltip,
     MatButtonModule, CommonModule],
  templateUrl: './login-form.component.html',
  styleUrl: './login-form.component.scss'
})
export class LoginFormComponent {
  serverMessage = '';
  //Form component creation
  username = new FormControl('', [Validators.required,Validators.
     minLength(4)]);
  password =  new FormControl('', [Validators.required,Validators.
     minLength(4)]);

  //Create a form group to eval the data at once
  controlsGroup = new FormGroup({
    username: this.username,
    password: this.password,
  })

  //Pay attention to::
  //  - Services must be injected in the constructor
  //  - Services are not imported in the component, in the imports
     array
  constructor(private apiService:ApiService, private authService:
     AuthService){}

  login(){
    this.serverMessage='';

    this.apiService.post('core/login/', this.controlsGroup.value).
       subscribe({
         next: (response: ServerAnswerModel) => {
           if (response.ok){
             this.authService.username = this.username.value!; //!
                is a non-null assertion operator
             this.authService.isAuthenticated = true;
           }
           this.serverMessage=response.message;
         },
         error: (error:any)=>{
           console.log(error.description)
         }
       })//subscribe
```

```
  }
}
```

As you can see I inject the auth-service on this component, in order to update the isloggedin, and username properties of the service.

Puth the following in the template:

```
<p>Login form</p>

<!--Form start-->
<form [formGroup]="controlsGroup">
    <mat-form-field>
        <input matInput placeholder="username" [formControl]="
           username" name="username" matTooltip="Type your username
           ">
        <mat-error *ngIf="username.hasError('required')">
            The username is mandatory.
        </mat-error>
        <mat-error *ngIf="username.hasError('minlength')">
            Type more characteres.
        </mat-error>
    </mat-form-field>
    <br>
    <mat-form-field>
        <input matInput placeholder="password" [formControl]="
           password" name="password" matTooltip="Type your password
           ">
        <mat-error *ngIf="password.hasError('required')">
            The username is mandatory.
        </mat-error>
        <mat-error *ngIf="password.hasError('minlength')">
            Type more characteres.
        </mat-error>
    </mat-form-field>
    <br>

    <!-- -->
    <br>
    <button mat-raised-button matTooltip="Login"
         [disabled]="!controlsGroup.valid" (click)="login()">Login</
            button>
    <p>{{serverMessage}}</p>
</form>
```

9.15.6 Logout form

Now you have to create a logout form to allow users to close the session. This will also remove the sessionid cookie.

Create the component:

```
ng g c components/forms/logout
```

Puth the following in the .ts:

```
import { Component } from '@angular/core';
//To use forms
```

```
//  Import in the imports on the component the following

import { MatButtonModule } from '@angular/material/button';

//To use the controls in the component
//  Import in the imports on the component the following
import { ServerAnswerModel } from '../../../models/server-answer.
   model';
import { ApiService } from '../../../services/api.service';
import { AuthService } from '../../../services/auth.service';

@Component({
  selector: 'app-logout-form',
  standalone: true,
  imports: [MatButtonModule],
  templateUrl: './logout-form.component.html',
  styleUrl: './logout-form.component.scss'
})
export class LogoutFormComponent {
  serverMessage = '';
  constructor(private apiService:ApiService, private authService:
     AuthService){}
  logout(){
    this.apiService.post('core/logout/', {}).subscribe({
          next: (response: ServerAnswerModel) => {
            if (response.ok){
              this.authService.username = '';
              this.authService.isAuthenticated = false;
            }
            this.serverMessage=response.message;
          },
          error: (error:any)=>{
            console.log(error.description)
            this
          }
        })//subscribe
  }

}
```

As you can see I inject the auth-service on this component, in order to update the isloggedin, and username properties of the service, and set them to blank.

Puth the following in the template:

```
<h1>Logout form</h1>

<p>Press the button to logout</p>
<button mat-raised-button matTooltip="Logout" (click)="logout()">
   Logout</button>
<br>
<p>{{serverMessage}}</p>
```

CAPÍTULO 10

Making a geoportal with Angular

10.1 Introduction

In this chapter you are going to learn how to make a geoportal with Angular

10.2 Goals in this chapter

Making a geoportal that:

- Create a map with Open Layers (https://openlayers.org/en/latest/examples/simple.html).
- Set the coordinate system of the map (25830: UTM Huso 30, Datum ETRS89. It is the official SRC in Spain) (https://openlayers.org/en/latest/examples/projection-and-scale.html).
- Load external WMS layers as PNOA an Cadastre (Spanish WMS service) (https://openlayers.org/en/latest/exampl tiled.html).
- Load the WMS layer of the buildings layer. The WMS service of the buildings layer must be published with the Docker GeoServer service. It shows the current content in the PostGis database of the buildings table (https://docs.geoserver.org/main/en/user/gettingstarted/postgis-quickstart/index.html).
- Create an empty vector layer to draw new buildings in the geoportal (https://openlayers.org/en/latest/examples/vec layer.html).
- Create a draw interaction to draw in the vector layer (https://openlayers.org/en/latest/examples/draw-features.html).
- Link a function to, on finishing the building draw, to execute a function to send the coordinates of the polygon to the Buildings form. This is the event *drawend*.
- To be able to add - remove, and enable - disable, the draw interaction.
- From the buildings form complete the form and send the new building to the database.
- On return to the map, to see the new building in the building WMS layer, which means it has been inserted.

Other important utilities as:

- Load WKT data (https://openlayers.org/en/latest/examples/wkt.html).
- Load geojson data (https://openlayers.org/en/latest/examples/geojson.html).
- Select interaction (https://openlayers.org/en/latest/examples/select-features.html).
- Modify interaction (https://openlayers.org/en/latest/examples/modify-features.html)
- Snap interaction (https://openlayers.org/en/latest/examples/snap.html).

are not going to be explained because lack of time, but you will have enough base to use them in your projects. You only have to know one thing: only one interaction active at the map. On enabling one interaction, you must disable the rest of interactions.

10.3 Install OpenLayers package

The Angular template repos already come with this dependency installed, but to install ol:

- Get into an Angular container terminal.
- Write npm install ol.

This will install the latest OpenLayers version, and will add the version to the packages.json file of the project.

Important to understand the following: The OpenLayers was installed on my computer, so the OpenLayers dependecy was added in the packages.json file in my computer. When you downloaded the repo, and on creating the Node image, OpenLayers is installed in it, as the npm install command is automatically executed on the image creation, and it install all dependencies in the file packages.json.

10.4 Infrastructure explanation to create the map

What I have tried is to separate responsibilities Figura 10.1, in order to separate the code, and the code be maintainable. The main functionality is in the service MapService, en charged of create:

- The external WMS layers.
- The Vector layer to draw.
- The WMS building layer.
- Create the map.
- Add the previous layers.
- The map is unlinked to any div.

On injecting this service in a component, first time it is going to execute its constructor, where all previous tasks are going to be done. The service is going to be alive wile the page is alive, therefore no more times initialized. So the map object is always the same, which is good because the state of the map is automatically saved in the map property of the service.

Two components inject this service in the component in its constructor: MapComponent and DrawBuildingComponent. The latter is child of the former. Child components are initialized before father components, so DrawBuildingComponent is going to initialize the MapService.

Once initialized the MapService it is allways alive, not like the MapComponent and the DrawBuildingComponent. They are going to be created and destroyed every time the user navigates to the Map section, or abandon the Map section. So the components modify the MapService on its creation - destruction, the events NgAfterViewInit and OnDestroy, that are automatically executed when the template has been rendered, and the component has been destroyed, respectively.

The OpenLayers map has to be linked to a html div element. This element is created - destroyed every time the component is created - destroyed. New divs in new components are different from previous divs, so it is necessary to unlink destroyed divs and link new divs to the map. This is done in the NgAfterViewInit and OnDestroy events, also called component hooks. This is the only responsibility of the MapComponent.

The DrawBuindingComponent is a child of the MapComponent. It is encharged of modify the map property of the MapService by adding the Draw interaction in the event NgAfterViewInit, but could have

been in the event NgOnInit, executed a bit before, in fact this would have been more appropriated, as we do not need the template to be rendered to modify the service. Once create the draw interaction, the template has two buttons with images. The buttons are showed alternatively depending on the state of the instance variable *drawMode*. These buttons click are linked to component methods that enable - disable the interaction, and change the state of the *drawMode* variable.

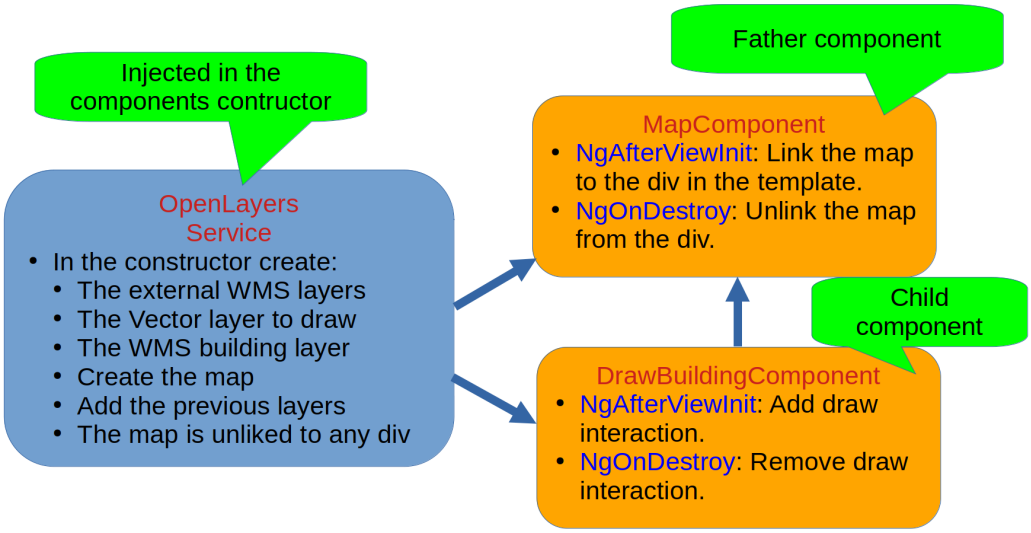

Figura 10.1: Responsibilities division

The good of this structure is that you can add more interactions as child of MapComponent, and manage them separately, modifying the MapService. **The drawback is** that only one interaction must be enabled at the same time. Fortunately I have added to the MapService a function to disable all interactions except MouseWheelZoom and DragPan (the default ones).

10.5 MapService

Here is where the map is. The rest of the components modify the map by injecting this service in them. The constructor is executed once as it is only initialized once. The constructor has the tasks to create the map divided in several methods:

```
constructor(public settingsService: SettingsService) {
    this.baseLayersGroup= this.createBaseLayers();
    this.myLayersGroup= this.createMyLayers();
    this.map= this.createMap();//Create the map and store it in
       the mapService
    this.addLayerSwitcherControl();
    this.addMousePositionControl();
}
```

I am not going to copy the code here. Please have a look to the repo.

An remarkable method is the following function that disables all interactions except MouseWheel-Zoom and DragPan (the default ones). This is necessary to stop drawing in a layer and start drawing in an other. Before to start drawing in any layer you must call this method:

```
disableMapInteractions(): void {
  if (this.map) {
```

```
      this.map.getInteractions().forEach((interaction: Interaction)
          => {
        // Comprueba si la interaccion NO es una instancia de
            MouseWheelZoom o DragPan
        if (!(interaction instanceof MouseWheelZoom) && !(
            interaction instanceof DragPan)) {
          interaction.setActive(false);
        }
      });
    }
  }
```

10.6 MapComponent

This component is the holder of the map. It is en charged of link the div to the map on component creation and unlink it on component destruction. Have a look to the repo.

10.7 DrawBuildingComponent

This component is a child of MapComponent. It is en charged of initialize the map, create the Draw interaction, and enable - disable the Draw interaction. The Draw interaction is destroyed on component destruction, and created on component creation.

10.7.1 Add Draw interaction

This deserve an explanation. I you have a look to the method:

```
addDrawBuildingInteraction() {
  //Add the draw interaction when the component is initialized
  var sourceBuildings: VectorSource = this.mapService.
     getLayerByTitle('Buildings vector')?.getSource();
  if(sourceBuildings){
      this.drawBuilding = new Draw({
       source: sourceBuildings, //source of the layer where the
           poligons will be drawn
      type: ('Polygon') //geometry type
    });
    this.drawBuilding.on('drawend', this.manageDrawEnd);

      //adds the interaction to the map. This must be done only
         once
    this.mapService.map!.addInteraction(this.drawBuilding);
  }else{
    console.error("Error: Buildings layer not found");
  }
}
```

It gets the vector layer for the map. Add the draw interaction to the layer, and links the draw end event to a class method:

```
      this.drawBuilding.on('drawend', this.manageDrawEnd);
```

On finishing to draw a polygon, an event *drawend* is raised, and the *manageDrawEnd* method will be called, receiving an event object, where all the information about the event is:

```
manageDrawEnd = (e: DrawEvent) => {
  var feature = e.feature;//this is the feature that fired the
     event
  var wktFormat = new WKT();//an object to get the WKT format of
     the geometry
  var wktRepresentation  = wktFormat.writeGeometry(feature.
     getGeometry()!);//geomertry in wkt
  console.log(wktRepresentation);//logs a message
  this.router.navigate(['/building-form'], { queryParams: {geom:
     wktRepresentation }});

}
```

The manageDrawEnd is not a normal method it is an arrow function. If you do not use an arrow function the key word *this* is not referring to the DrawBuildingComponent class but to the manageDrawEnd fuction, so this.router is not going to exist, because this is manageDrawEnd, and router is not a property from manageDrawEnd but DrawBuildingComponent. Use always this syntax to link events to functions.

```
  this.router.navigate(['/building-form'], { queryParams: {geom:
     wktRepresentation }});
```

The router property is also a service, injected in the component constructor. It allows to navigate to other component. It also allows to pass data to other components trough the path, in the *queryParams* component. In this case we are passing the *geom* field, with the value of the WKT representation of the polygon just drown.

So on draw end, the building form component is showed, and I have made a modification to get the geom field value:

```
//components/forms/building-form.component.ts
 constructor(private apiService:ApiService, private activatedRoute:
     ActivatedRoute,
   public router: Router
 ){}

 ngOnInit(): void {
   this.activatedRoute.queryParamMap.subscribe(params => {
     var geom = params.get("geom");
     if (geom){
       this.geom.setValue(geom);
       this.geomInUrl=true
     }
   });
 }
```

In the above code, the ActivatedRoute is injected in the component. This allows to get the variable in the path. To do this, I coded the ngOnInit hook, which is executed after the constructor, and use the this.activatedRoute.queryParamMap.subscribe method to access to the WKT representation of the geometry.

If the *geom* field in the url is not null I set it in the geom form control.

This is automatically done on draw a polygon, but what if you want to edit the geometry. In this case you can draw a polygon, the geom will be put in the url and the buildings form showed. You can then select an other polygon and press the button *Use geom in url*. This button executes the following method thar gets the geom from the url, and puts it into the geom form control:

```
  useGeomInUrl(){
      this.activatedRoute.queryParamMap.subscribe(params => {
        this.geom.setValue(params.get("geom"));
    });
  }
```

10.8 Inter communication between components by using events (optional)

This guide demonstrates how to achieve robust inter-component communication in Angular 18 using a shared service and RxJS Subjects. This approach is highly recommended for decoupled communication between components.

You can find an example of the explaining in this section in the angular-template-solution. If you click on the draw building button and latter in the flowers draw button in the map, you will see the icons change. One action in one component alerts the other component to change, and vice-versa.

10.8.1 Create a model to send data between components

Create the file models/event.model.ts:

```
export class EventModel {
    type: string='';
    data: any;
  }
```

10.8.2 Create a Communication Service (events.service.ts)

This service acts as a central hub for emitting and subscribing to events.

```
import { Injectable } from '@angular/core';
import { Subject, Observable } from 'rxjs';
import { EventModel } from '../models/event.model';

@Injectable({
  providedIn: 'root'
})
export class EventsService {
  // Subject to emit events
  private eventSource = new Subject<EventModel>();

  // Observable to listen to events
  public eventActivated$: Observable<EventModel> = this.eventSource.
      asObservable();

  constructor() { }

  // Method to emit events and pass data to subscribers
  emitEvent(event: EventModel): void {
    this.eventSource.next(event);
  }
}
```

10.8.3 Emit an event

If you want to alert other components of something, and even to pass data to them, you have to inject this service to the component and use the emitEvent method. For example:

```
constructor(public eventsService: EventsService) {}

alertToOthers(){
   this.eventsService.emitEvent({ type: 'drawBuildingActivated',
      data: {'field':'value'} });
}
```

10.8.4 Subscribe to an event

To react to the events emitted by other components, you must subscribe to the EventService eventActivated$ property:

```
constructor(public eventsService: EventsService) {
  this.eventsService.eventActivated$.subscribe((event: EventModel)
      => {
    console.log("Event received:", event.type);
    if (event.type != 'drawBuildingActivated') {
      this.drawMode = false; // Reset draw mode if a different
         event is received
      this.disableDrawBuildings(); // Disable drawing if not in
         draw mode
    }
    // Handle the event as needed
  });
}
```

10.9 Evaluation 3

10.9.1 Part 1. Theorical written exam. Value 2 points

In this test you can be asked about anything explained in the theoretical classes.

10.9.2 Part 2. Practical project. Create a geoportal with Angular and OpenLayers. Value 2 points.

In this practical exercise you have to add a map to you webpage. Currently you must be able to list and add geometries by typing the coordinates in the forms. Here you must manage to draw in your three layers in your geoportal and automatically, on *drawend*, show the appropriate form with the coordinates already set in the geom field. You have to manage at least three tables.

Perform the following steps (2 points):

- Add two components to allow users to login - logout.
- On login the username must appear at the right of the navigation menu.
- On logout the username must disappear at the right of the navigation menu.
- On get into the web, automatically send a request to the core/isLoggedIn/ endpoint to know if the user was already authenticated, and put its name at the right of the navigation menu.

- Do not allow unauthenticated users to use your views. To do this you have to use LoginRequiredMixin.
- Publish in the Docker GeoServer service the WMS services for your three layers
- Change the default symbology of the layer and label the layer with the id field (https://docs.geoserver.org/main/en/user/styling/sld/cookbook/polygons.html).
- Add to the map, in the MapService the tree WMS services created before.
- Add to the map three vector layers, each one from one service.
- Add tree new components to manage the draw interactions. One component for each draw interaction.
- Each component to draw have to be a button to manage the enable - disable the Draw interaction of one vector layer.
- Add the components to draw as a child components of the MapComponent, and put them on the upper-left of the map, by using the CSSS of the component.
- Create the draw interaction on the NgOnInit hook of the component. Disable here all previous interactions on the map.
- On clicking in the button of the component once enable the interaction and change the icon button. On clicking again disable the interaction and change the icon button.
- On destroying the each draw components, remove the Draw interactions.
- On finishing the draw of a geometry, show the related form with the geom control filled in WKT. Use the Router, and the RouterLink classes to achieve this.
- You must be able to draw in each of the three layers, without other interactions to disturb.

Delivery:

- Upload the complete project project, compressed as *exercise3.zip*, to the subject shared space in Poliformat. Please only upload obly source code. **Avoid to upload the *node_modules* folder**.
- Upload a file called *groupMembers.txt*, where you specify name and surname of all the members of the group.
- All the members have to upload the same two files: *groupMembers.txt* and *exercise2.zip*.
- You will show the project to the teacher. You have to have prepared a demonstration.
- You will have to answer correctly the teacher questions about the code to obtain the whole note of the exercise.

You can get the remaining 2 points by developing the following functionalities:

- Manage communicate the draw components by using the EventService and subscriptions (0.5 points).
- Use the selectall endpoint to get all the geometries in WKT, and draw them in the map in a vector layer (0.5 pts).

- Manage the select interaction to show the appropriate form of the geometry with all fields ready to edit. To do this you need the geometries of the layer in vector format, that is to have done the previous optional step. You need a new button on the map to enable - disable the select interaction. On select a geometry show the form with all fields filled (0.5).
- Manage the edit interaction for a layer. You need a new button on the map to enable - disable the edit interaction. On click a geometry you will be able to graphically edit the vertex. Once finished click on the select interaction, select the edited geometry and will appear the form to send the new geometry to the database with the Update button (0.5).

I will sum points to your current subject score if you perform any of the following optional tasks:

- Add the Snap interaction on edit to all layers (0.5 points).
- Make the web responsive. The menu bar must disappear when the screen is too small, and a hamburger button must appear. On click in the hamburger menu, a left sidebar with the menu must appear (0.5 points).
- On finish editing a geometry, the geometry form appears with all the field values and the new geometry coordinates, ready to edit and send to the database (0.5 points).

Bibliografía

[1] Jose Carlos Martínez Llario, *PostGIS 2. Análisis Espacial Avanzado*, 2012.

[2] Adrian Holovaty, Jakob Kaplan-Moss, *La guía definitiva de Django*, 2009.

[3] Joseph W. Lowerly, Mark Fletcher, *HTML 5 para desarrolladores*, 2011.

[4] Juan Diego Gauchat, *El gran libro de HTML5, CSS y JavaScript, 2012*

Páginas web.

[5] Página de referencia HTML5: http://www.w3schools.com/html

[6] Página de referencia de CSS3: http://www.w3schools.com/css/default.asp

[7] Tutorial: http://www.tutosytips.com/aprende-html5-desde-0

[8] Referencia HTML 5 para desarrolladores. http://www.html-5.com

[9] Compatibilidad de HTML 5 con dispositivos móviles: http://mobilehtml5.org

[10] Tipos de controles input: http://www.w3schools.com/html/html_form_input_types.asp

[11] Control canvas http://www.html5canvastutorials.com/advanced/html5-canvas-mouse-coordinates/

[12] Open Geospatial Consortium, OGC. www.opengeospatial.org

[13] PostgreSQL. http://www.postgresql.org

[14] PostGis. http://postgis.refractions.net. Extensión que da soporte de datos espaciales a la base de datos

[15] Descripción del lenguaje SQL http://www.postgresql.org/docs/9.1/static/tutorialsql.html

[16] Descripción del lenguaje PL/SQL http://www.postgresql.org/docs/9.1/static/plpgsql.html

[17] Quantum Gis, Qgis. http://www.qgis.org

[18] Python. http://www.python.org

[19] PyQt4. http://www.riverbankcomputing.co.uk/software/pyqt/intro

[20] PIL. http://www.pythonware.com/products/pil. Biblioteca Python el trabajo con imágenes

[21] psycopg2. http://pypi.python.org/pypi/psycopg2. Biblioteca Python para la conexión con PostgreSQL

[22] Eclipse. http://www.eclipse.org. Entorno de desarrollo (IDE)

[23] PyDev http://pydev.org. Plugin para Eclipse para el desarrollo en el lenguaje Python